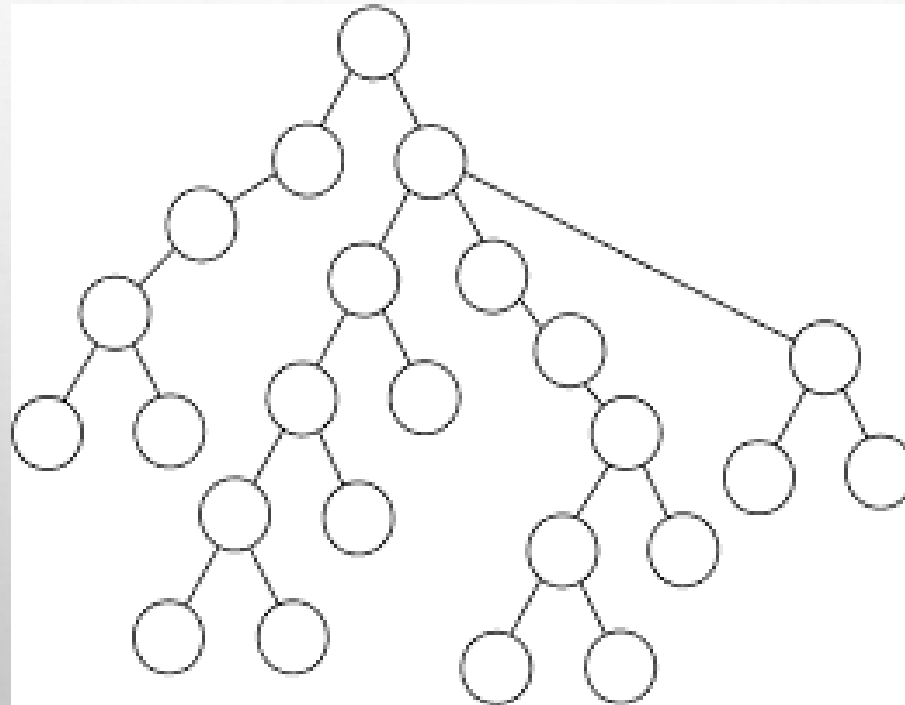


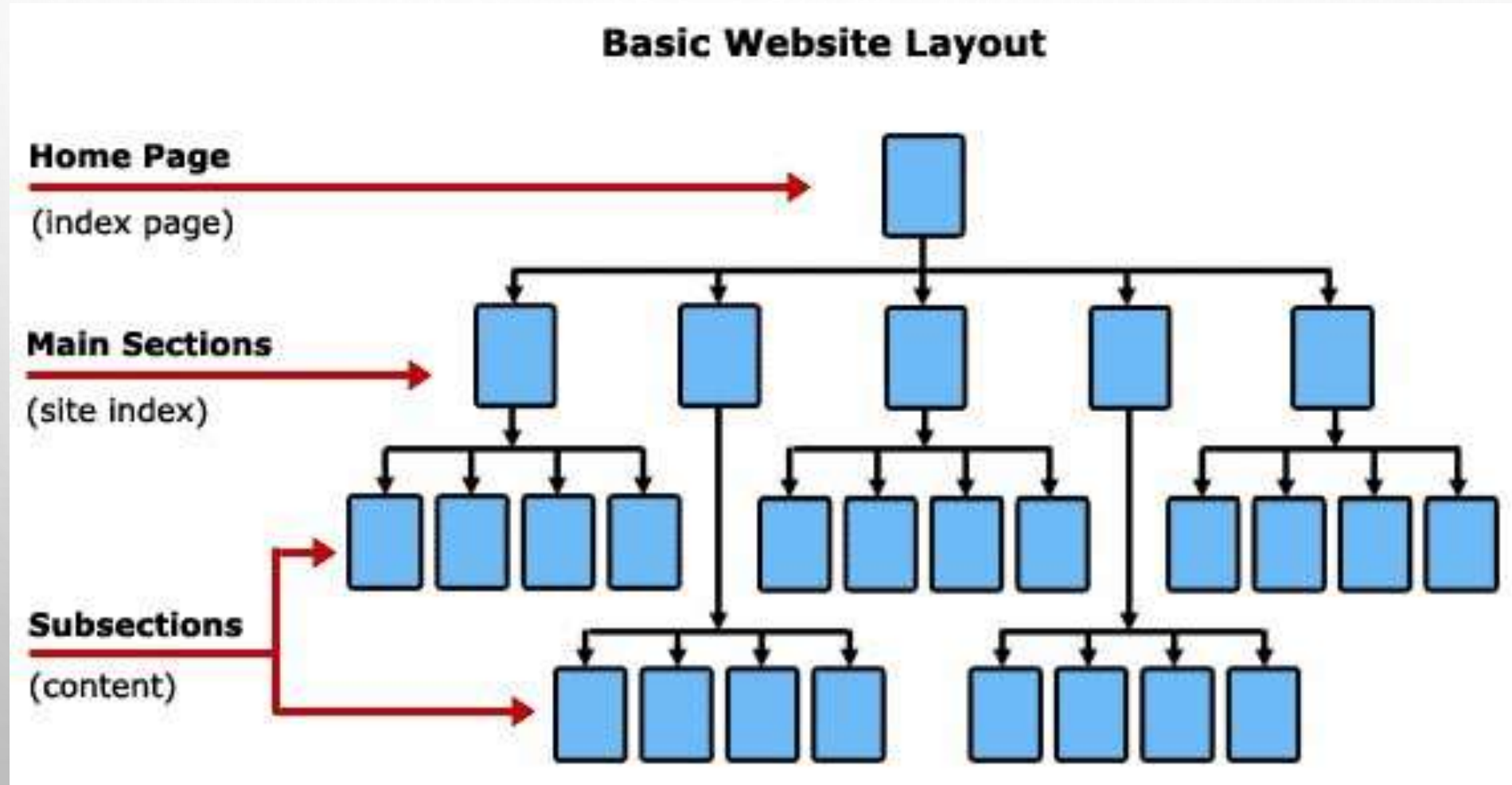
NON-LINEAR DATA STRUCTURES

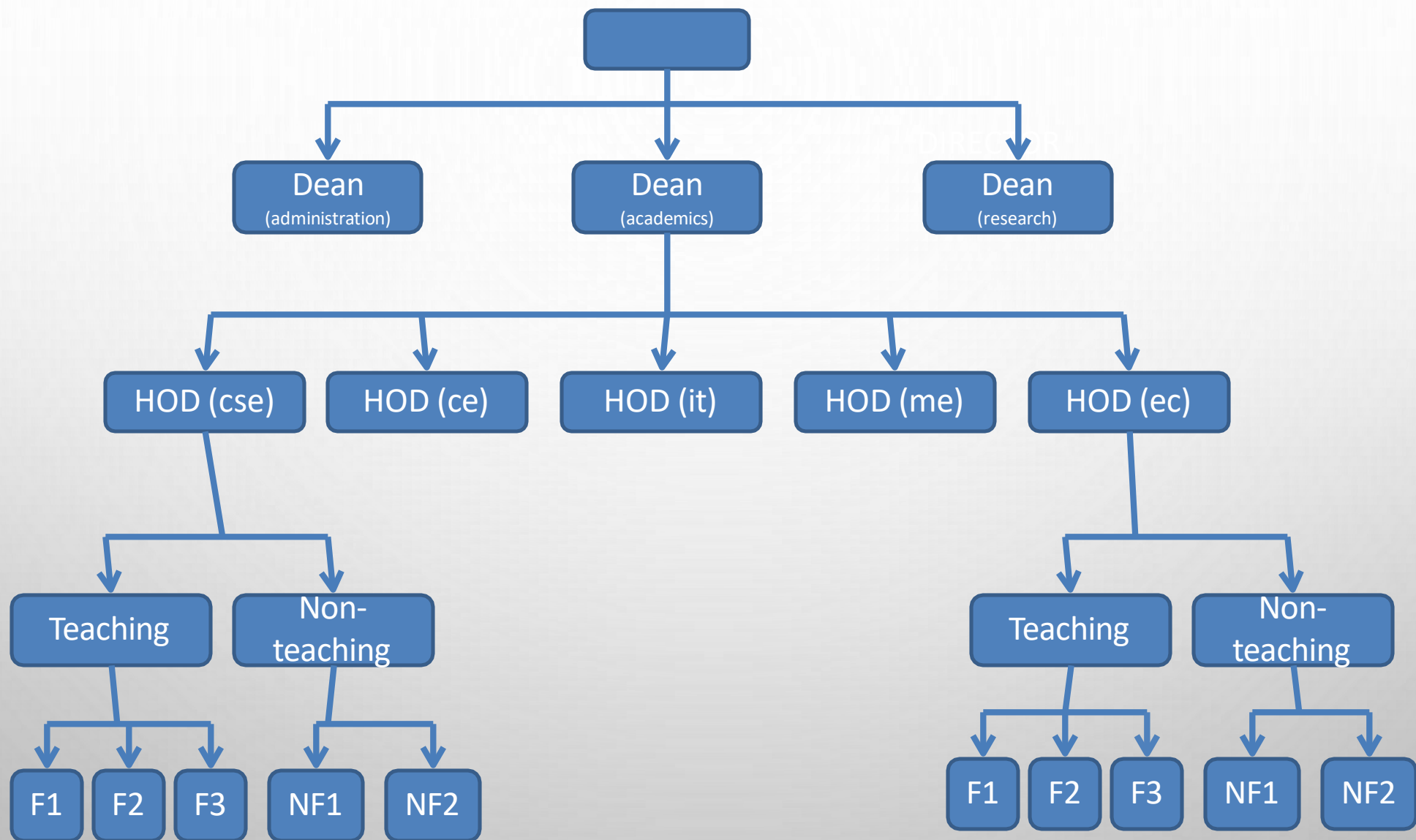
- TREE IS ANOTHER VERY USEFUL DATA STRUCTURE IN WHICH ELEMENTS ARE APPEARING IN A **NON-LINEAR FASHION**, WHICH REQUIRES A TWO DIMENSIONAL REPRESENTATION.



APPLICATIONS/REAL TIME EXAMPLES

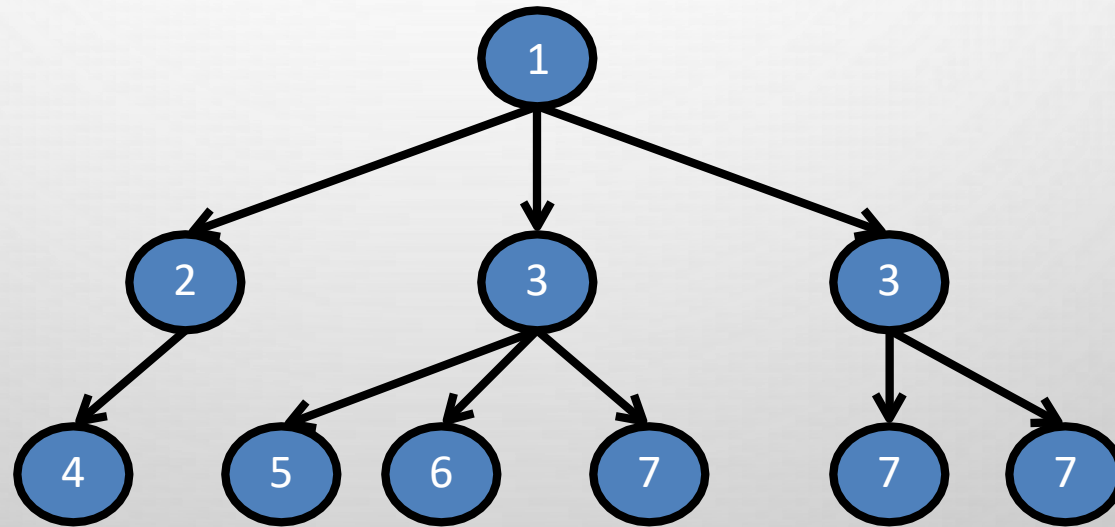
- Webpage Layout





TREE

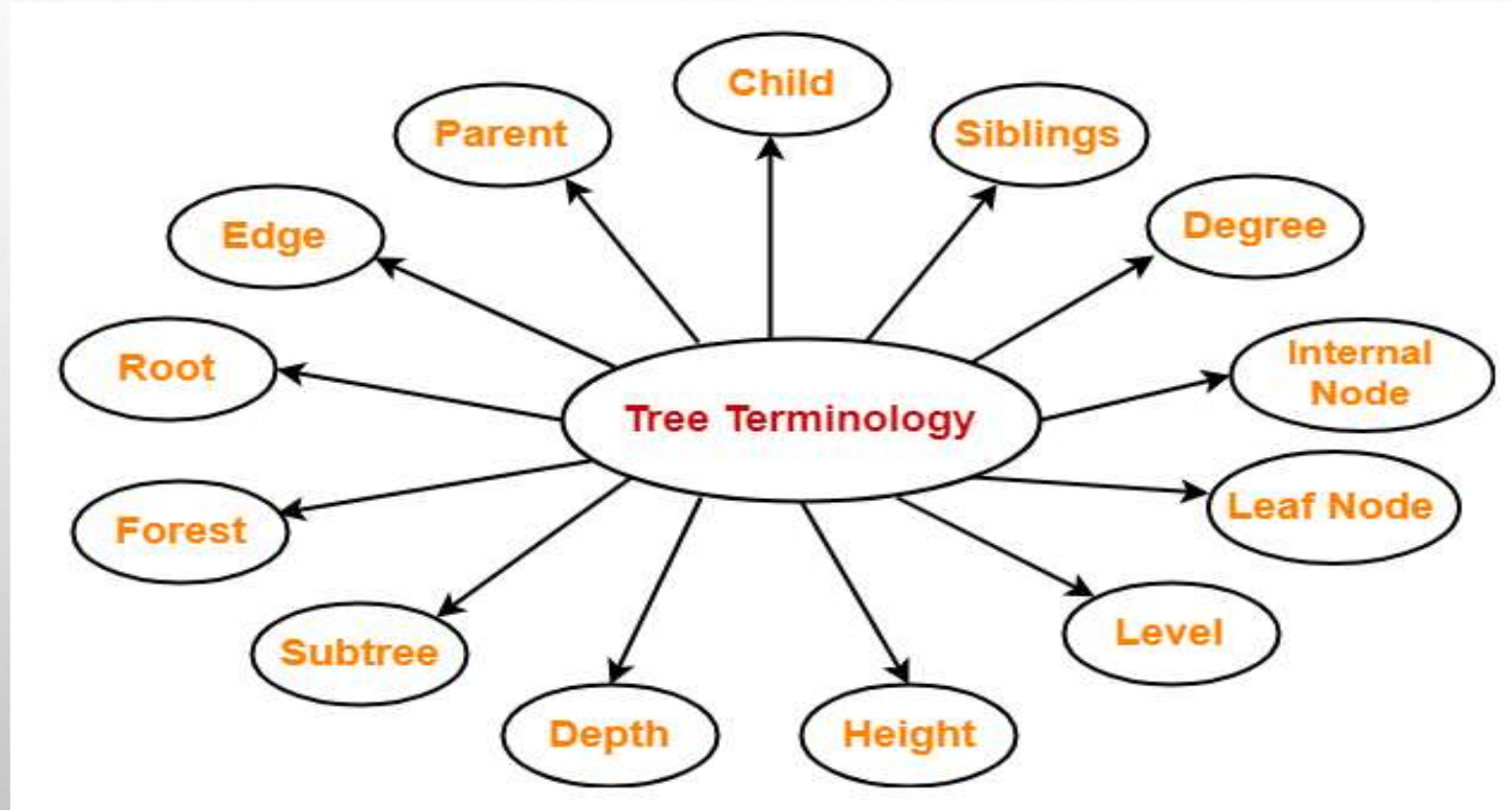
- Finite collection of special data items called **nodes**
- Nodes of the tree are arranged in **hierarchical structures** to represent the **parent-child relationship**
- **No restriction** on the **number of children** for a node



TREE APPLICATIONS

- BINARY SEARCH TREES(BSTS) ARE USED TO QUICKLY CHECK WHETHER AN ELEMENT IS PRESENT IN A SET OR NOT.
- HEAP IS A KIND OF TREE THAT IS USED FOR HEAP SORT.
- A MODIFIED VERSION OF A TREE CALLED TRIES IS USED IN MODERN ROUTERS TO STORE ROUTING INFORMATION.
- MOST POPULAR DATABASES USE B-TREES AND T-TREES, WHICH ARE VARIANTS OF THE TREE STRUCTURE WE LEARNED ABOVE TO STORE THEIR DATA
- COMPILERS USE A SYNTAX TREE TO VALIDATE THE SYNTAX OF EVERY PROGRAM YOU WRITE.

TREE - TERMINOLOGIES

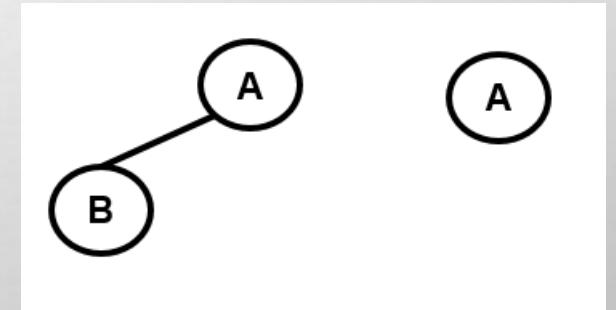
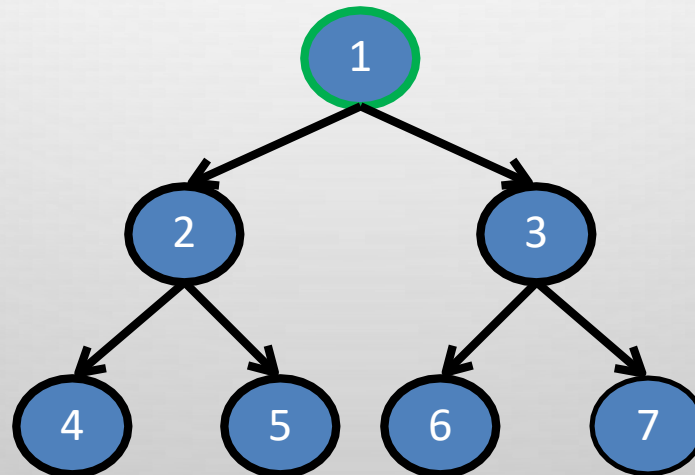


TYPES OF TREES

- ❖ GENERAL TREE
- ❖ BINARY TREE
- ❖ BINARY SEARCH TREE
- ❖ AVL TREE
- ❖ EXPRESSION TREE
- ❖ HEAP

BINARY TREE: DEFINITION

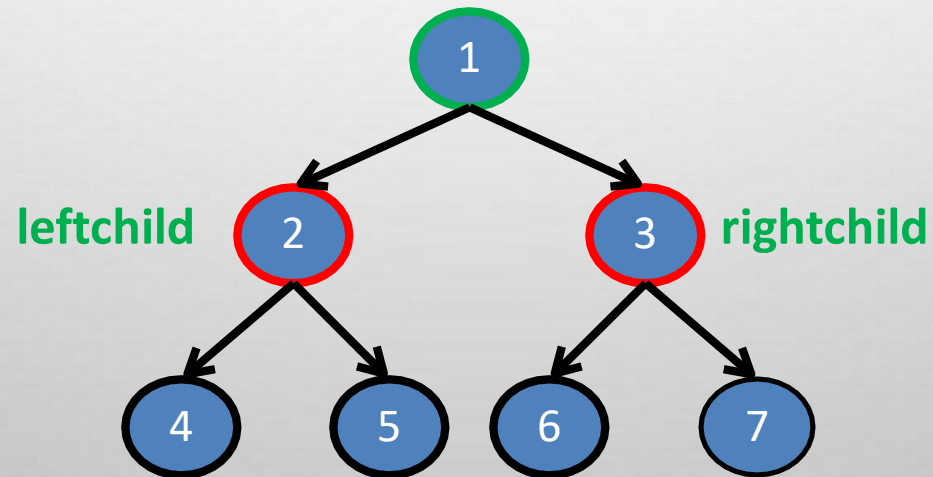
- A binary tree T is a finite set of nodes such that
 1. T is empty or T contains a specially designated node called the root
 2. The remaining nodes of T are partitioned into two disjoint sets T_1 and T_2 where each of these sets is a tree $[T_n \text{ where } n=0,1 \text{ or } 2]$
 T_1 and T_2 are called the subtrees of the root- Left and Right subtrees



TERMINOLOGIES

Binary TREE

- a tree which is **either empty** or each node can have **maximum 2 children**
- each node can have either **0 , 1 or 2 children**
- **Leftchild** - Node on the left of the parent
- **Rightchild** - Node on the right of the parent



DEGREE

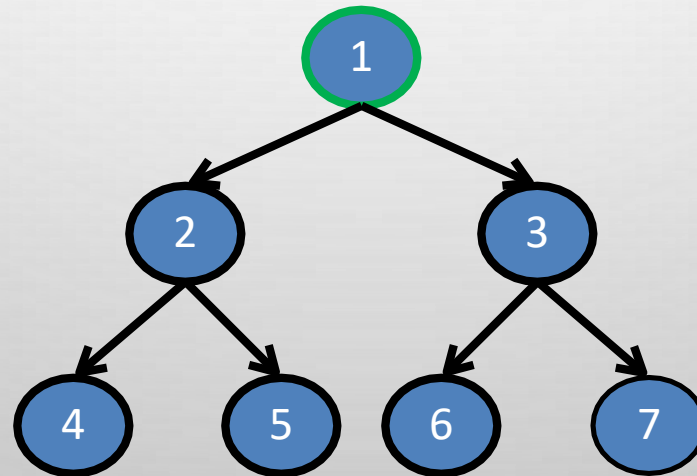
- IN A **BINARY TREE**, THE **DEGREE OF A NODE** REFERS TO THE **NUMBER OF CHILDREN** THAT NODE HAS.

Degree	Description	Example
0	Leaf node – has no children	Node with no left or right child
1	Single child node – has only one child	Node with only left or only right child
2	Internal node – has both left and right children	Node with both children present

TERMINOLOGIES

Binary TREE

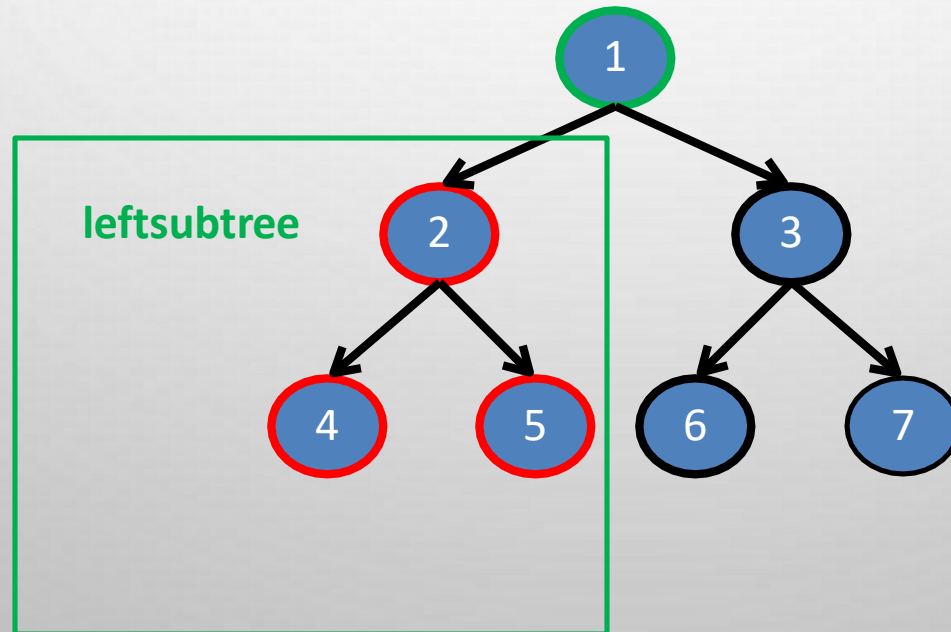
- Leftsubtree - Tree on the left of the parent
- Rightsubtree - Tree on the right of the parent



TERMINOLOGIES

Binary TREE

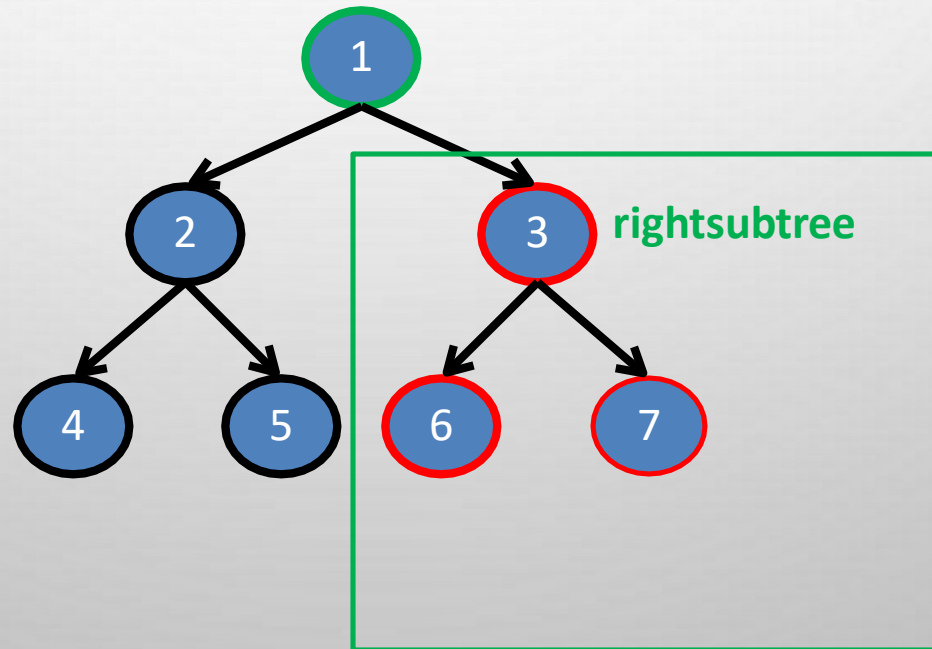
- Leftsubtree - Tree on the left of the parent
- Rightsubtree - Tree on the right of the parent



TERMINOLOGIES

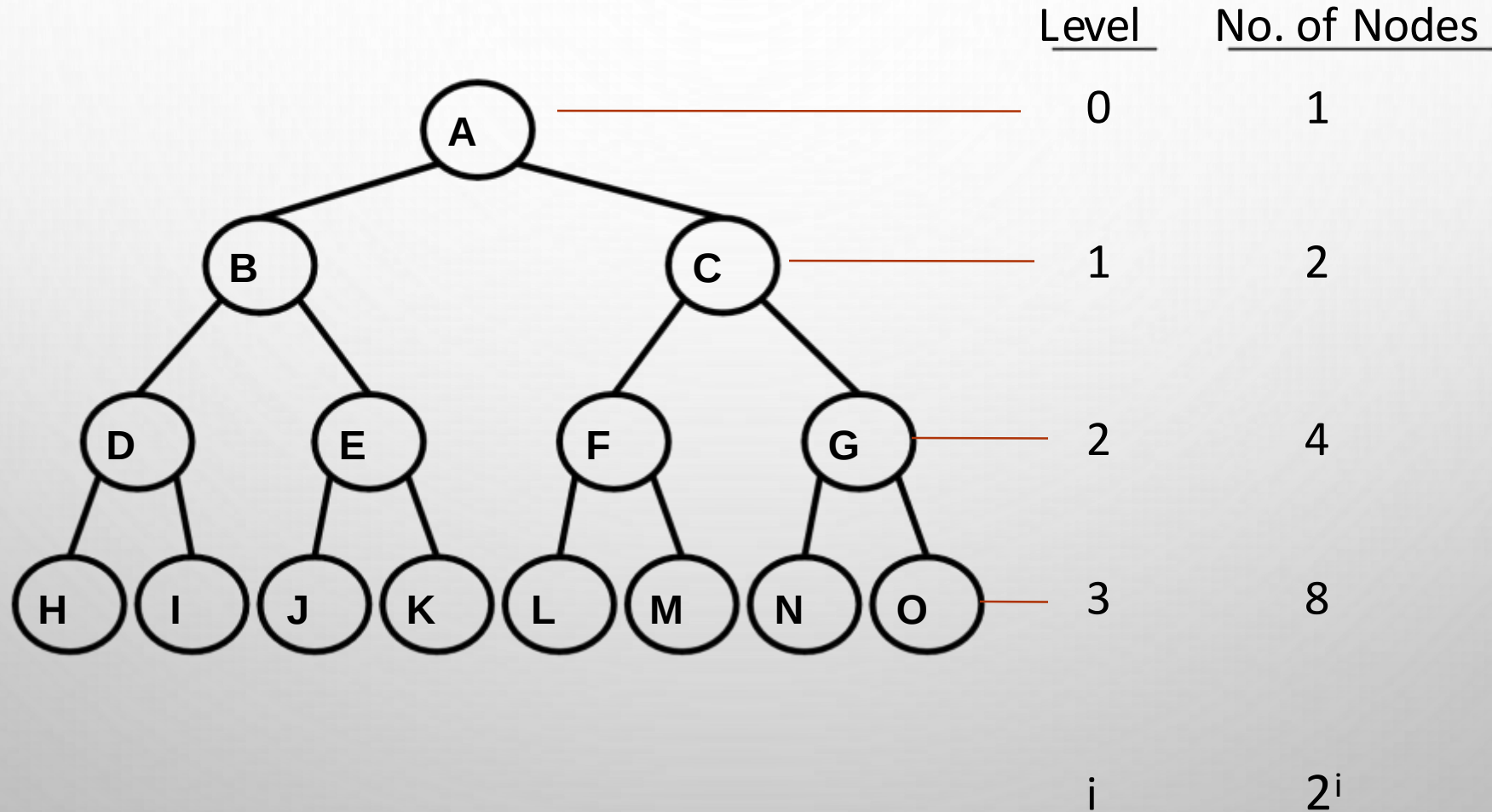
Binary TREE

- Leftsubtree - Tree on the left of the parent
- Rightsubtree - Tree on the right of the parent



BINARY TREE -PROPERTIES

- The maximum number of nodes on level i of a binary tree is 2^i , $i \geq 0$



BINARY TREE -PROPERTIES

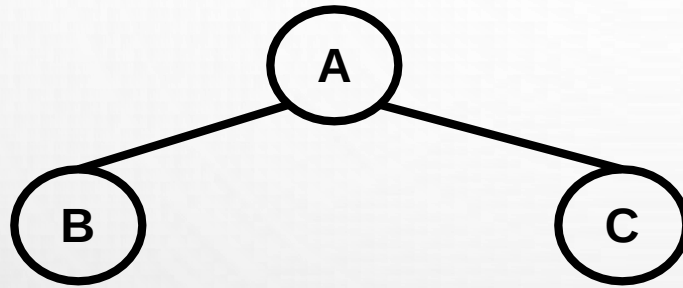
- The maximum number of nodes in a binary tree of height h is $2^{h+1}-1$, $h \geq 0$



<u>Height</u>	<u>No. of Nodes</u>
0	1

BINARY TREE -PROPERTIES

- The maximum number of nodes in a binary tree of height h is $2^{h+1}-1$, $h \geq 0$



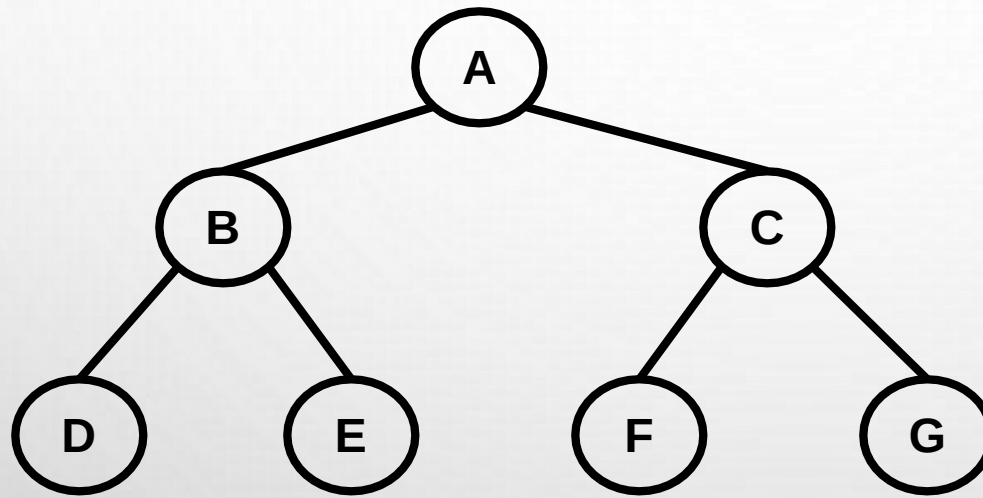
<u>Height</u>	<u>No. of Nodes</u>
---------------	---------------------

0	1
---	---

1	3
---	---

BINARY TREE -PROPERTIES

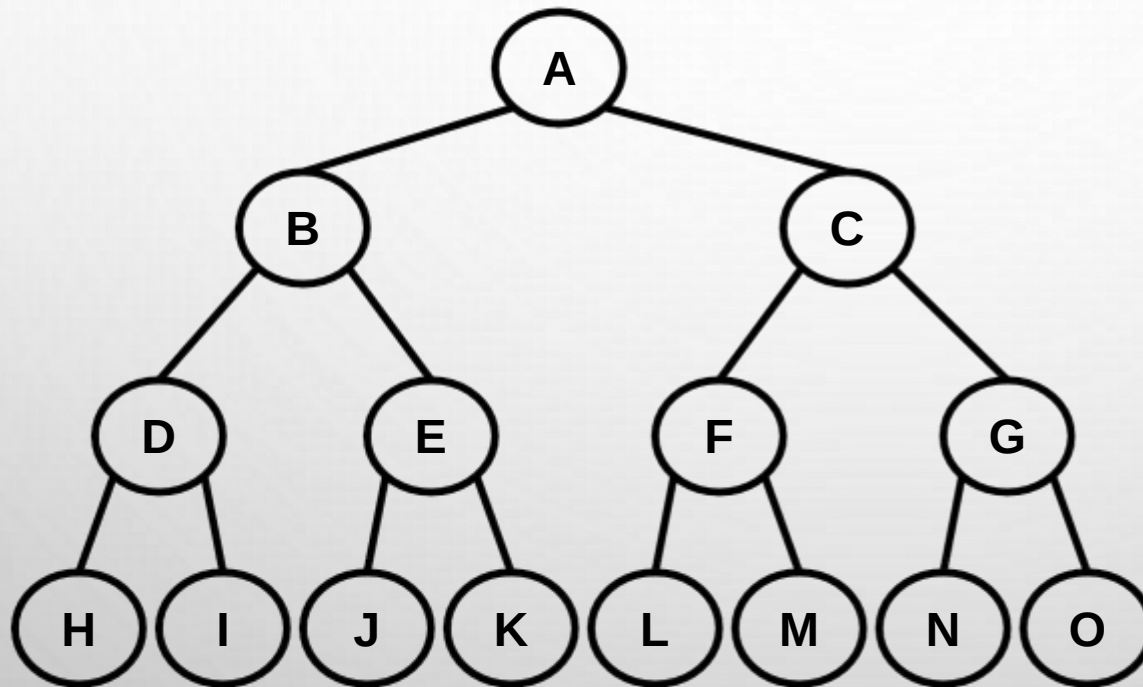
- The maximum number of nodes in a binary tree of height h is $2^{h+1}-1$, $h \geq 0$



<u>Height</u>	<u>No. of Nodes</u>
0	1
1	3
2	7

BINARY TREE -PROPERTIES

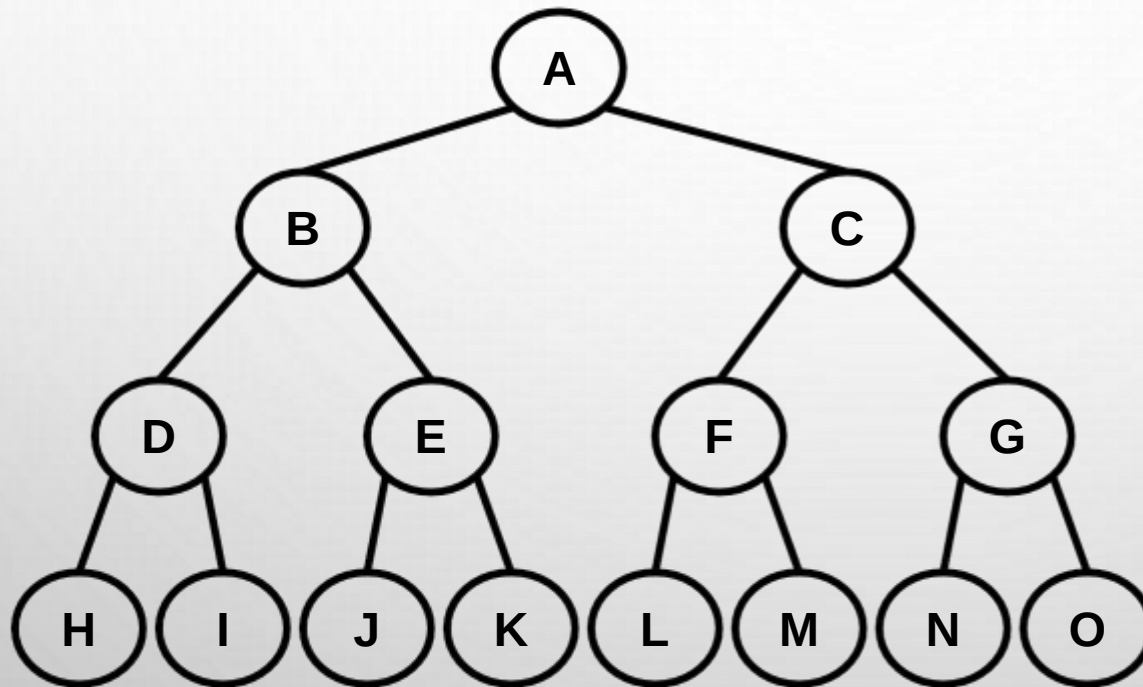
- The maximum number of nodes in a binary tree of height h is $2^{h+1}-1$, $h \geq 0$



<u>Height</u>	<u>No. of Nodes</u>
0	1
1	3
2	7
3	15

BINARY TREE -PROPERTIES

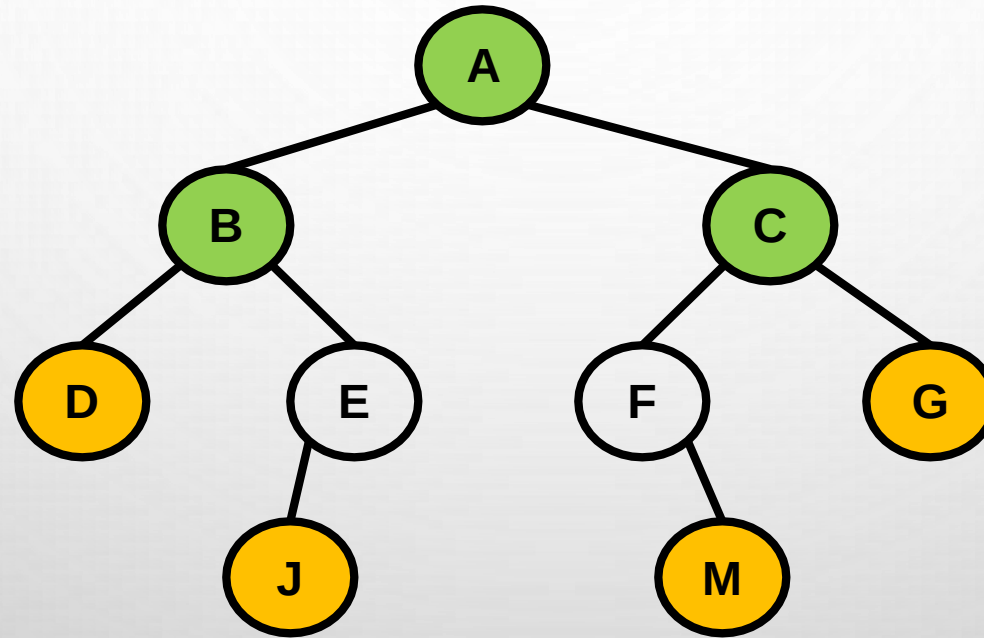
- The maximum number of nodes in a binary tree of height h is $2^{h+1}-1$, $h \geq 0$



<u>Height</u>	<u>No. of Nodes</u>
0	1
1	3
2	7
3	15
h	$2^{h+1}-1$

BINARY TREE -PROPERTIES

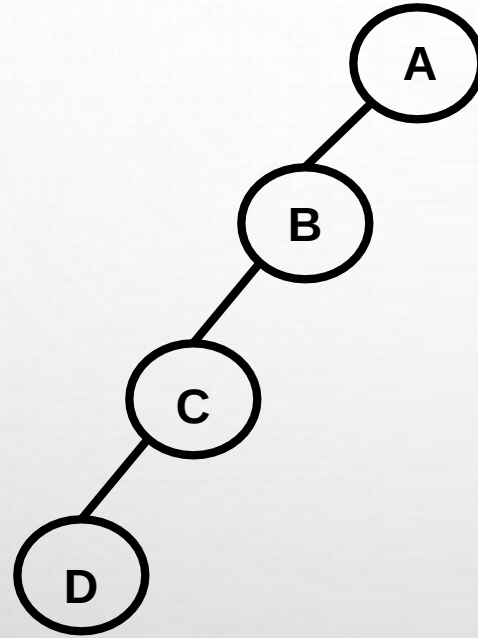
- For any nonempty binary tree, T, if n_0 is the number of leaf nodes and n_2 the number of nodes of degree 2, then $n_0 = n_2 + 1$



- Number of nodes with degree 2 = $n_2 = 3$
- Number of leaf nodes = $n_0 = 4$

BINARY TREE -PROPERTIES

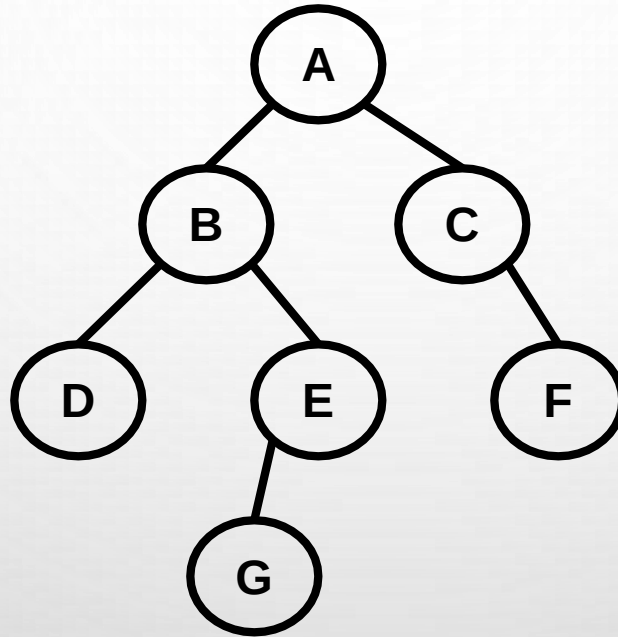
- Minimum number of nodes possible in a binary tree of height h is $h+1$



- Consider this skewed binary tree
- Here, height of the tree = 3
- Minimum number of nodes possible = 4

BINARY TREE -PROPERTIES

- For any non empty binary tree, if n is the number of nodes and e is the number of edges then $n=e+1$



- Number of nodes = $n = 7$
- Number of edges = $e = 6$

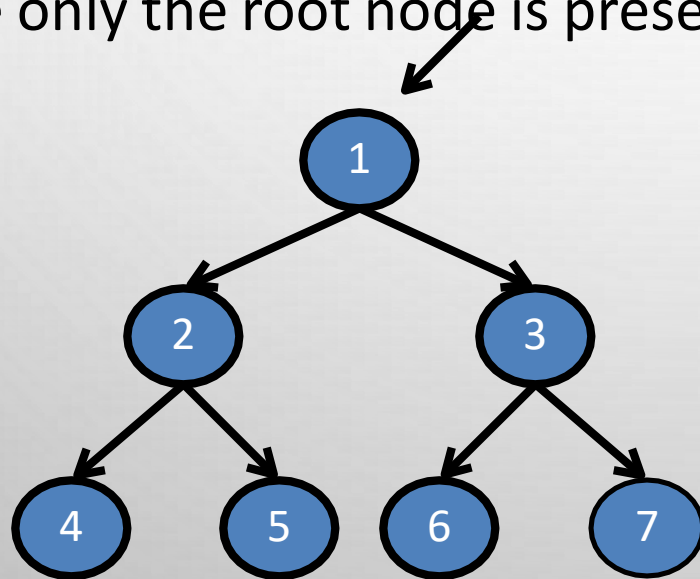
REPRESENTATIONS OF BINARY TREE

1. Linear or sequential representation
2. Linked or pointer representation

REPRESENTATIONS OF BINARY TREE

Linear or sequential representation

- A **block of memory for an array** is allocated before storing the actual tree in it
- Once memory is allocated, the size of the tree is restricted as permitted by the memory
- In this representation, the **nodes are stored level by level**, starting from the zero level where only the root node is present



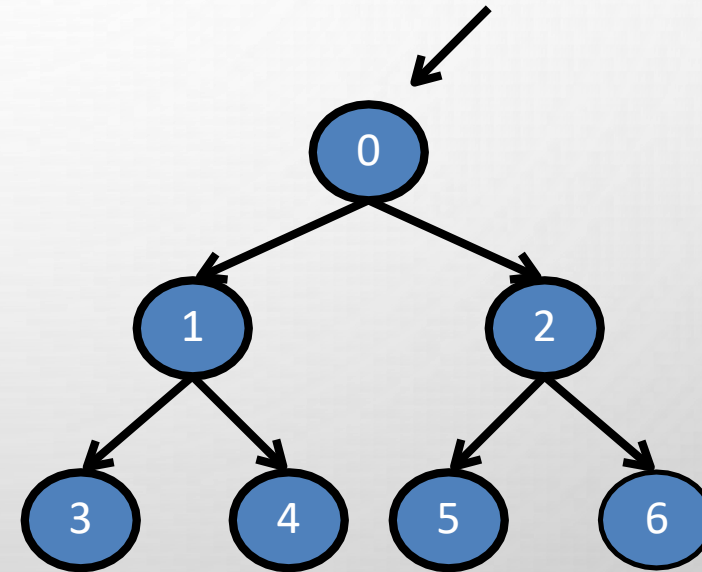
A[0]	1
A[1]	2
A[2]	3
A[3]	4
A[4]	5
A[5]	6
A[6]	7

REPRESENTATIONS OF BINARY TREE

Linear or sequential representation

Following rules can be used to decide the location of any node

1. The **root node** is at **index 0**
2. For any node with index ,
 1. **parent (i)** is at **$(i - 1)/2$**
 2. **Left child of i** is at **$2i+1$**
 3. **Right child of i** is at **$2i + 2$**



A[0]	0
A[1]	1
A[2]	2
A[3]	3
A[4]	4
A[5]	5
A[6]	6

REPRESENTATIONS OF BINARY TREE

Linear or sequential representation

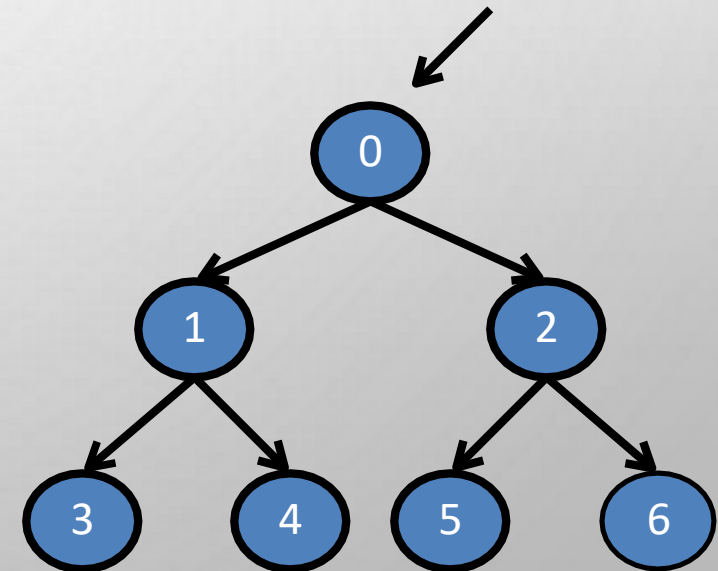
Left child of i is at $2i+1$:

- If $2i + 1 > n - 1$, then the node does not have a left child
- Eg: $i=3$, then $2i+1 = 7$, but $7 > n-1 \rightarrow$ no left child

Right child of i is at $2i + 2$:

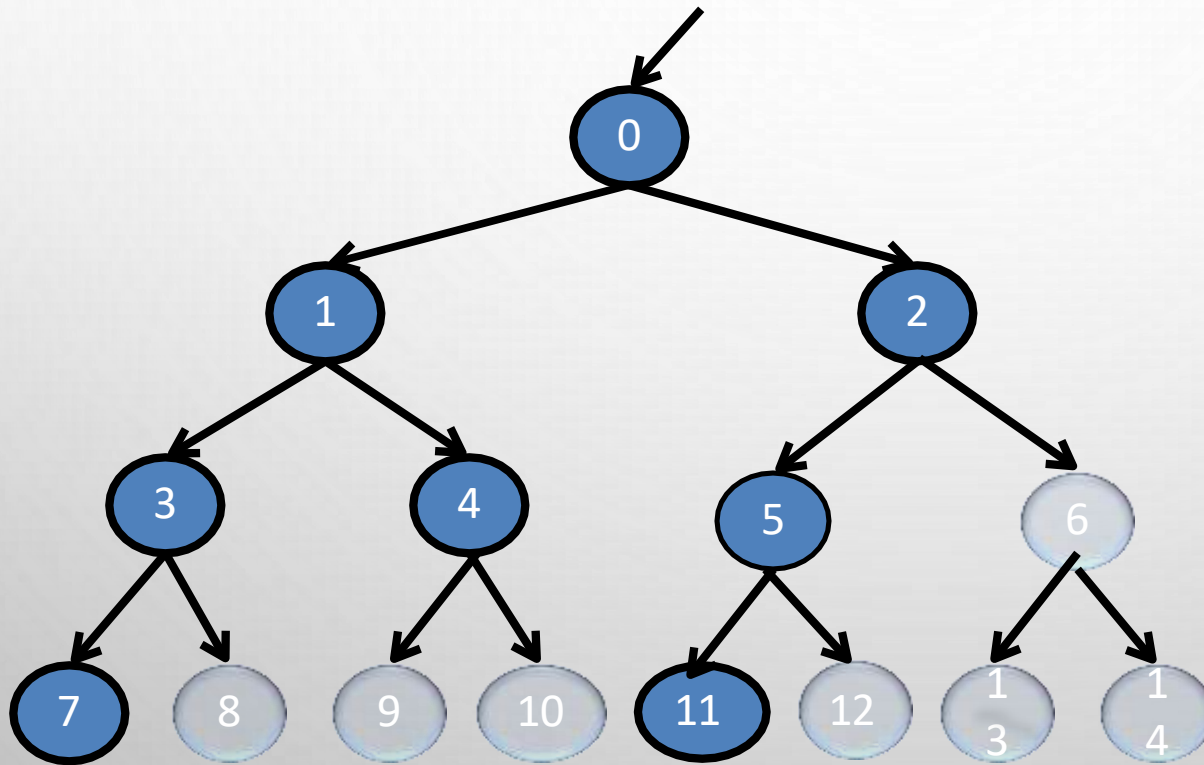
- If $2i + 2 > n - 1$, then the node does not have a right child
- Eg: $i=3$, then $2i+2 = 8$, but $8 > n-1 \rightarrow$ no right child

A[0]	0
A[1]	1
A[2]	2
A[3]	3
A[4]	4
A[5]	5
A[6]	6



REPRESENTATIONS OF BINARY TREE

Linear or sequential representation



A[0]	0
A[1]	1
A[2]	2
A[3]	3
A[4]	4
A[5]	5
A[6]	-
A[7]	7
A[8]	-
A[9]	-
A[10]	-
A[11]	11
A[12]	-
A[13]	-
A[14]	-

BINARY TREE -ARRAY REPRESENTATIONS

- Advantages
 - Any node can be accessed from any other node by calculating the index.
 - Data are stored without any pointer
 - Programming languages, where dynamic memory allocation is not possible (like BASIC, FORTRAN), array representation is only possible
 - Good for Complete Binary tree.
 - Searching is fast

BINARY TREE -ARRAY REPRESENTATIONS

- Disadvantages
 - Good for Complete Binary tree. If it is not a complete binary tree then, a lot of memory is wasted
 - Dynamic memory allocation is not possible

REPRESENTATIONS OF BINARY TREE

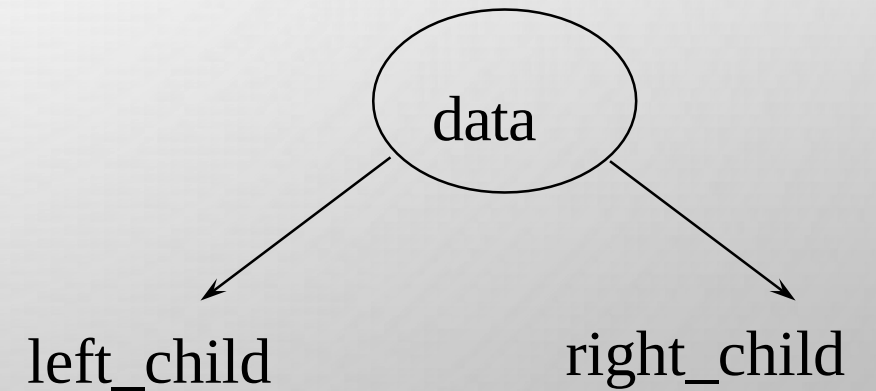
2 Linked or pointer representation

- Tree can also be defined as a finite collection of nodes where each node is divided into 3 parts containing
 - Left child address
 - Information (data)
 - Right child address

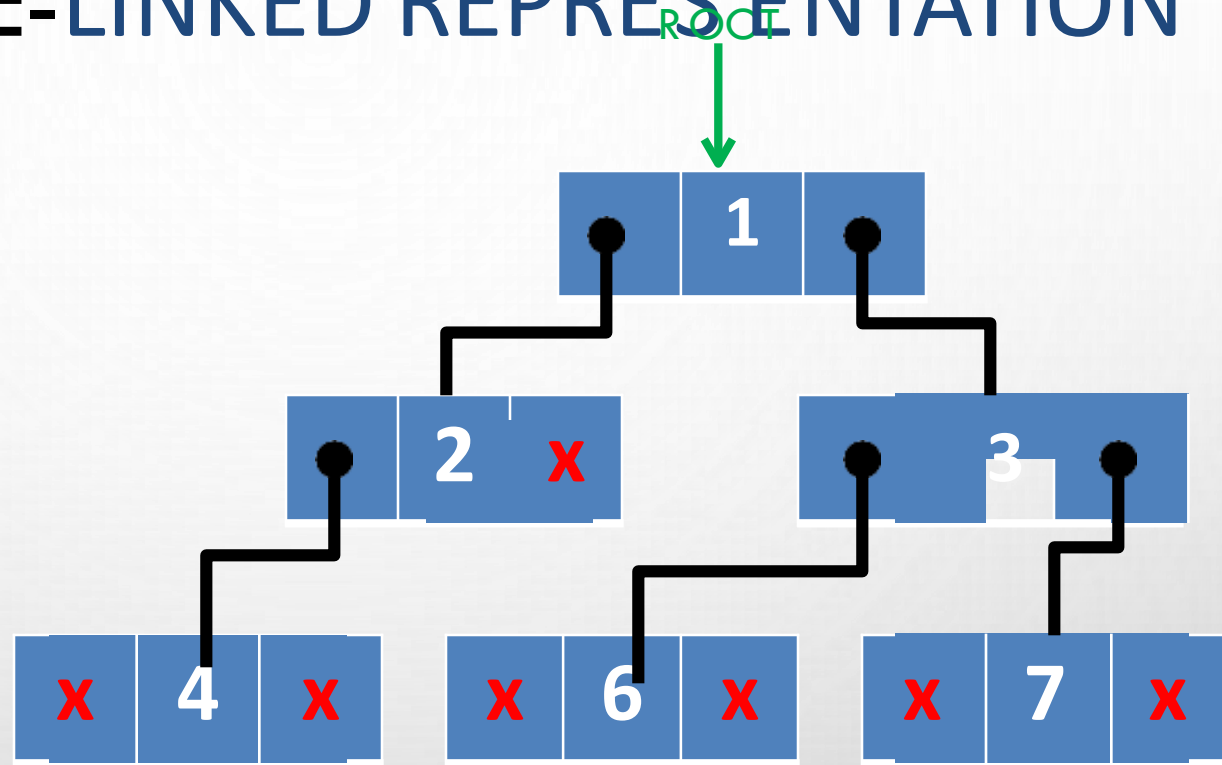
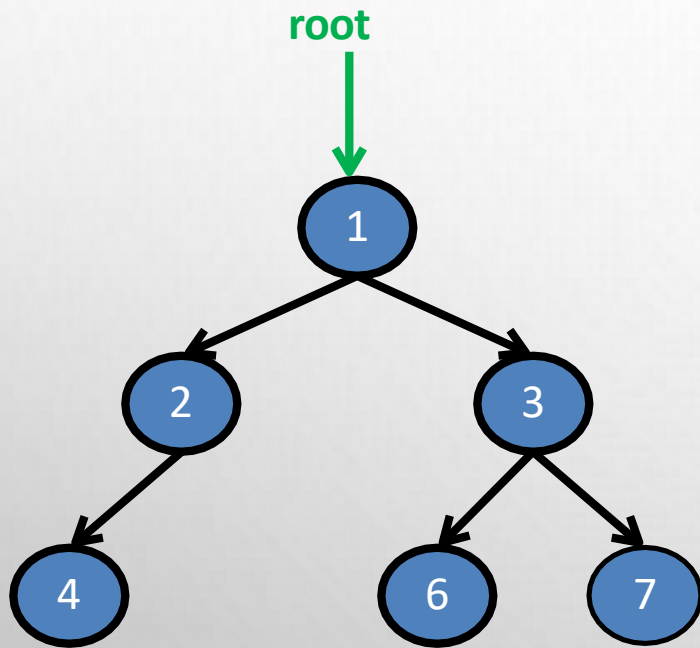


BINARY TREE-LINKED REPRESENTATION

```
struct treenode {  
    int data;  
    struct treenode *left_child, *right_child;  
};
```



BINARY TREE-LINKED REPRESENTATION



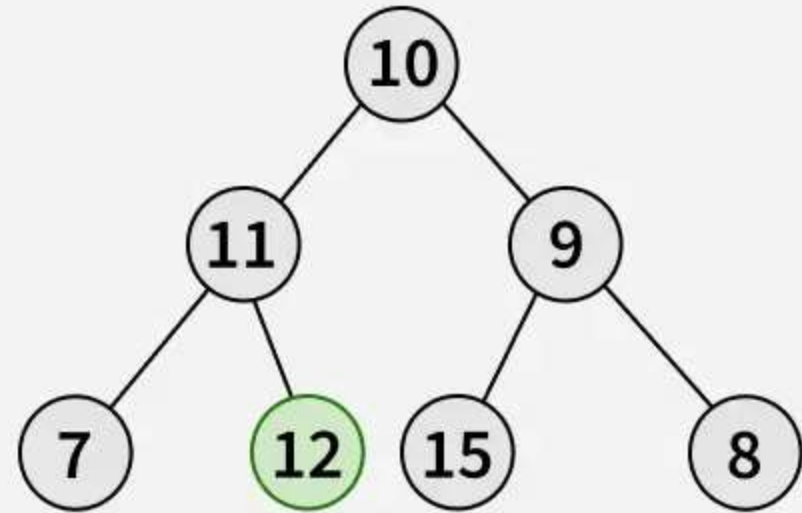
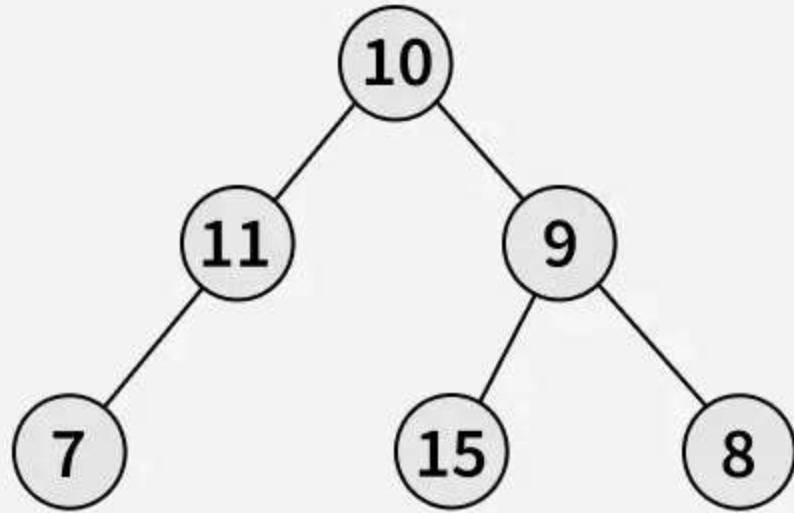
OPERATIONS ON BINARY TREE

- Insertion
- Deletion
- Traversals

INSERTION IN A BINARY TREE

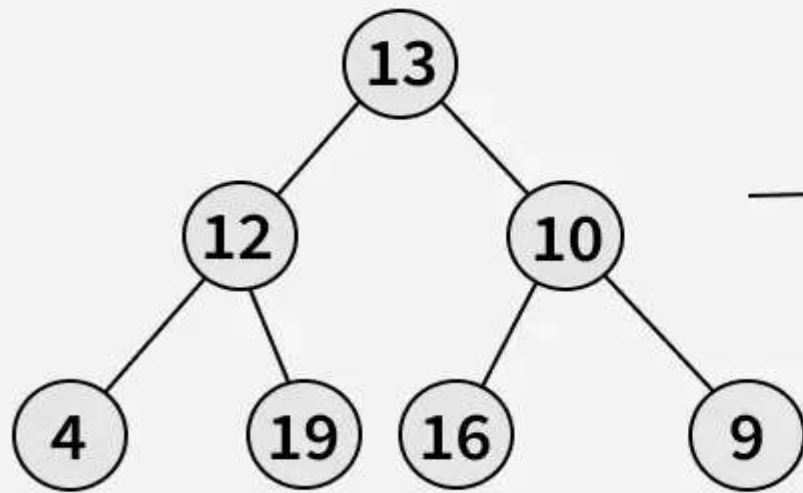
- FOR A GENERAL BINARY TREE (NOT A BINARY SEARCH TREE), THERE IS NO INHERENT ORDER. THE MOST COMMON APPROACH IS TO INSERT THE NEW NODE AT THE FIRST AVAILABLE POSITION, OFTEN IDENTIFIED USING LEVEL-ORDER TRAVERSAL (BREADTH-FIRST SEARCH).
- TRAVERSE THE TREE LEVEL BY LEVEL.
- FIND THE FIRST NODE THAT HAS EITHER ITS LEFT CHILD OR RIGHT CHILD AS NULL.
- INSERT THE NEW NODE AS THE MISSING CHILD (E.G., IF THE LEFT CHILD IS NULL, INSERT IT THERE; OTHERWISE, IF THE RIGHT CHILD IS NULL, INSERT IT THERE).

- **INPUT:** KEY = 12

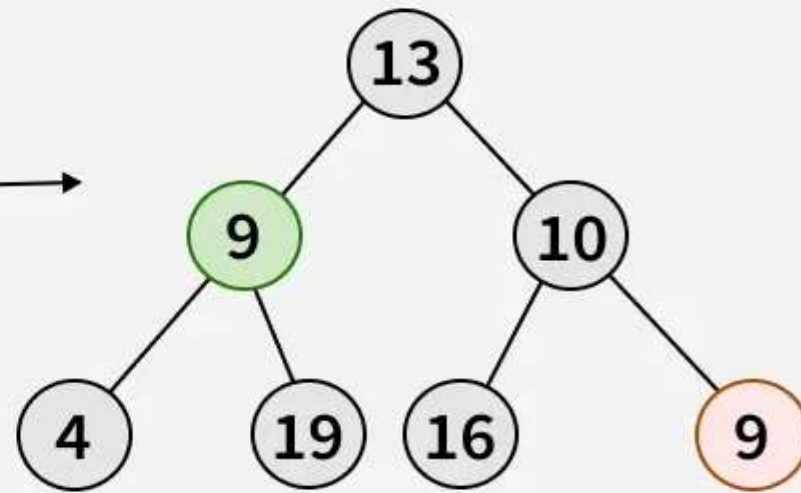


DELETION IN A BINARY TREE

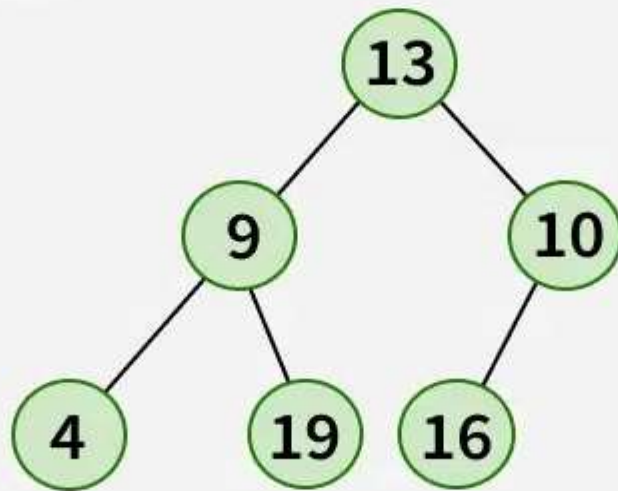
- THE IDEA IS TO TRAVERSE THE TREE IN LEVEL ORDER MANNER. TO PERFORM THE **DELETION IN A BINARY TREE** FOLLOW BELOW:
- STARTING AT THE ROOT, FIND THE **DEEPEST** AND **RIGHTMOST NODE** IN THE BINARY TREE AND THE NODE WHICH WE WANT TO **DELETE**.
- REPLACE THE DEEPEST RIGHTMOST NODE'S DATA WITH THE NODE TO BE DELETED.
- THEN **DELETE** THE DEEPEST RIGHTMOST NODE.



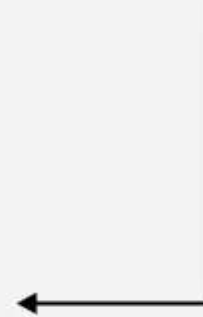
Node to be deleted is 12



Replacing 12 with deepest node

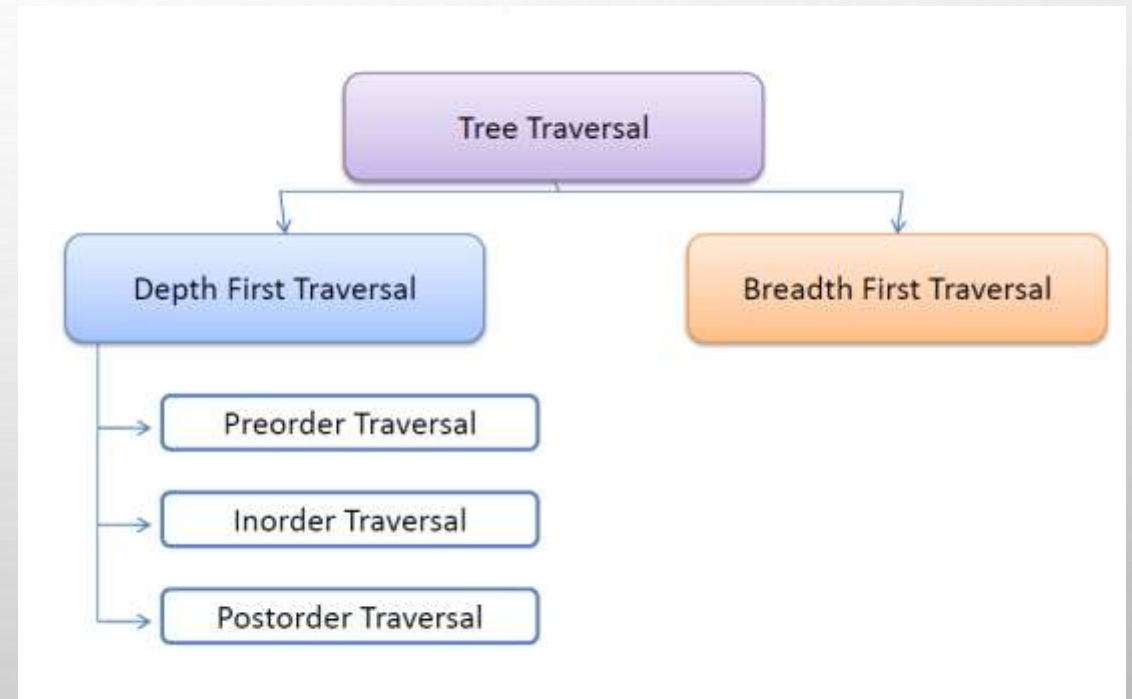


Deleting the deepest node



BINARY TREE TRAVERSALS

- Traversal means visit each node in the binary tree exactly once
- There are three commonly used tree traversals
 - Inoder Traversal
 - Preorder Traversal
 - Postorder Traversal



TREE TRAVERSAL

- Used to visit each node in the tree exactly once
- Three types of traversal
 1. Preorder
 2. Inorder
 3. Postorder

TREE

TRAVERSAL

- Three types of traversal

Preorder

- » Visit root
- » Visit left subtree in preorder
- » Visit right subtree in preorder

TREE TRAVERSAL

- Three types of traversal

Inorder

- » Visit left subtree in inorder
- » Visit root
- » Visit right subtree in inorder

TREE TRAVERSAL

- Three types of traversal

Postorder

- » Visit **left** subtree in inorder
- » Visit **right** subtree in inorder
- » Visit **root**

- Let L, V, and R stand for moving left, visiting the node/root, and moving right
 - Inorder- LVR
 - Postorder- LRV
 - Preorder- VLR

Preorder Traversal

- **Algorithm-**

- Visit the root
- Traverse the left sub tree i.e. call Preorder (left sub tree)
- Traverse the right sub tree i.e. call Preorder (right sub tree)

1-PREORDER TRAVERSAL

Algorithm PREORDER(root)

Input: root is the pointer to the root node of the binary search tree

Output: Visiting of all the nodes in preorder fashion

Data structure: Linked structure

Steps:

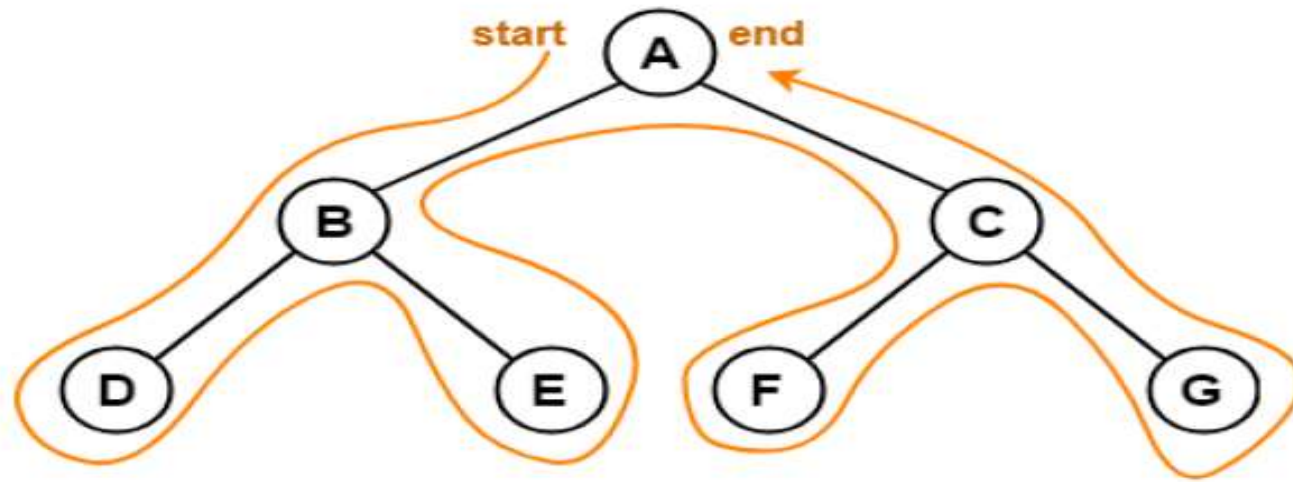
1. ptr=root
2. If(ptr!=NULL)
 1. VISIT(ptr)
 2. PREORDER(ptr->llink)
 3. PREORDER(ptr->rlink)
1. EndIf
2. Stop

Code:

```
PREORDER(struct tree *ptr) // ptr=root
{
    If(ptr!=NULL)
        printf("%d",ptr->data);
        PREORDER(ptr->llink);
        PREORDER(ptr->rlink);
    }
}
```

Preorder Traversal Shortcut

Traverse the entire tree starting from the root node keeping yourself to the left.



Preorder Traversal : A , B , D , E , C , F , G

2-INORDER TRAVERSAL

Algorithm INORDER(root)

Input: root is the pointer to the root node of the binary search tree

Output: Visiting of all the nodes in inorder fashion

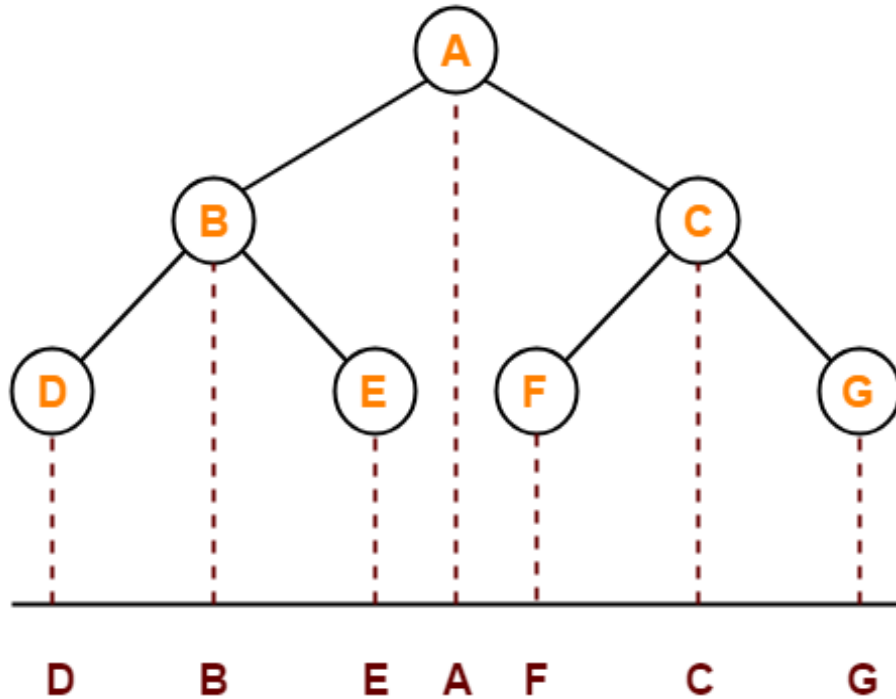
Data structure: Linked structure

Steps:

1. ptr=root
2. If(ptr!=NULL)
 1. INORDER(ptr->llink)
 2. VISIT(ptr)
 3. INORDER(ptr->rlink)
3. EndIf
4. Stop

Inorder Traversal Shortcut

Keep a plane mirror horizontally at the bottom of the tree and take the projection of all the nodes.

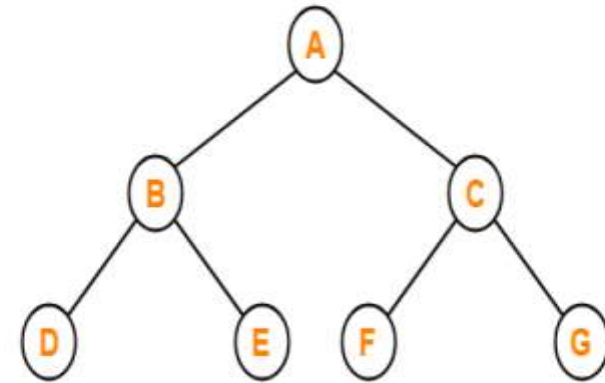


Inorder Traversal : D , B , E , A , F , C , G

Inorder Traversal

- Algorithm-

- Traverse the left sub tree i.e. call Inorder (left sub tree)
- Visit the root
- Traverse the right sub tree i.e. call Inorder (right sub tree)



Inorder Traversal : D , B , E , A , F , C , G

3-POSTORDER TRAVERSAL

Algorithm POSTORDER(root)

Input: root is the pointer to the root node of the binary search tree

Output: Visiting of all the nodes in postorder fashion

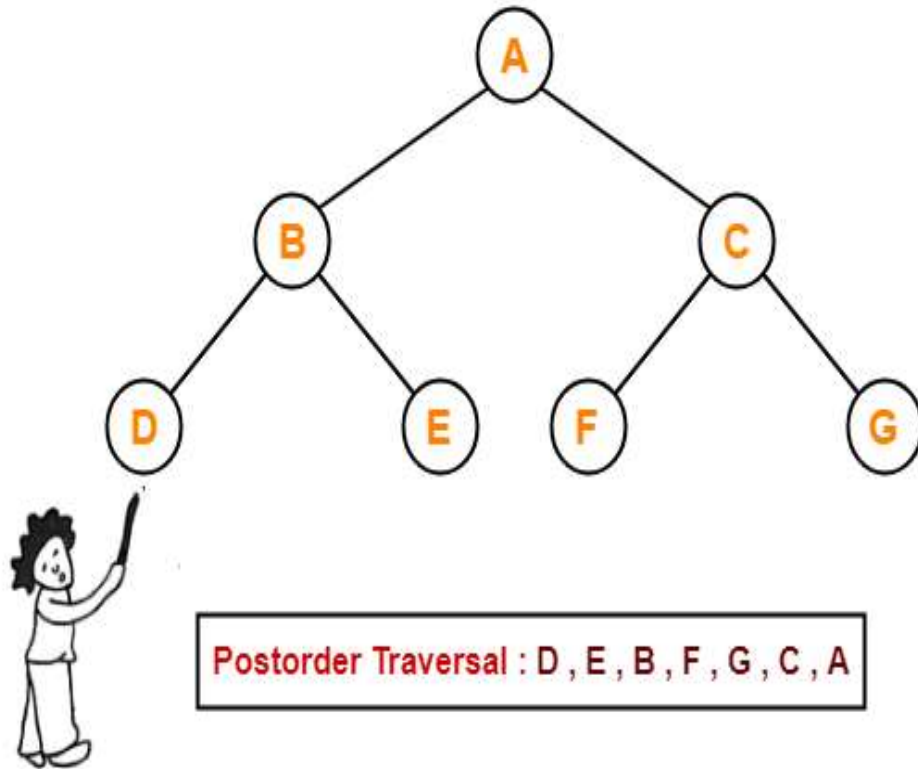
Data structure: linked structure

Steps:

1. ptr=root
2. If(ptr!=NULL)
 1. POSTORDER(ptr->llink)
 2. POSTORDER(ptr->rlink)
 3. VISIT(ptr)
3. EndIf
4. Stop

Postorder Traversal Shortcut

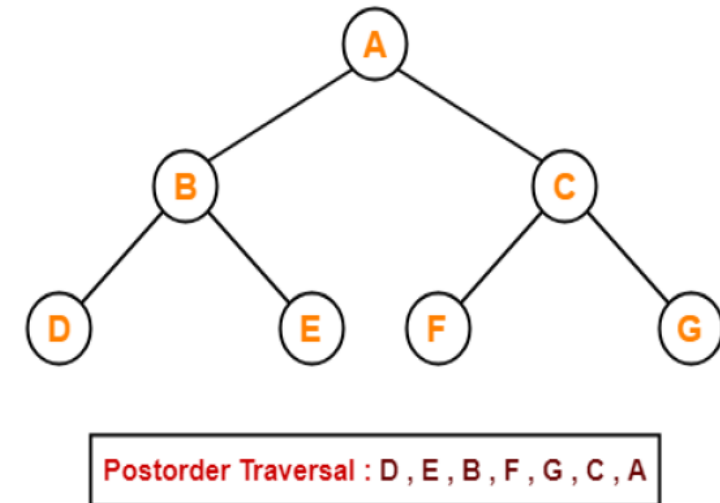
Pluck all the leftmost leaf nodes one by one.



Postorder Traversal

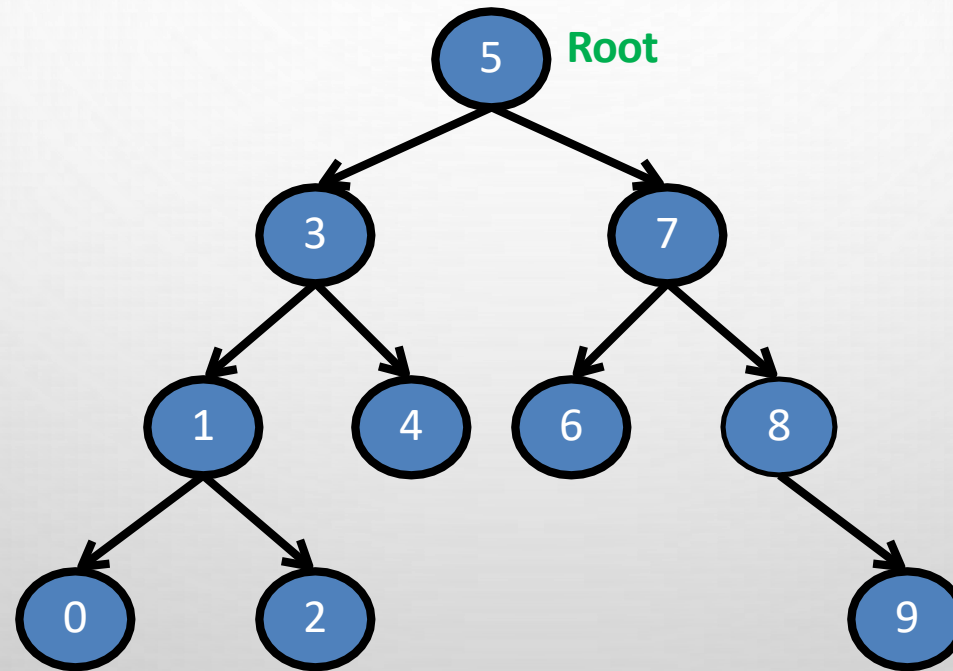
- Algorithm-

- Traverse the left sub tree i.e. call Postorder (left sub tree)
- Traverse the right sub tree i.e. call Postorder (right sub tree)
- Visit the root



QUESTION :

- Write the preorder, inorder and postorder traversal of

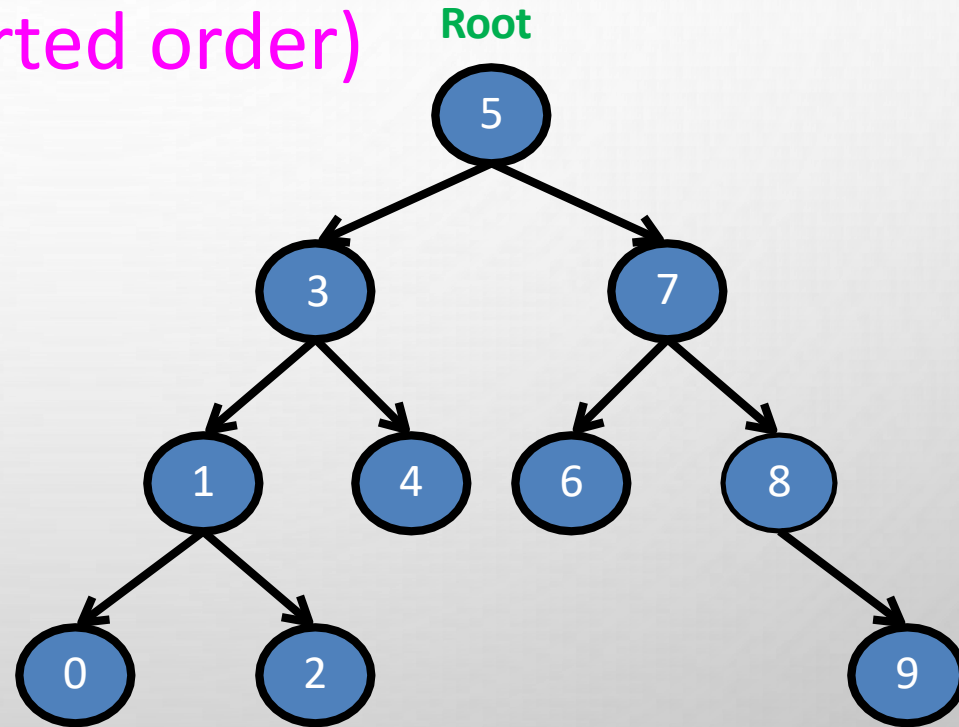


ANSWER:

PREORDER = 5 3 1 0 2 4 7 6 8 9

INORDER= 0 1 2 3 4 5 6 7 8 9 (sorted order)

POSTORDER= 0 2 1 4 3 6 9 8 7 5



CONSTRUCT TREE FROM THE INORDER AND PREORDER TRAVERSALS

PREORDER = 5 3 1 0 2 4 7 6 8 9

INORDER= 0 1 2 3 4 5 6 7 8 9

CONSTRUCT TREE FROM THE FOLLOWING INORDER AND PREORDER TRAVERSALS

PREORDER = 5 3 1 0 2 4 7 6 8 9

INORDER= 0 1 2 3 4 5 6 7 8 9

- Preorder starts with the root
- Now split the inorder traversal into 3 sets
 - Left subtree, root, right subtree
- Repeat the same process on each subtrees

PREORDER = 5 3 1 0 2 4 7 6 8

9

INORDER = 0 1 2 3 4 5 6 7 8

9

Root = 5

[01234] [5] [6789]

Root

0123456789

PREORDER = 5 3 1 0 2 4 7 6 8

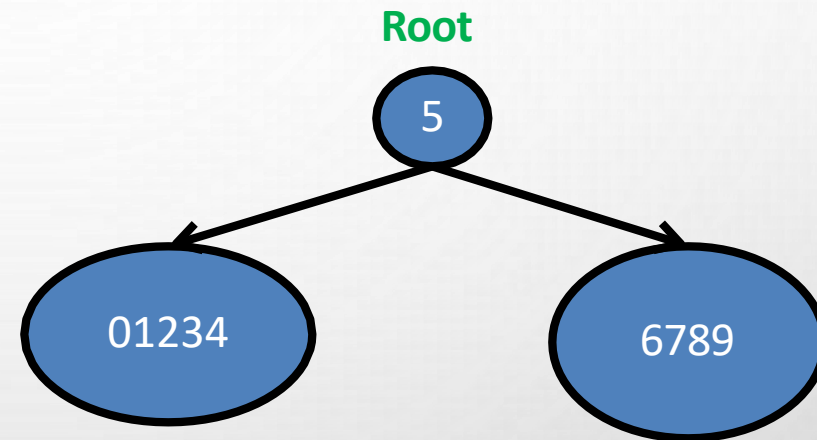
9

INORDER = 0 1 2 3 4 5 6 7 8

9

Root = 5

[01234] **[5]** [6789]

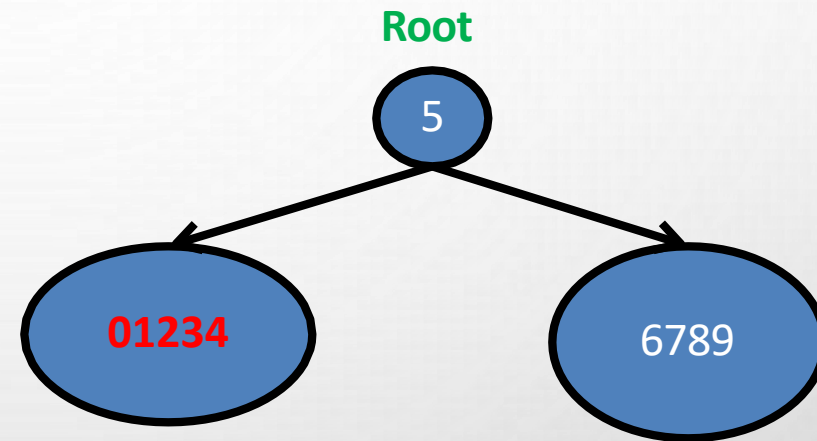


PREORDER = 5 3 1 0 2 4 7 6 8

9

INORDER = 0 1 2 3 4 5 6 7 8

9



PREORDER = 5 3 1 0 2 4 7 6 8

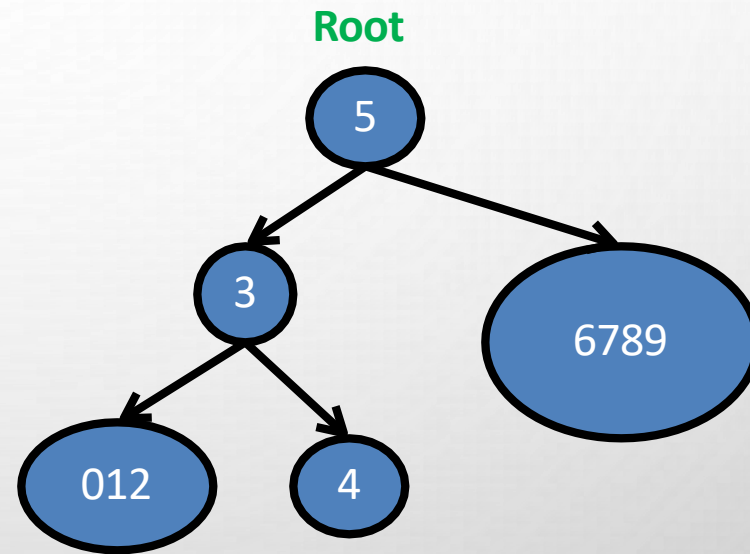
9

INORDER = 0 1 2 3 4 5 6 7 8

9

Root = 3

[012] [3] [4]

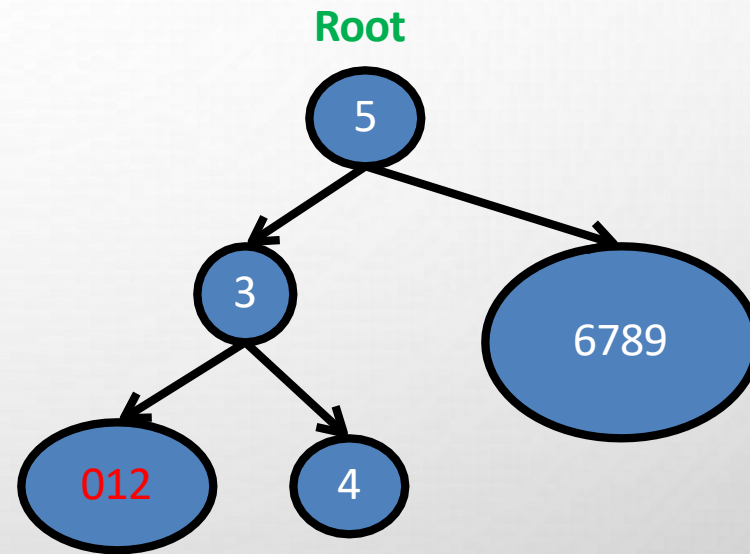


PREORDER = 5 3 1 0 2 4 7 6 8

9

INORDER = 0 1 2 3 4 5 6 7 8

9

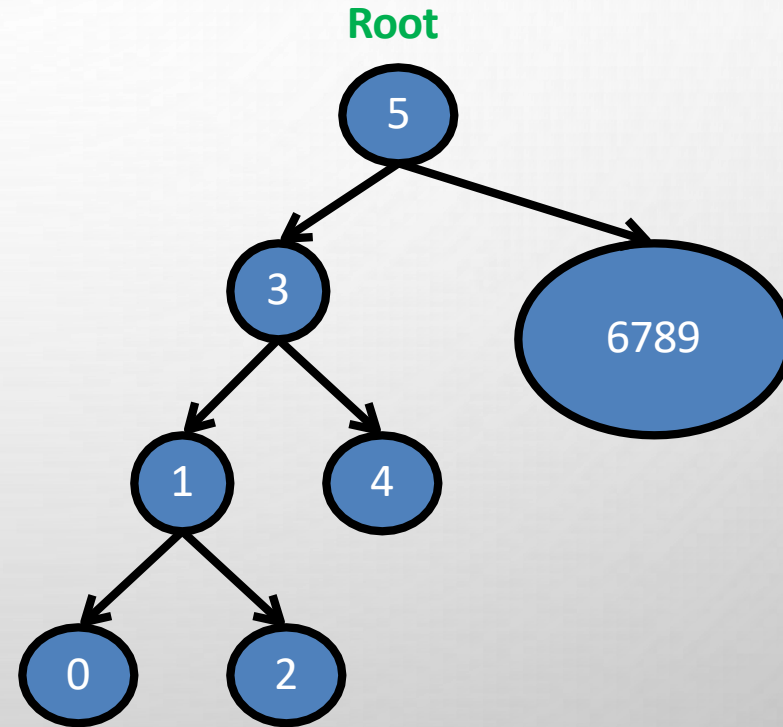


PREORDER = 5 3 1 0 2 4 7 6 8
9
INORDER = 0 1 2 3 4 5 6 7 8
9

Root = 1

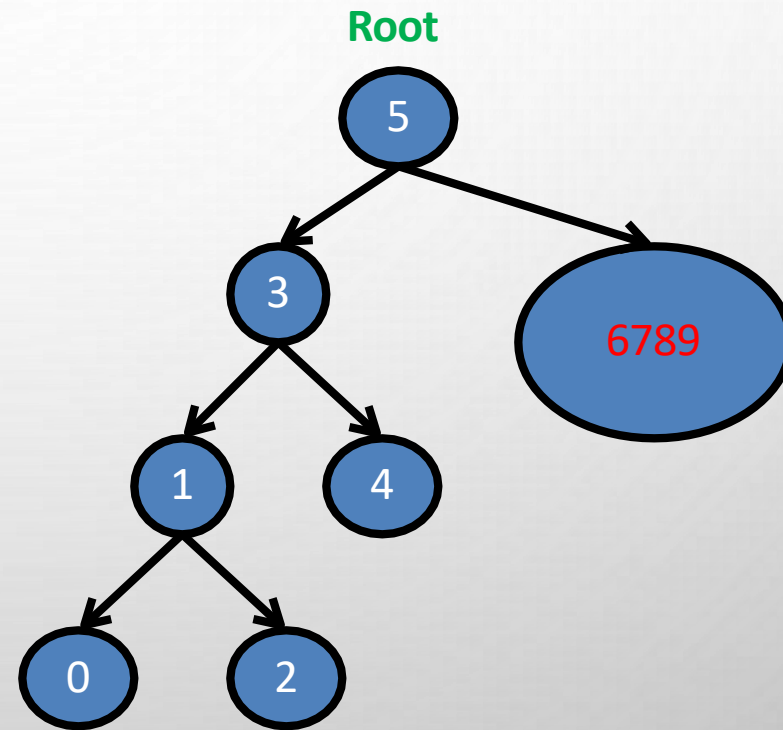
[0] [1] [2]

No more elements to be divided
In the left subtree



PREORDER = 5 3 1 0 2 4 7 6 8 9

INORDER = 0 1 2 3 4 5 6 7 8 9

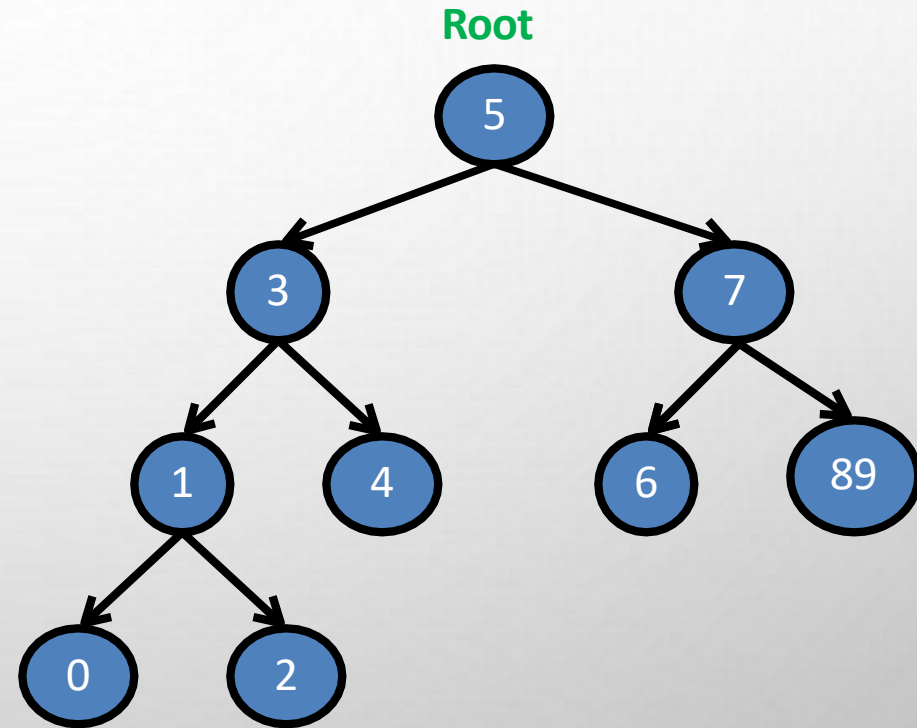


PREORDER = 5 3 1 0 2 4 7 6 8 9

INORDER = 0 1 2 3 4 5 6 7 8 9

Root = 7

[6] [**7**] [89]

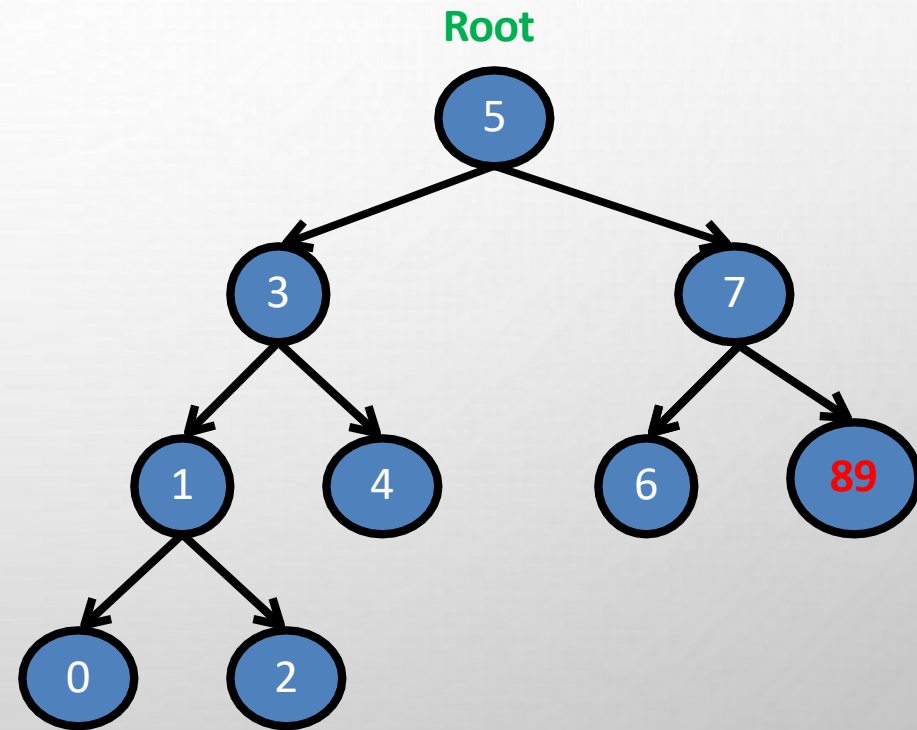


PREORDER = 5 3 1 0 2 4 7 6 8

9

INORDER = 0 1 2 3 4 5 6 7 8

9



PREORDER = 5 3 1 0 2 4 7 6 8

9

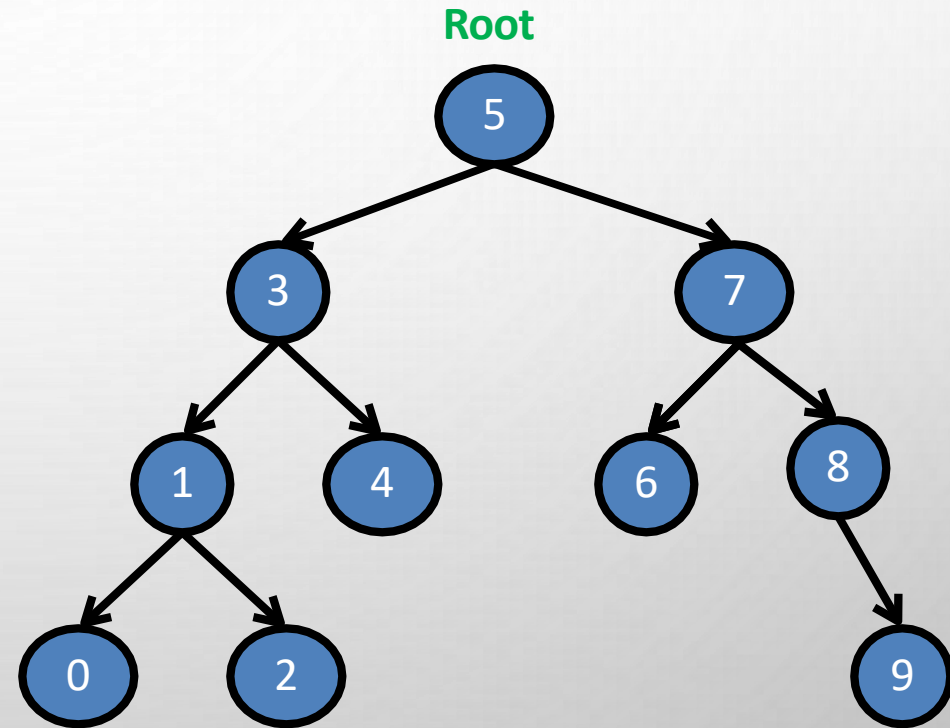
INORDER = 0 1 2 3 4 5 6 7 8

9

Root = 8

[8] [9]

No more elements to be divided
In the right subtree



TYPES OF BINARY TREE

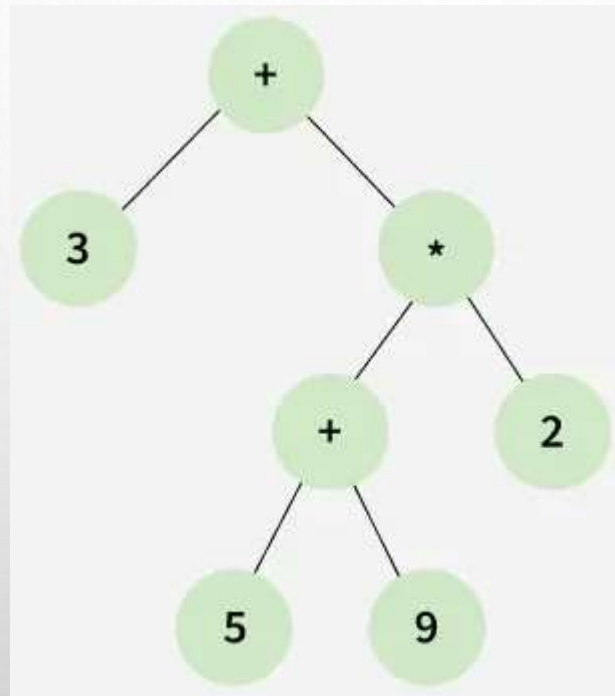
1. Expression tree
2. Binary search tree
3. Heap tree
4. Height balanced tree (AVL tree)

1. EXPRESSION TREE

- AN EXPRESSION TREE IS A BINARY TREE IN WHICH EACH INTERNAL NODE CORRESPONDS TO THE OPERATOR AND EACH LEAF NODE CORRESPONDS TO THE OPERAND.

- If we get an operand in the given expression, then push it in the stack. It will become the root of the expression Tree.
- If an operator gets two values in the expression, then add in the expression tree as its child, and push them in the current node.
- Repeat Step-1 and Step-2 until we do not complete over the given expression.
- Now check if every root node contains nothing but operands and every child node contains only values.

$$3 + ((5 + 9) * 2)$$

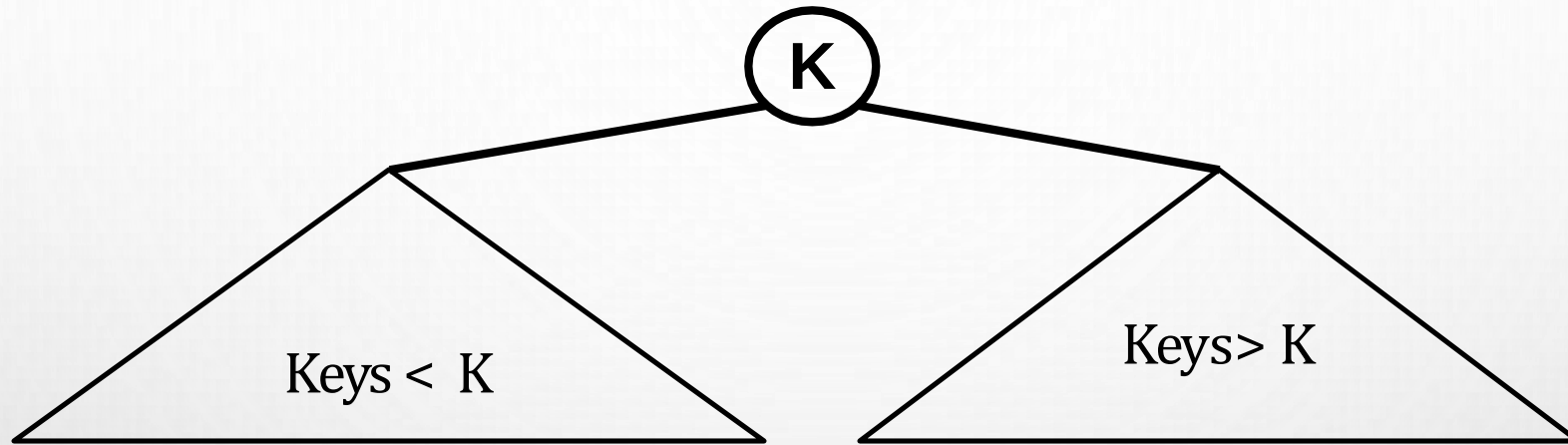


2. BINARY SEARCH TREE

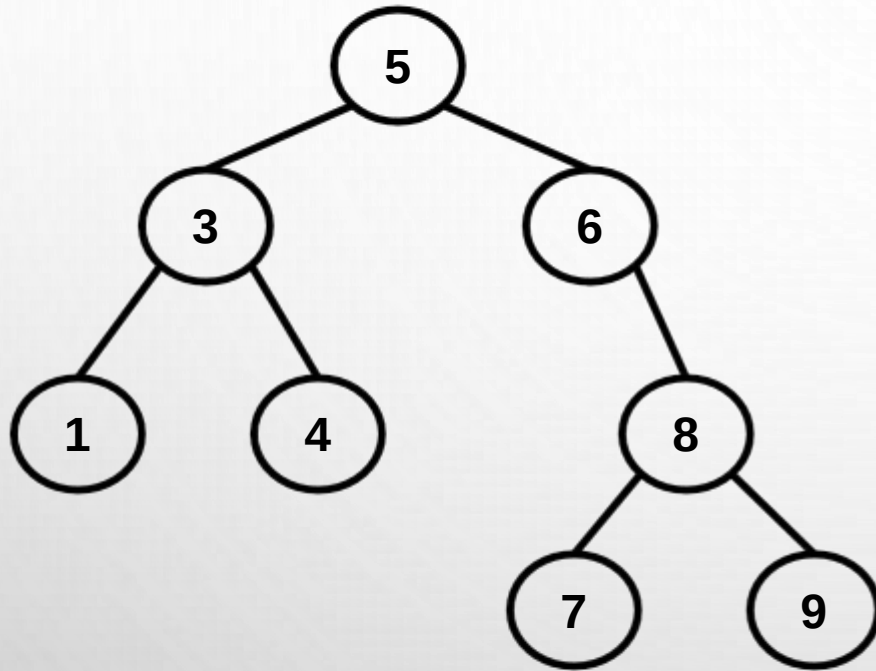
A Binary search tree is a binary tree, such that

- It may be empty
- If it is not empty, then it satisfies the following properties:
 1. Every element has a key and no two elements have the same key (no duplicates)
 2. The keys (if any) in the left subtree are smaller than the key in the root
 3. The keys (if any) in the right subtree are larger than the key in the root
 4. The left and right subtrees are also binary search trees

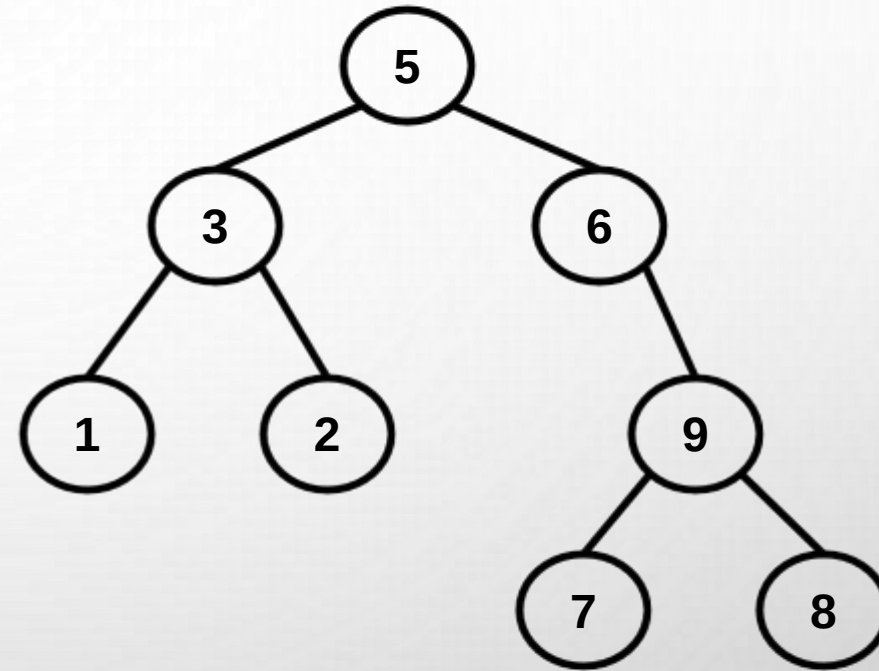
BINARY SEARCH TREES



BINARY SEARCH TREES

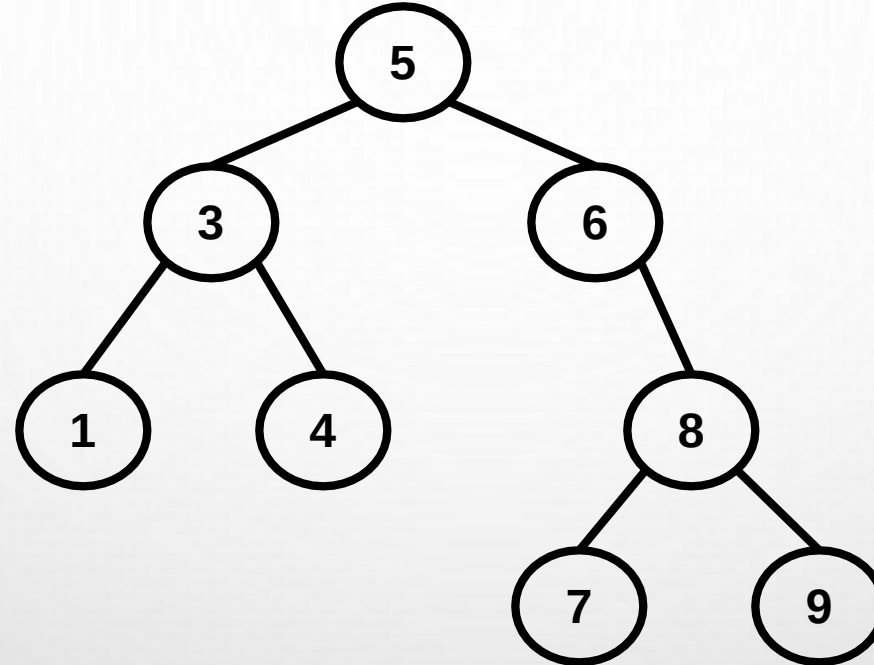


Binary Search Tree



Not a Binary Search Tree

BINARY SEARCH TREES



Inorder Traversal: 1 3 4 5 6 7 8 9

Inorder traversal of BST generate a sorted list

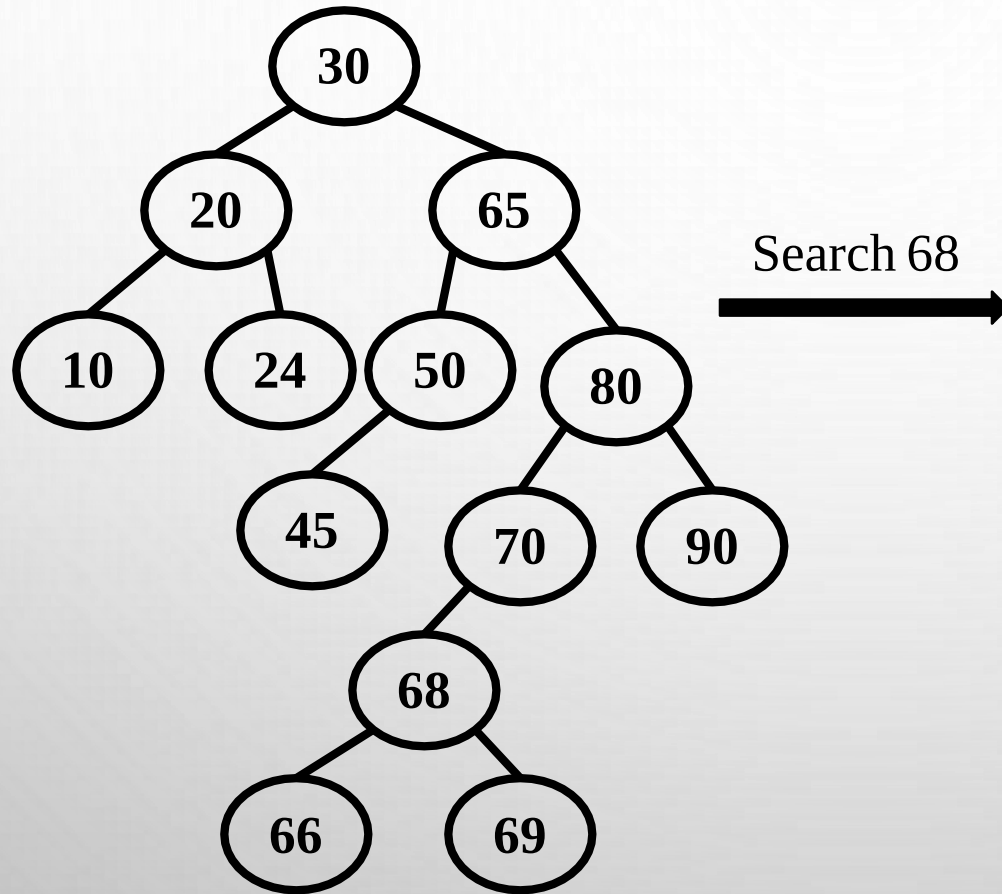
OPERATIONS ON BINARY SEARCH TREES

- Four operations
 - Searching
 - Insertion
 - Deletion
 - Traversal
- The time complexity of searching, insertion and deletion = $O(h)$
where h is the height of the tree.

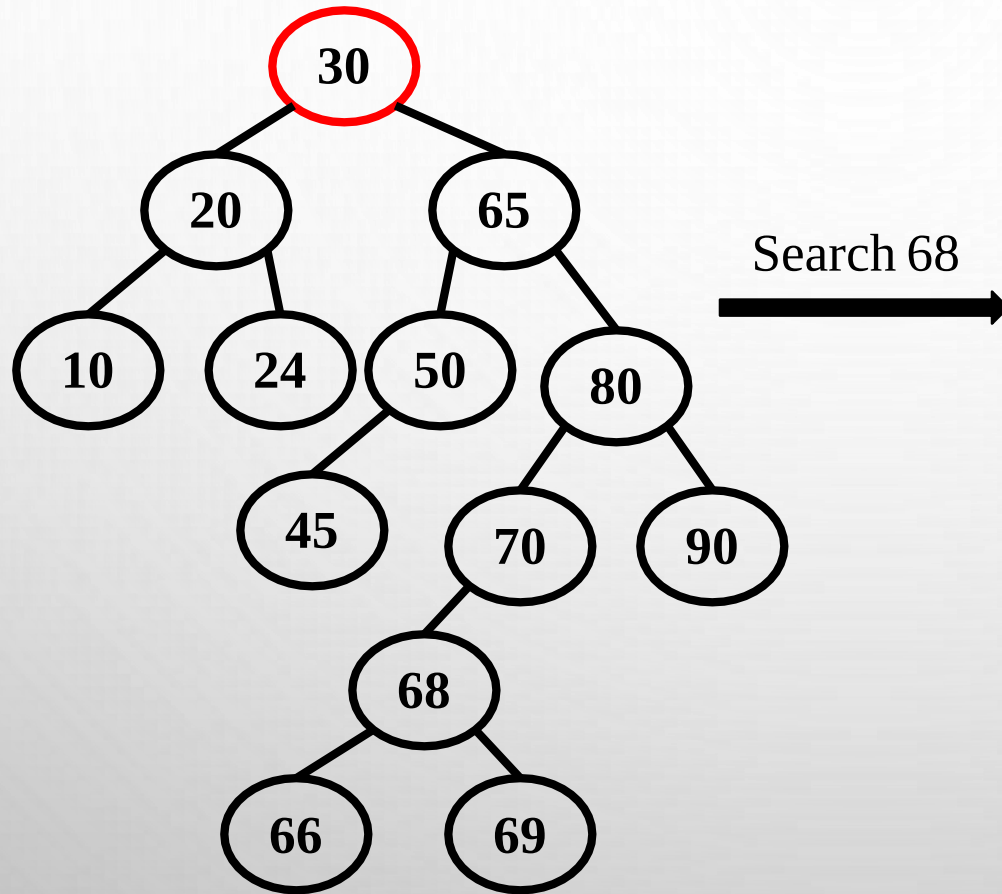
BINARY SEARCH TREES: SEARCHING

- Let item is the value to be searched
- Search begins at root
- If root is NULL search unsuccessful
- Else compare item with root
- If item is less than root then only the left sub tree is to be searched. The sub tree may be searched recursively
- If item is greater than root then only the right sub tree is to be searched. The sub tree may be searched recursively

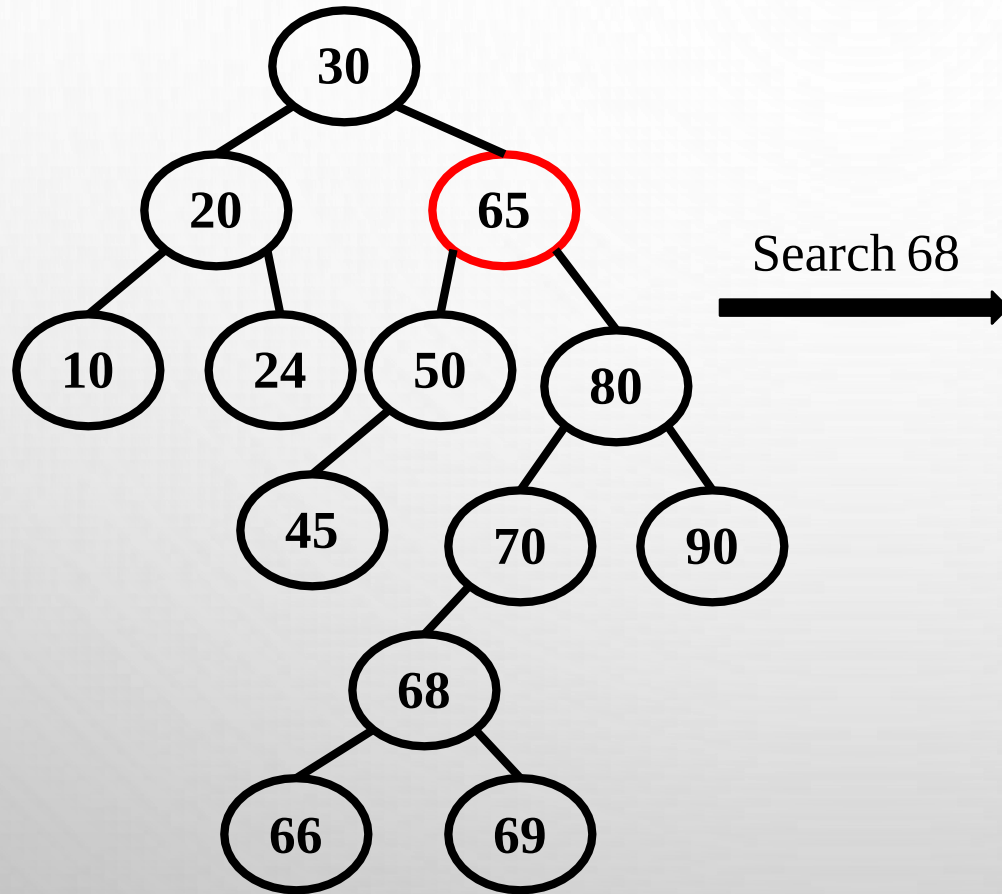
BINARY SEARCH TREES: SEARCHING



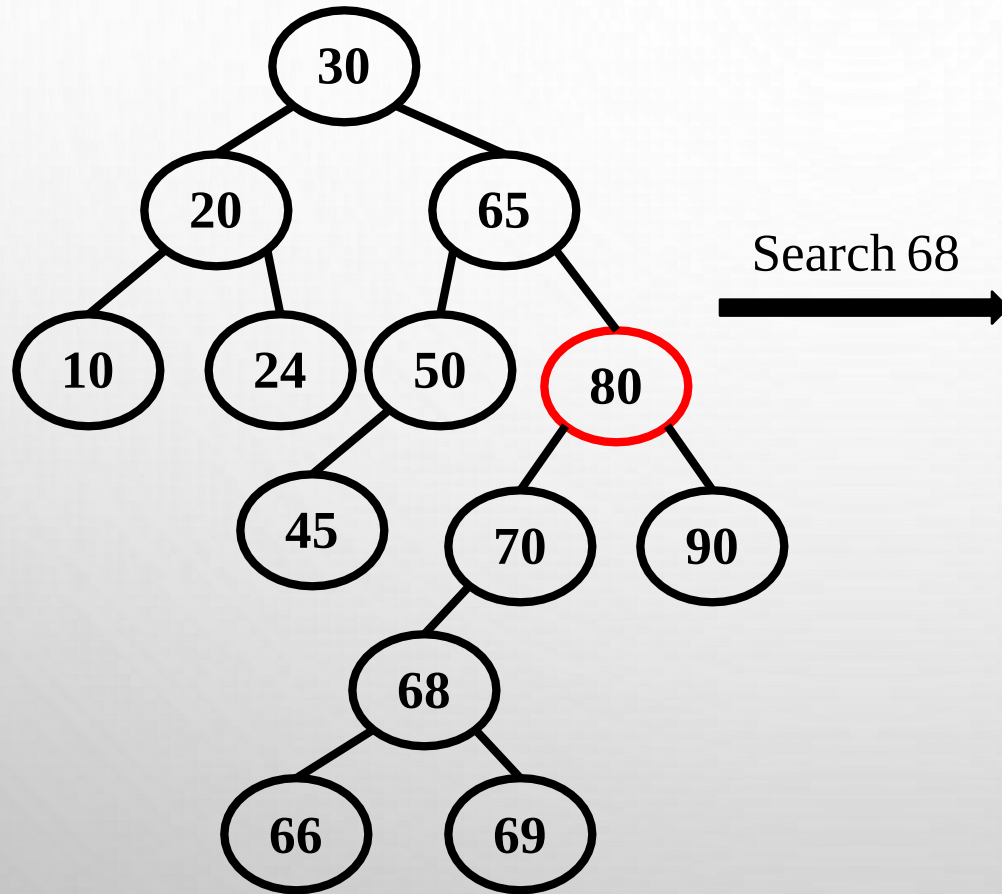
BINARY SEARCH TREES: SEARCHING



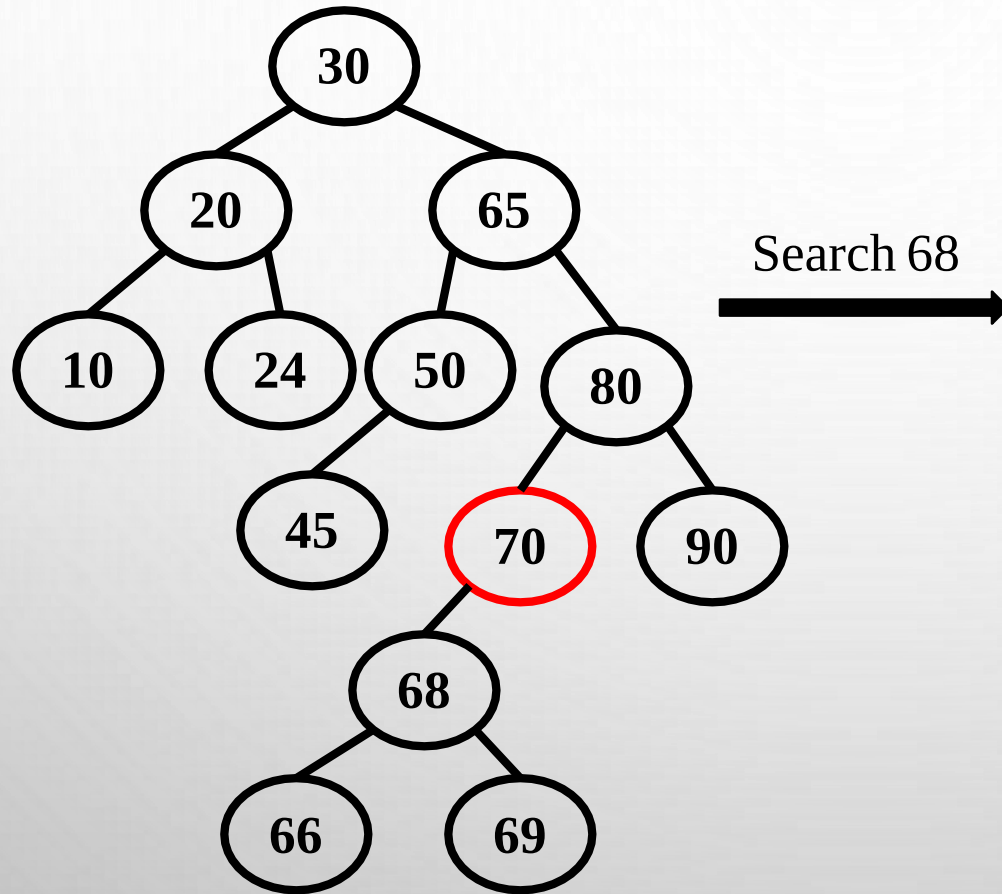
BINARY SEARCH TREES: SEARCHING



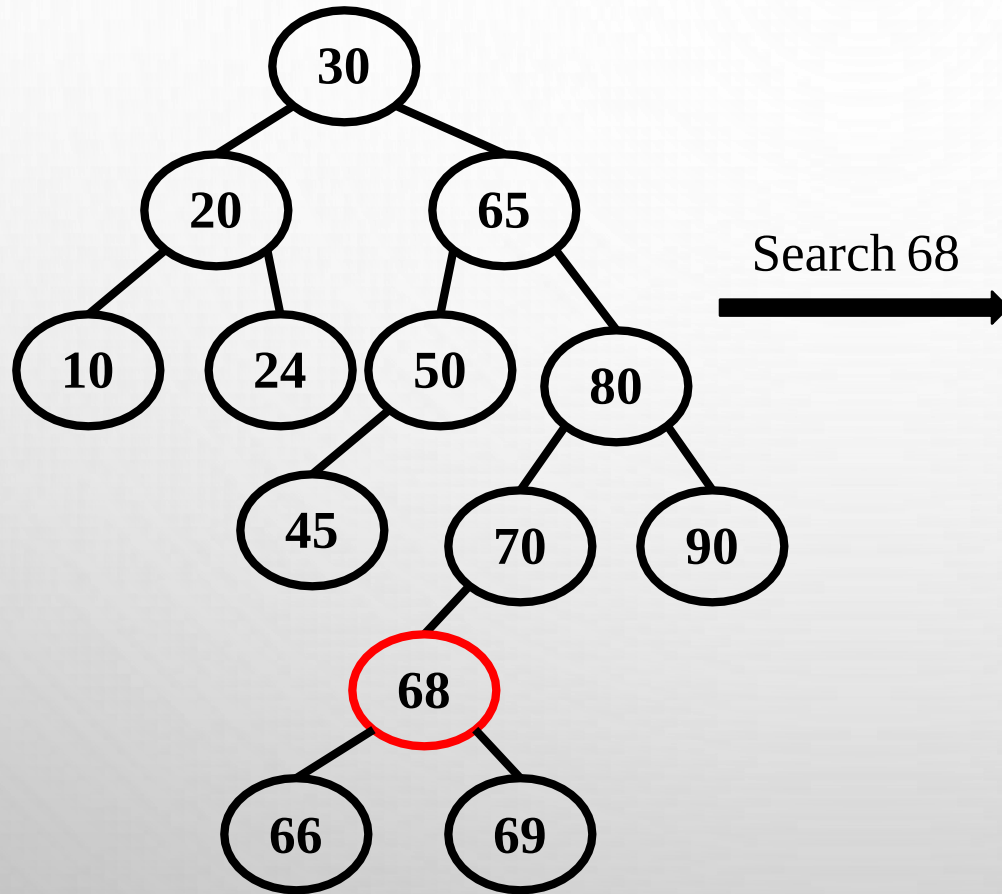
BINARY SEARCH TREES: SEARCHING



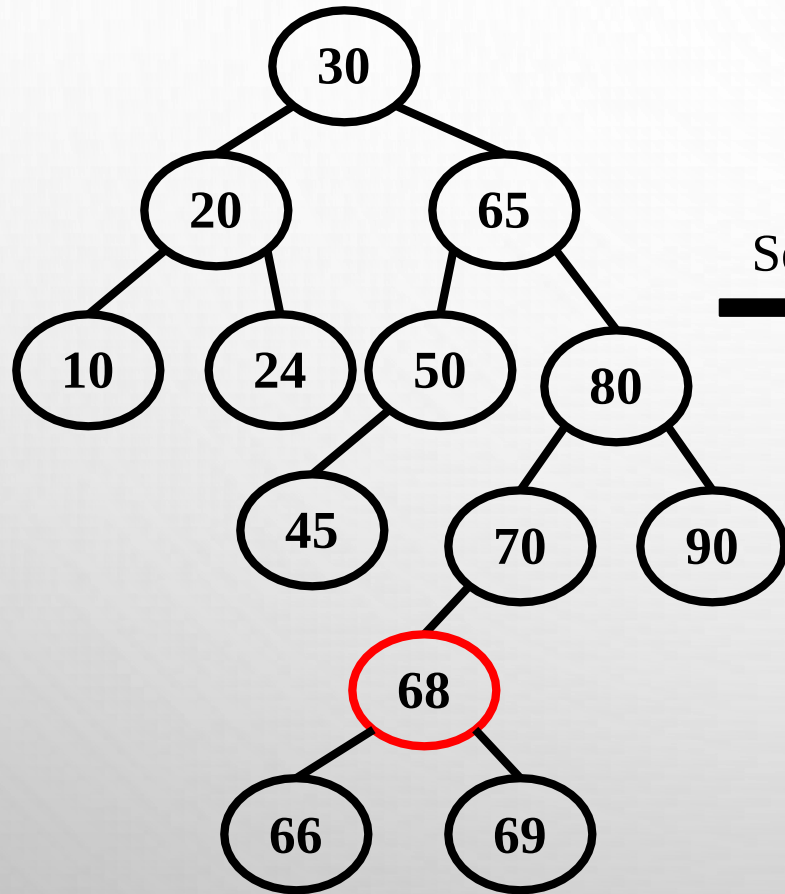
BINARY SEARCH TREES: SEARCHING



BINARY SEARCH TREES: SEARCHING



BINARY SEARCH TREES: SEARCHING

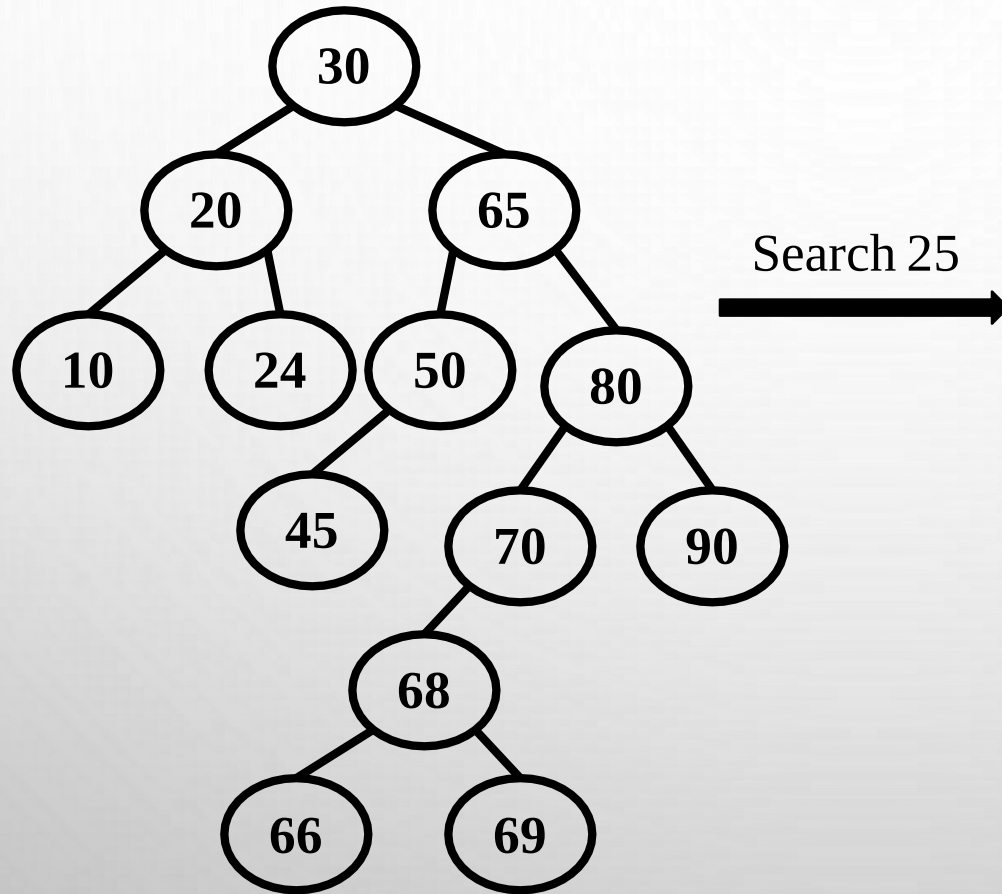


Search 68

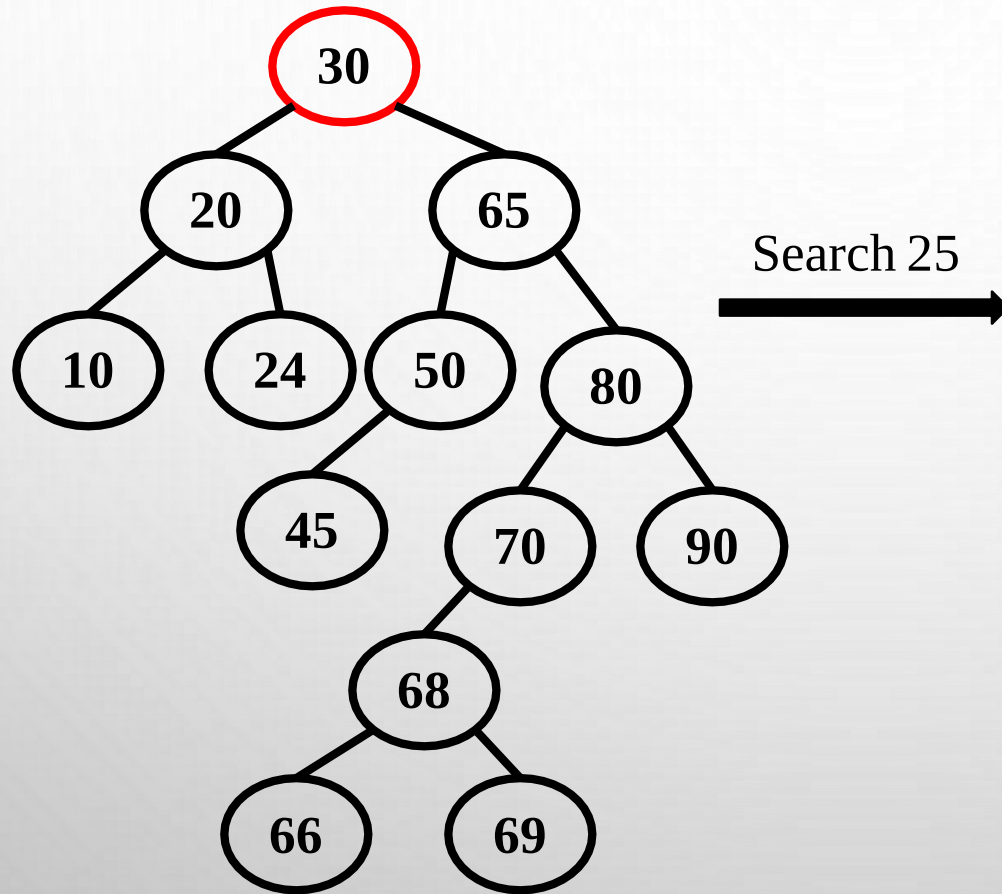


Search Data Found

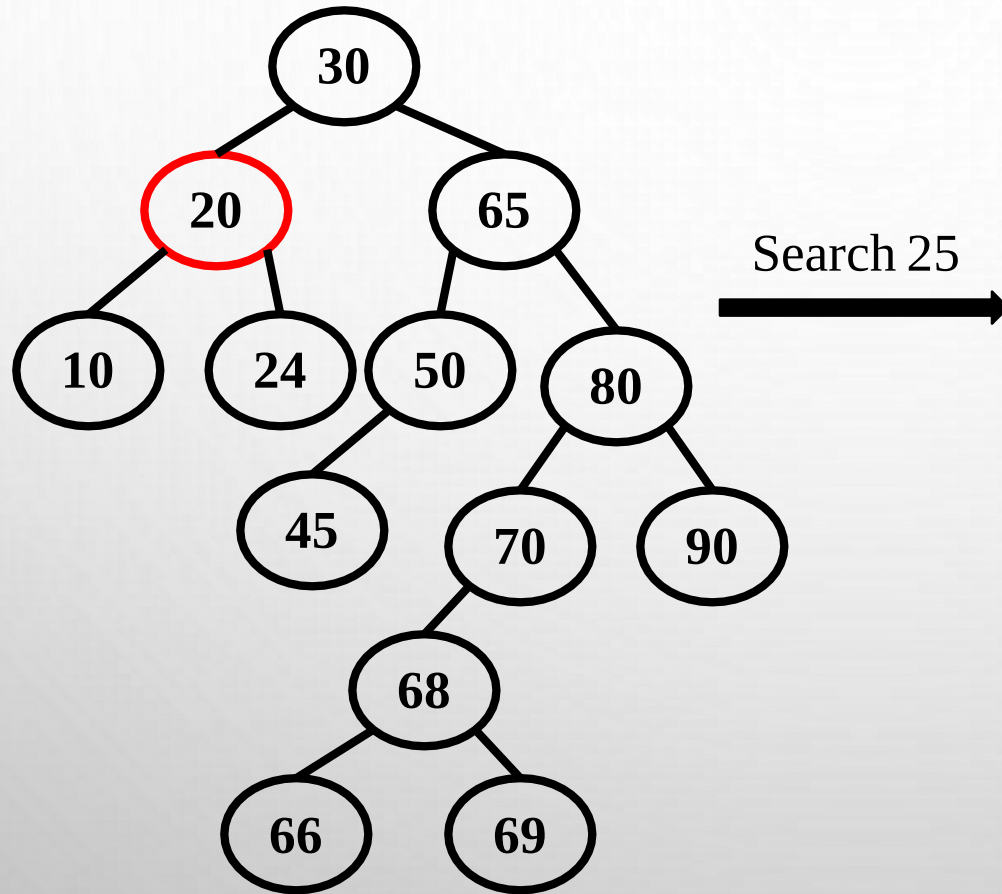
BINARY SEARCH TREES: SEARCHING



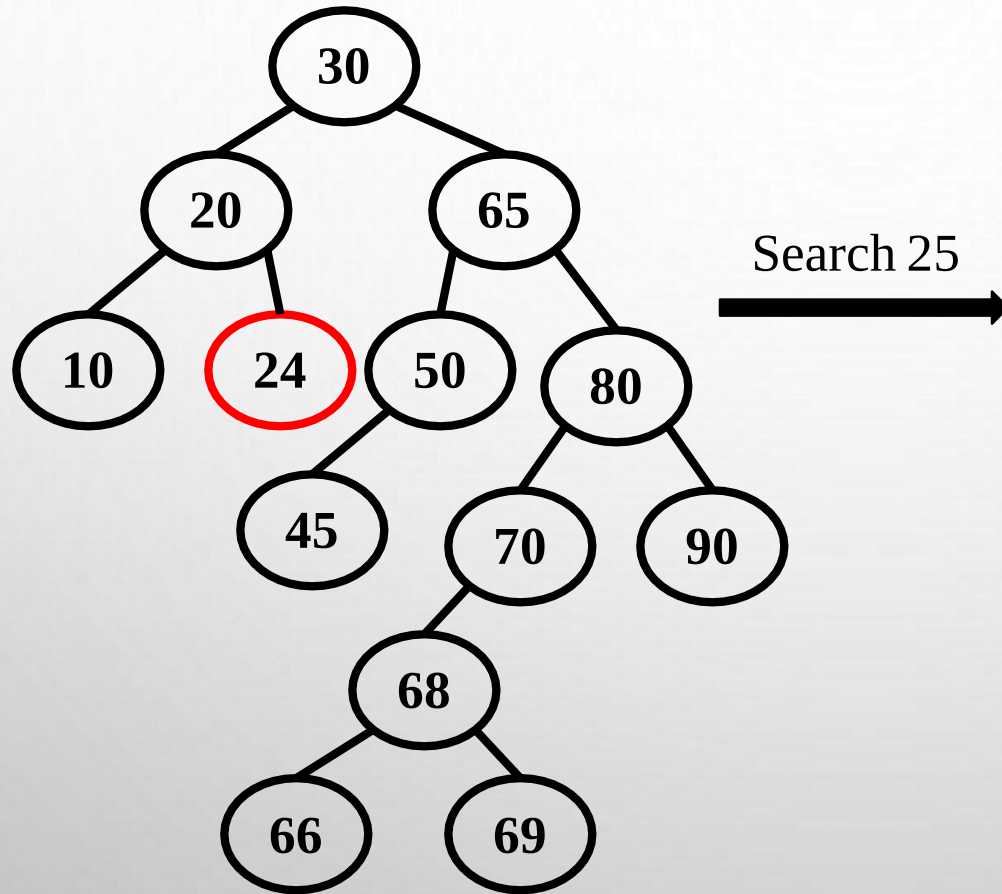
BINARY SEARCH TREES: SEARCHING



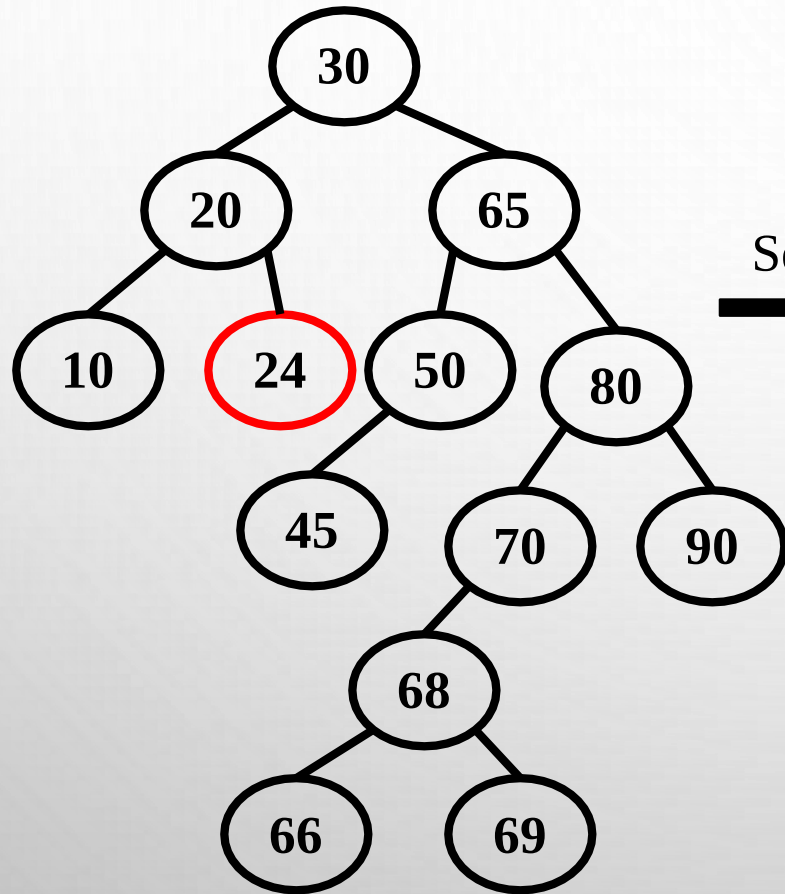
BINARY SEARCH TREES: SEARCHING



BINARY SEARCH TREES: SEARCHING



BINARY SEARCH TREES: SEARCHING



Search 25



Search Data Not Found

Algorithm BST_Search(item)

1. ptr=root
2. flag=0
3. While ptr!=NULL and flag=0 do
 1. If ptr→data=item then
 1. flag=1
 2. Else if item > ptr→data then
 1. ptr=ptr→rChild
 3. Else
 1. ptr=ptr→lChild
4. If flag=1 then
 1. Print "Search data found"
5. Else
 1. Print "Search data not found"

BINARY SEARCH TREES: INSERTION

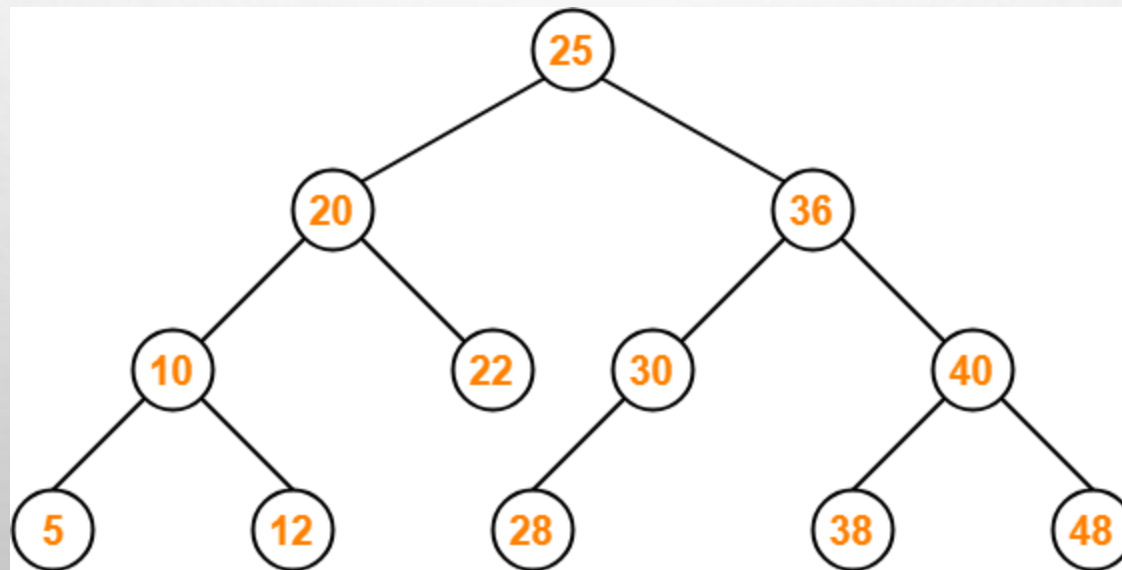
- Suppose „item“ is to be inserted in the BST
- Search for item
- If item is found then do nothing
- Else the item is inserted as left or right child where the search halts

Algorithm BST_Insertion(item)

1. If ROOT=NULL then
 1. Create a node new and insert item in to it. Set it as ROOT node
2. Else
 1. Search item in the tree
 2. If search data found, insertion is not possible
 3. Else
 1. Find the parent node in which the item is to be inserted
 2. Create a node new and insert item in to it.
 3. If $\text{item} < \text{parent} \rightarrow \text{data}$ then
 1. Insert new as the left child of parent
 4. Else
 1. Insert new as the right child of parent

BINARY SEARCH TREE CONSTRUCTION

- In a binary search tree (BST), each node contains:
 - Only smaller values in its left sub tree
 - Only larger values in its right sub tree



EXAMPLE

- Construct a Binary Search Tree (BST) for the following sequence of numbers:

50, 70, 60, 20, 90, 10, 40, 100

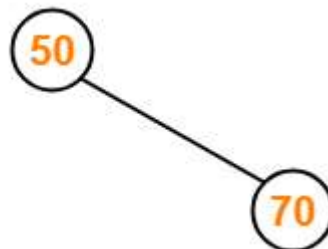
- When elements are given in a sequence,
 - Always consider the first element as the root node.
 - Consider the given elements and insert them in the BST one by one.

Insert 50-



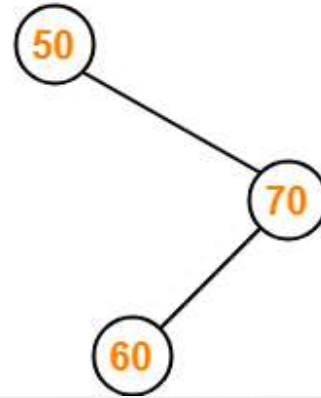
Insert 70-

- As $70 > 50$, so insert 70 to the right of 50.



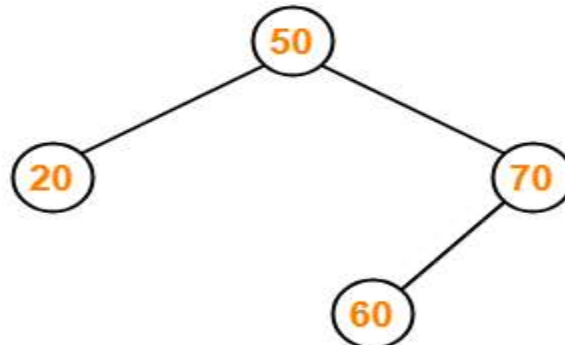
Insert 60-

- As $60 > 50$, so insert 60 to the right of 50.
- As $60 < 70$, so insert 60 to the left of 70.



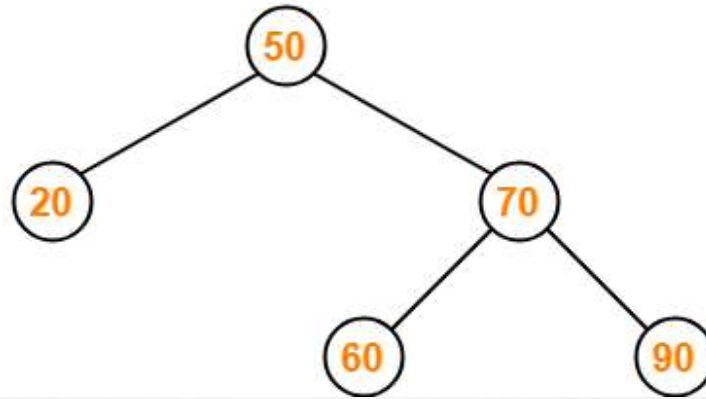
Insert 20-

- As $20 < 50$, so insert 20 to the left of 50.



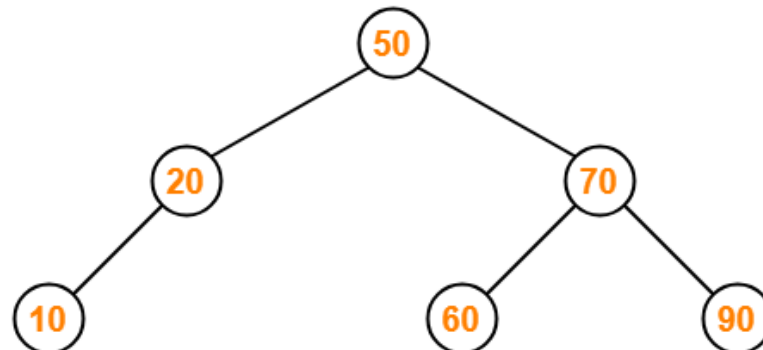
Insert 90-

- As $90 > 50$, so insert 90 to the right of 50.
- As $90 > 70$, so insert 90 to the right of 70.



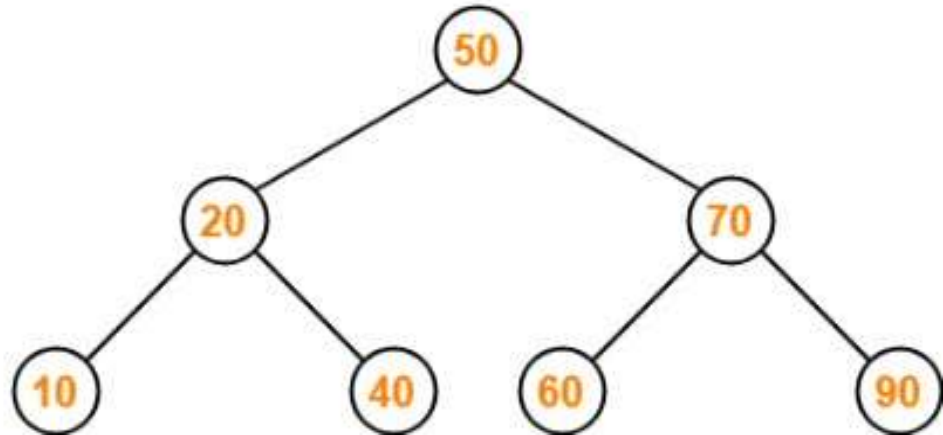
Insert 10-

- As $10 < 50$, so insert 10 to the left of 50.
- As $10 < 20$, so insert 10 to the left of 20.



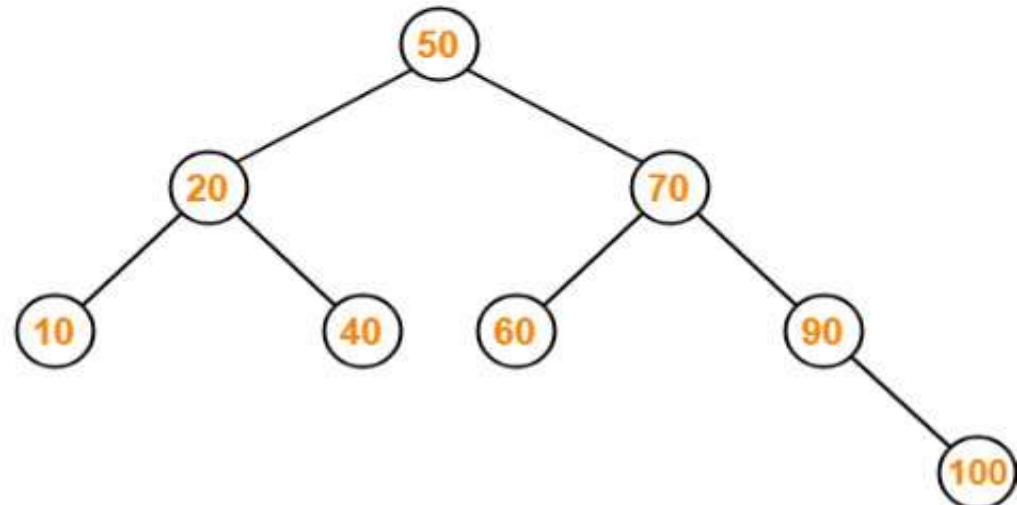
Insert 40-

- As $40 < 50$, so insert 40 to the left of 50.
- As $40 > 20$, so insert 40 to the right of 20.



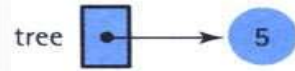
Insert 100-

- As $100 > 50$, so insert 100 to the right of 50.
- As $100 > 70$, so insert 100 to the right of 70.
- As $100 > 90$, so insert 100 to the right of 90.

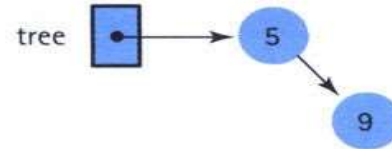


(a) tree 

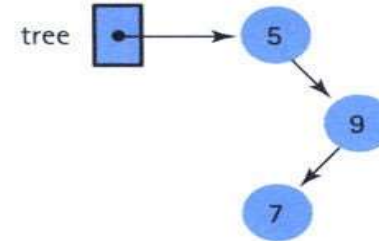
(b) Insert 5



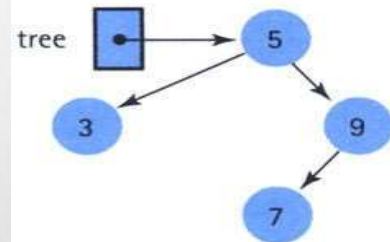
(c) Insert 9



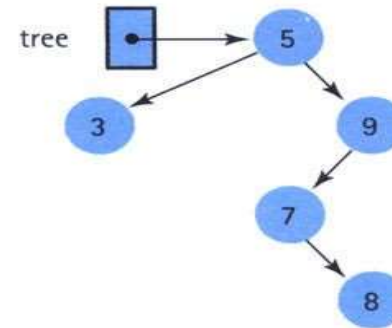
(d) Insert 7



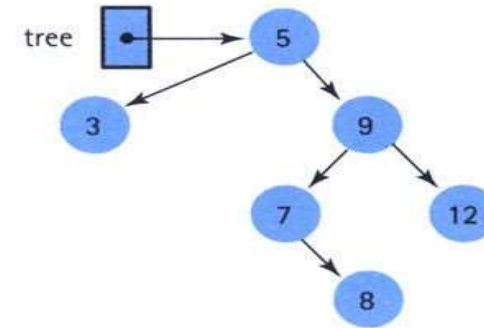
(e) Insert 3



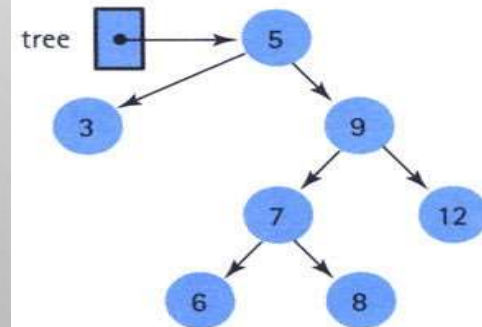
(f) Insert 8



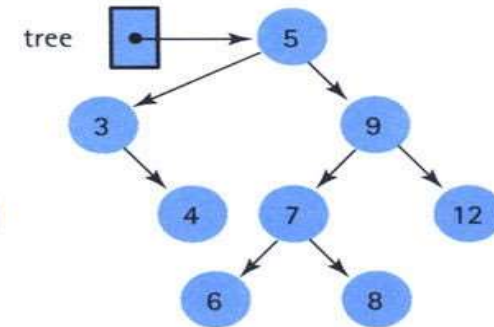
(g) Insert 12



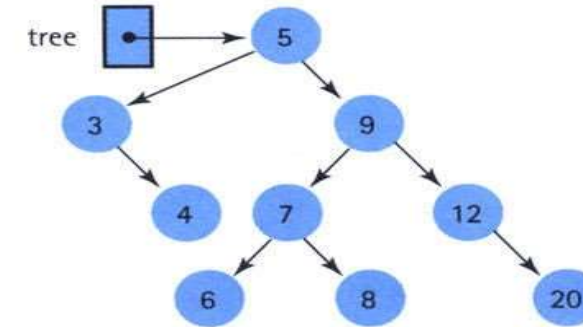
(h) Insert 6



(i) Insert 4

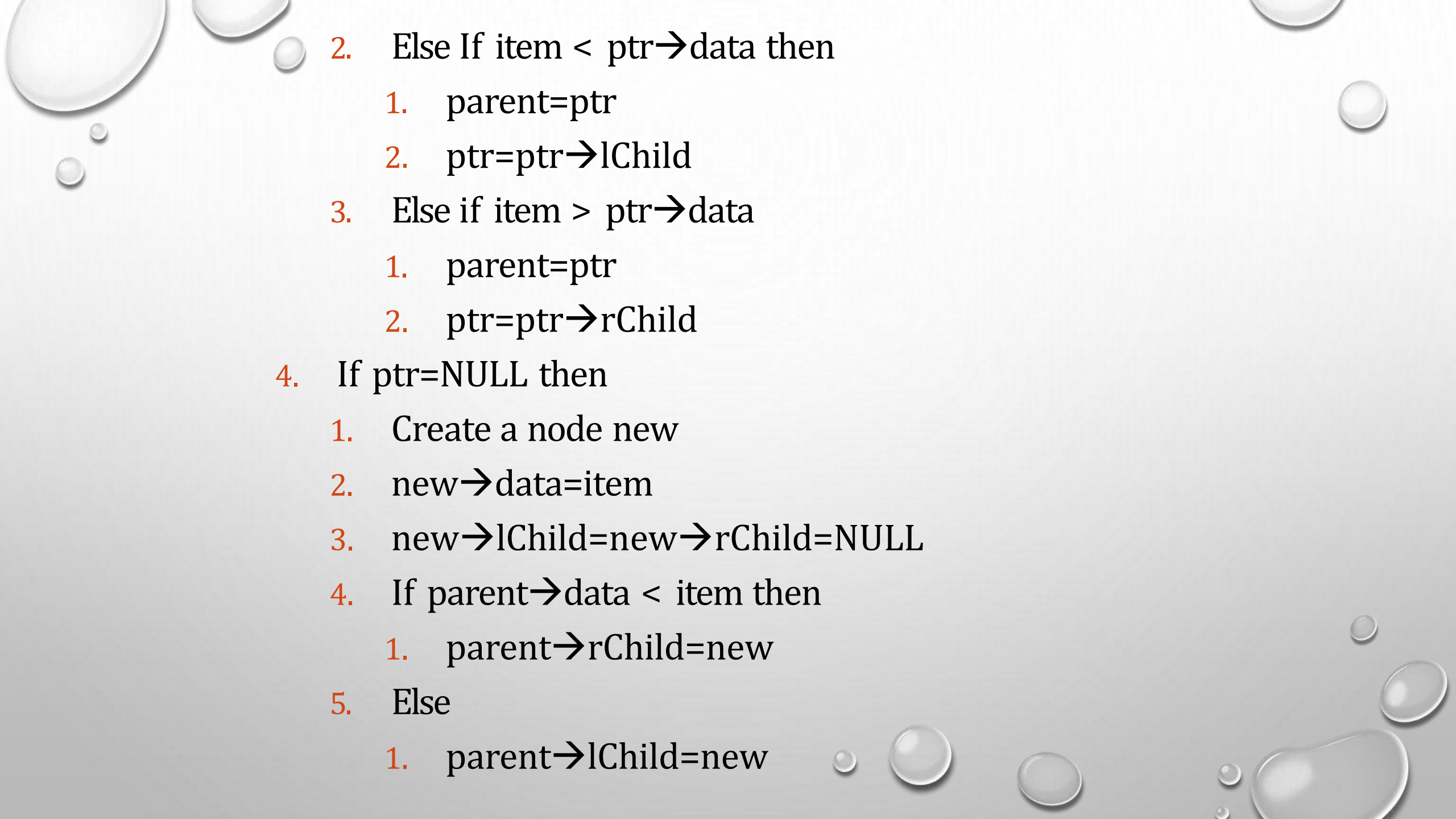


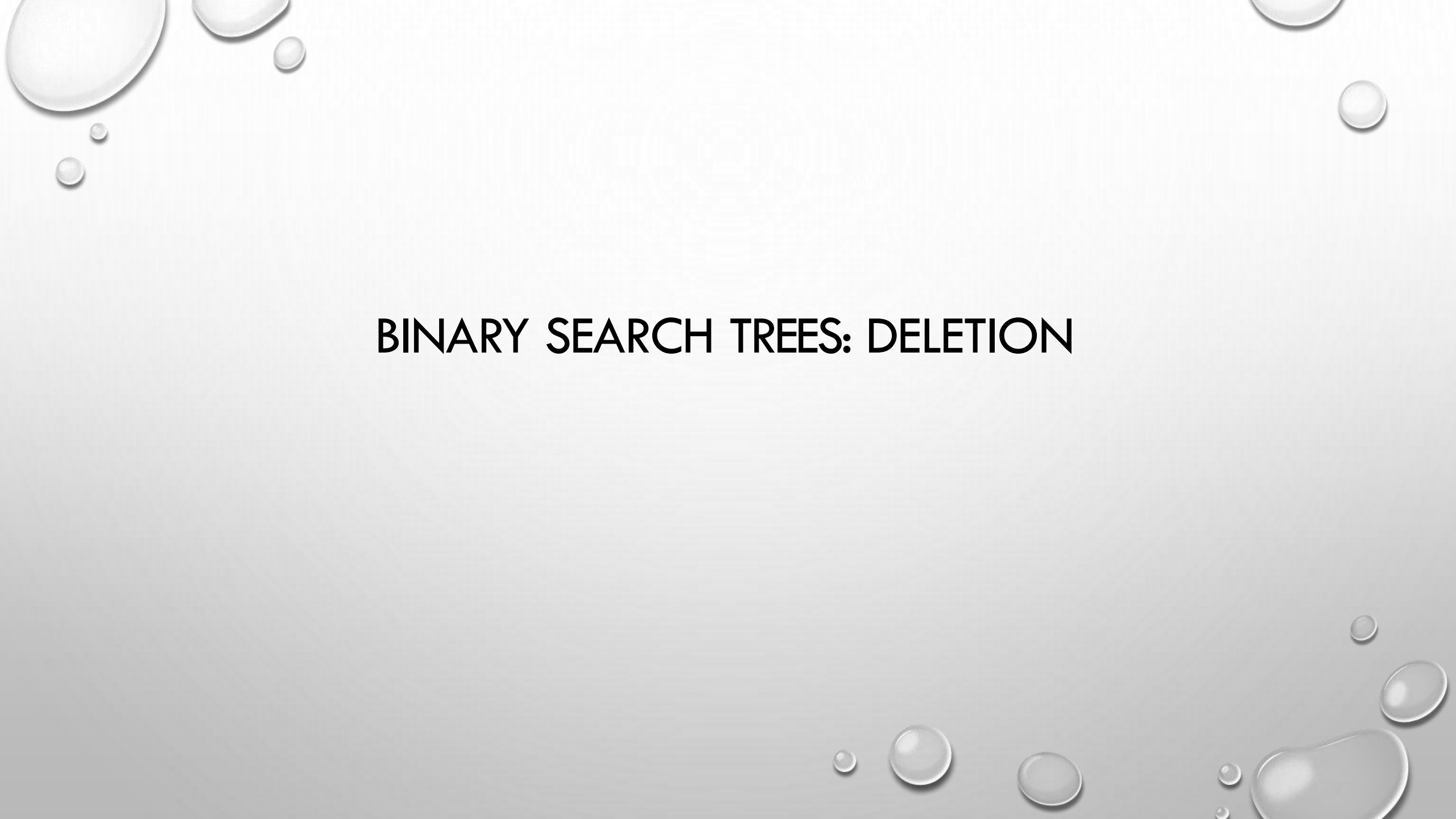
(j) Insert 20



Algorithm BST_Insertion(item)

1. If ROOT=NULL then
 1. Create a node new
 2. new→data=item
 3. new→lChild=new→rChild=NULL
 4. ROOT=new
2. Else
 1. ptr=ROOT
 2. flag=0
 3. While ptr!=NULL and flag=0 do
 1. If item = ptr→data then
 1. flag=1
 2. Print “item al ready exist”
 3. Exit

- 
2. Else If $\text{item} < \text{ptr} \rightarrow \text{data}$ then
 1. $\text{parent} = \text{ptr}$
 2. $\text{ptr} = \text{ptr} \rightarrow \text{lChild}$
 3. Else if $\text{item} > \text{ptr} \rightarrow \text{data}$
 1. $\text{parent} = \text{ptr}$
 2. $\text{ptr} = \text{ptr} \rightarrow \text{rChild}$
 4. If $\text{ptr} = \text{NULL}$ then
 1. Create a node new
 2. $\text{new} \rightarrow \text{data} = \text{item}$
 3. $\text{new} \rightarrow \text{lChild} = \text{new} \rightarrow \text{rChild} = \text{NULL}$
 4. If $\text{parent} \rightarrow \text{data} < \text{item}$ then
 1. $\text{parent} \rightarrow \text{rChild} = \text{new}$
 5. Else
 1. $\text{parent} \rightarrow \text{lChild} = \text{new}$

The background of the slide is a light gray gradient. It is decorated with several realistic water droplets of various sizes. In the top-left corner, there are several droplets, including a large one. In the top-right corner, there are a few smaller droplets. In the bottom-right corner, there is a cluster of droplets, including a large one. In the bottom-center, there are a few more droplets.

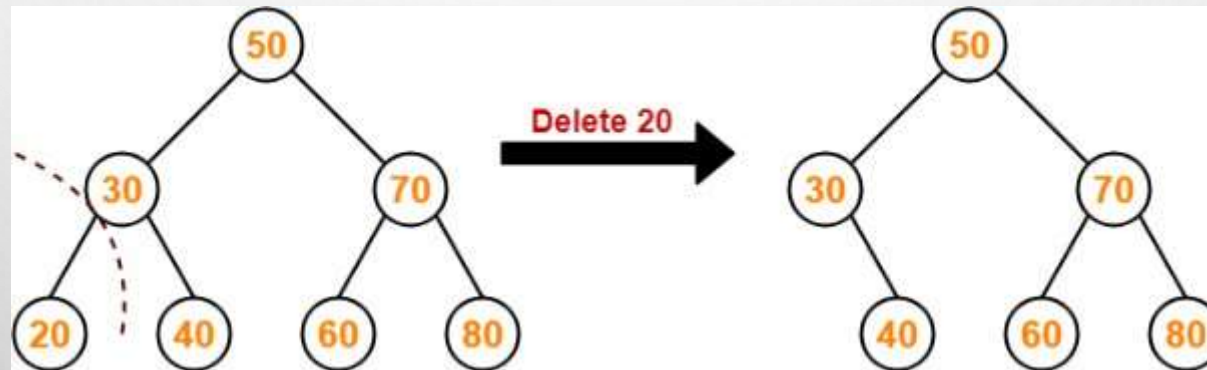
BINARY SEARCH TREES: DELETION

DELETION OPERATION

- Deletion Operation is performed to delete a particular element from the Binary Search Tree.
- When it comes to deleting a node from the binary search tree, three cases are possible.

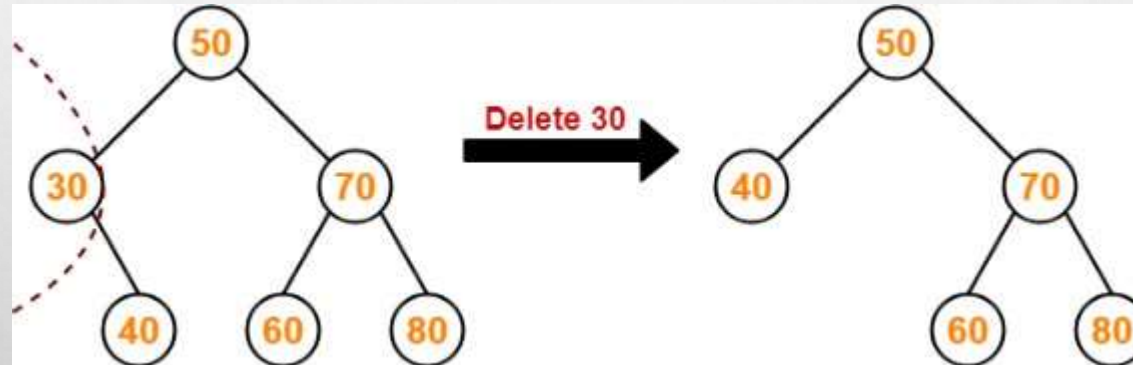
CASE-01: DELETION OF A NODE HAVING NO CHILD (LEAF NODE)

- Just remove / disconnect the leaf node that is to be deleted from the tree.



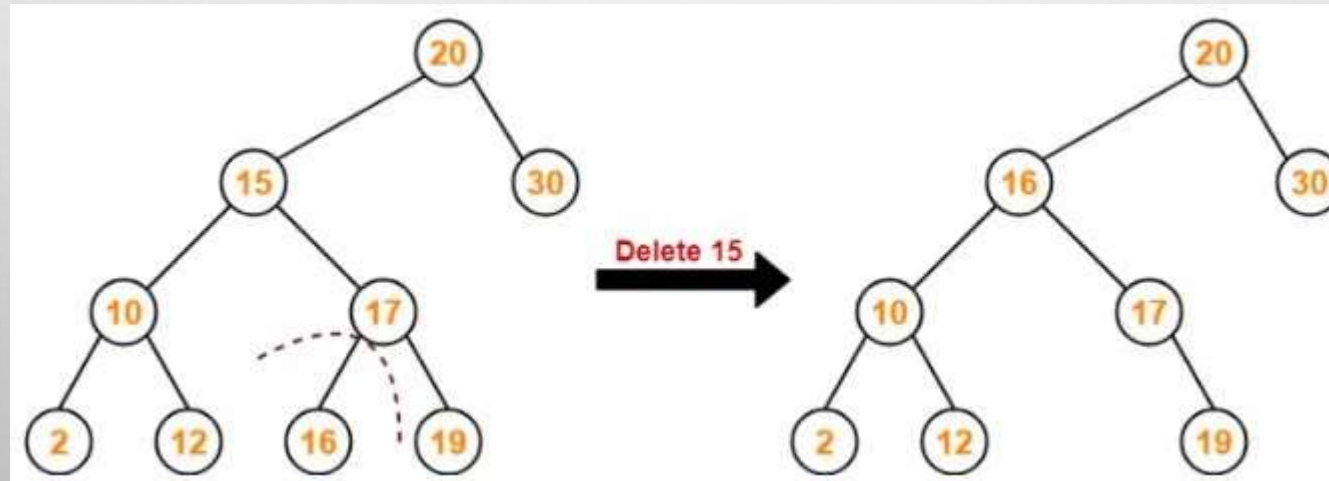
CASE-02: DELETION OF A NODE HAVING ONLY ONE CHILD

- Consider the following example where node with value = 30 is deleted from the BST.



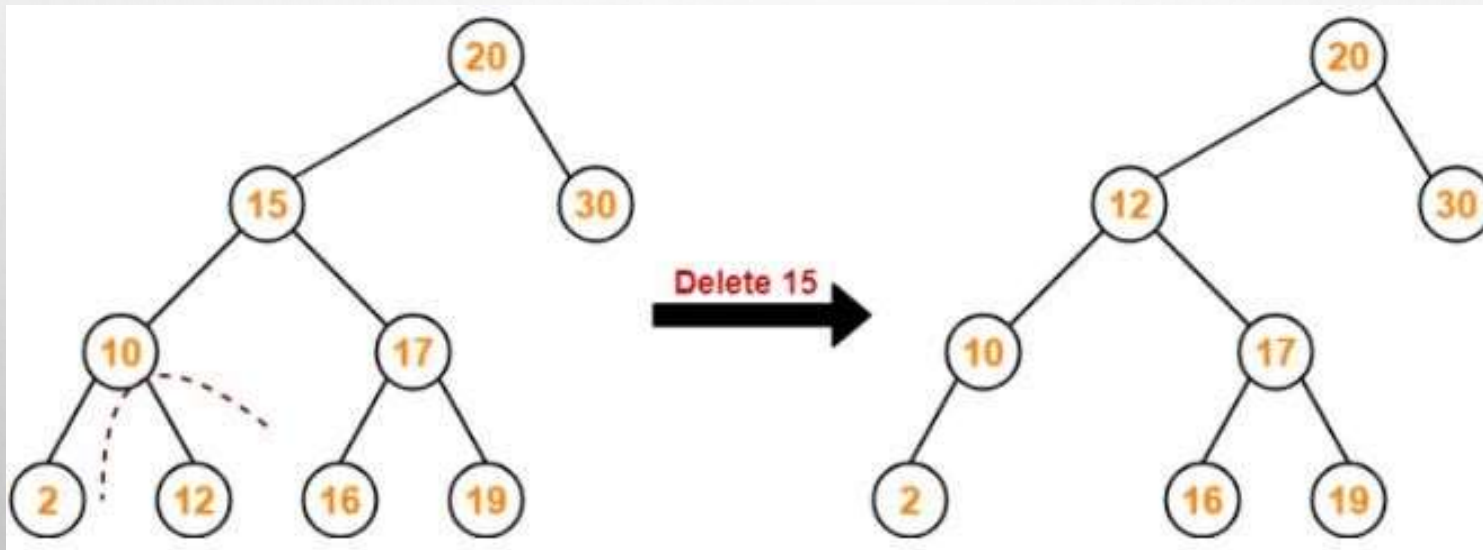
CASE-03: DELETION OF A NODE HAVING TWO CHILDREN

- Consider the following example where node with value = 15 is deleted from the BST
- Method-1:
 - Visit to the right subtree of the deleting node.
 - Pluck the least value element called as inorder successor.
 - Replace the deleting element with its inorder successor.



- Method-2:

- Visit to the left subtree of the deleting node.
- Pluck the greatest value element called as inorder successor.
- Replace the deleting element with its inorder successor.



BINARY SEARCH TREES: DELETION

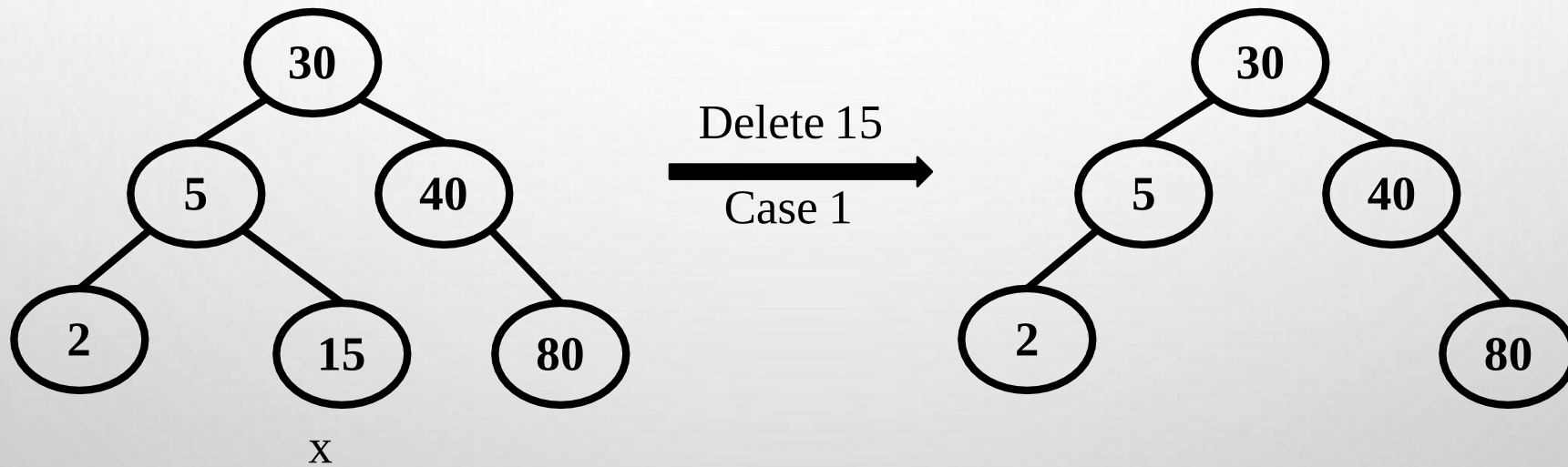
Algorithm BST_Deletion()

1. Let x be a node to be deleted
2. If x is a leaf node then
 1. Delete x
3. Else if x is a node with only one child then
 1. Replace x with the child node of x
 2. Delete x
4. Else if x having two children then
 1. Find the inorder successor/predecessor of x , say y
 2. Find the right/left child of y , say z
 3. Copy the data from y to x
 4. Replace y by z

BINARY SEARCH TREES: DELETION

Case 1: x is a leaf node

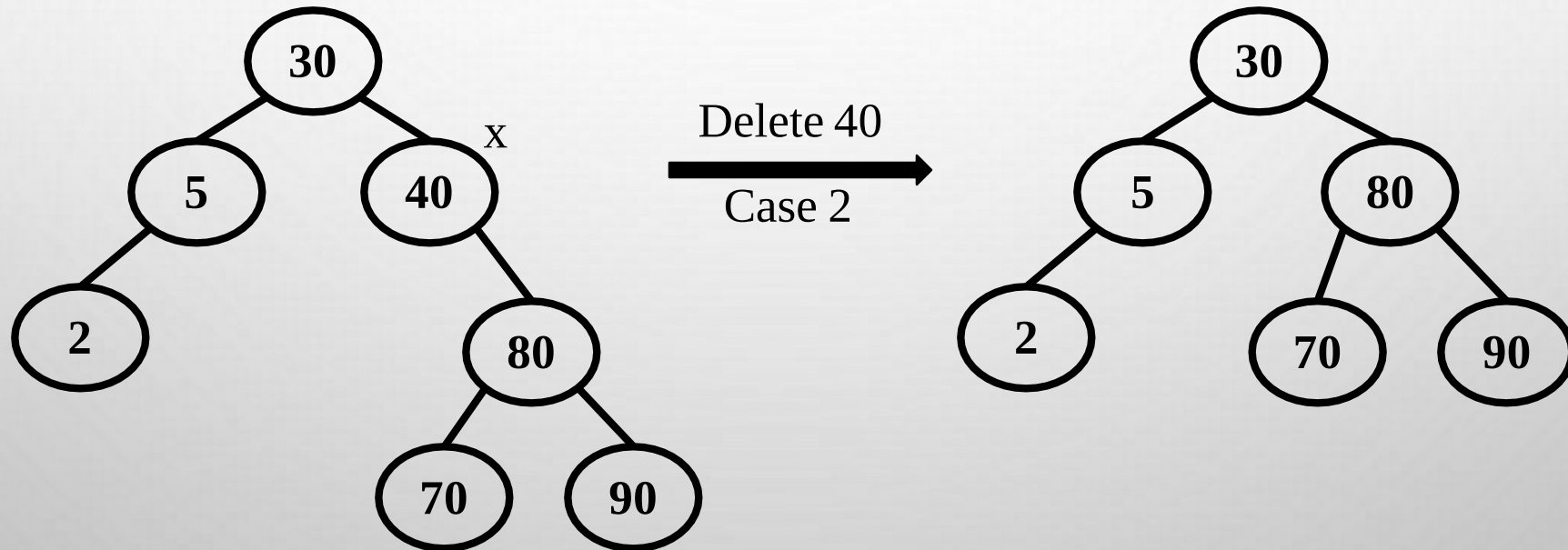
- Delete x



BINARY SEARCH TREES: DELETION

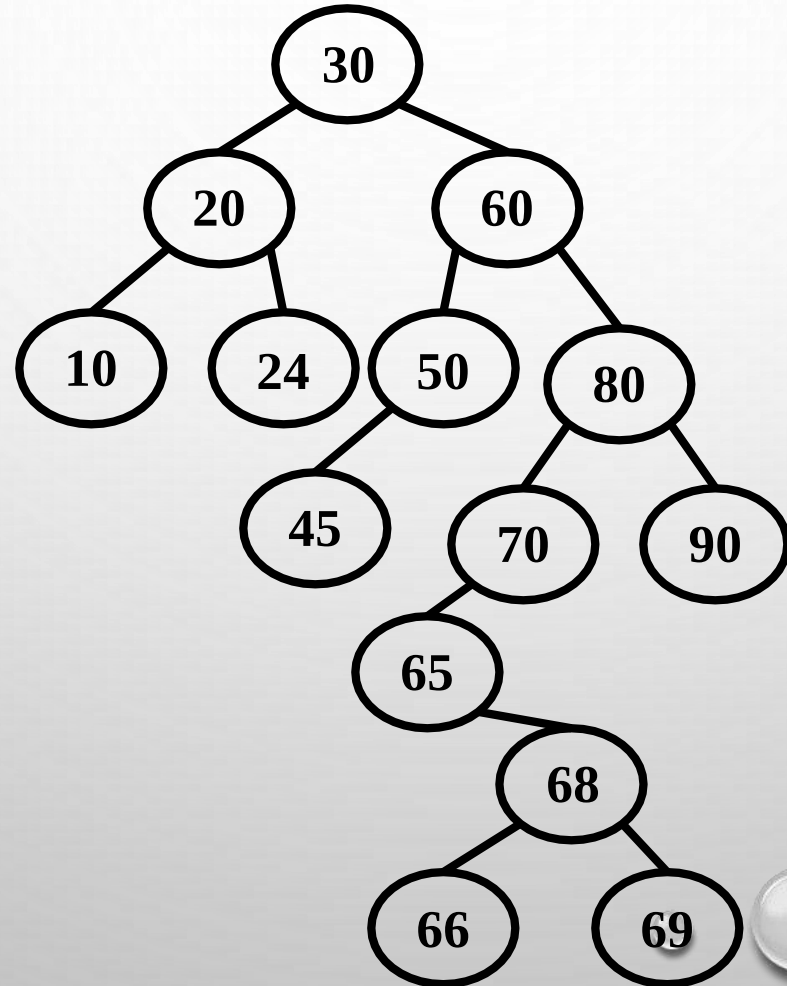
Case 2: x is a node with only one child

- Replace x with the child node of x
- Delete x



Case 3: x having two children

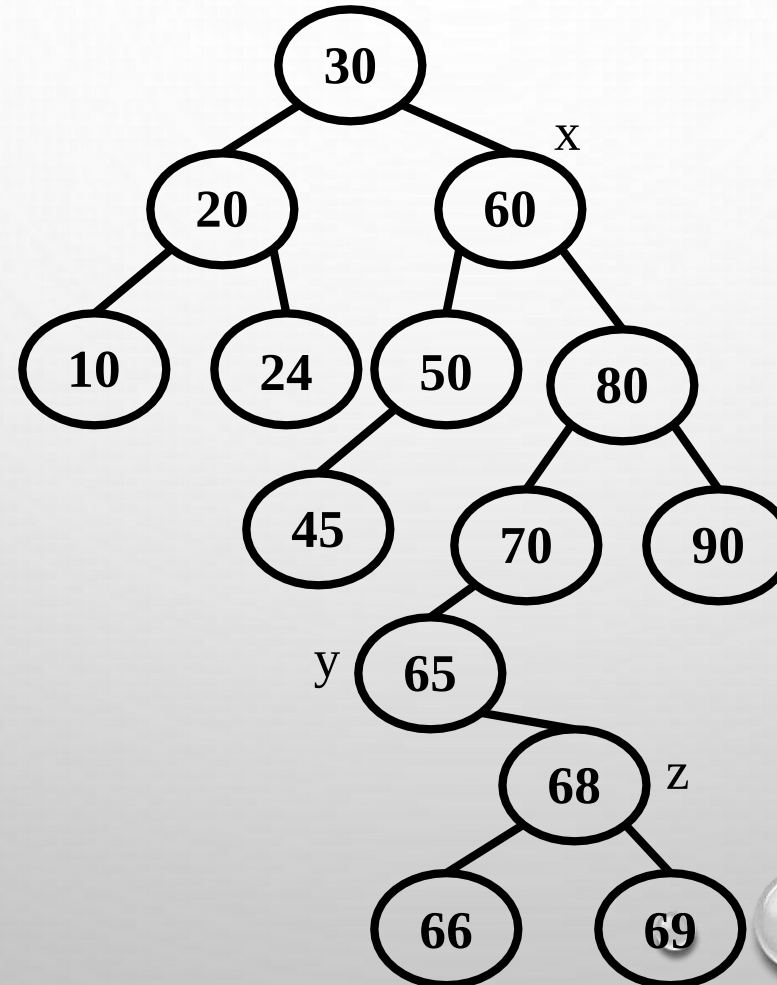
- Find the inorder successor, say y
- Find the right, say z
- Copy the data from y to x
- Replace y by z



Delete 60

Case 3: x having two children

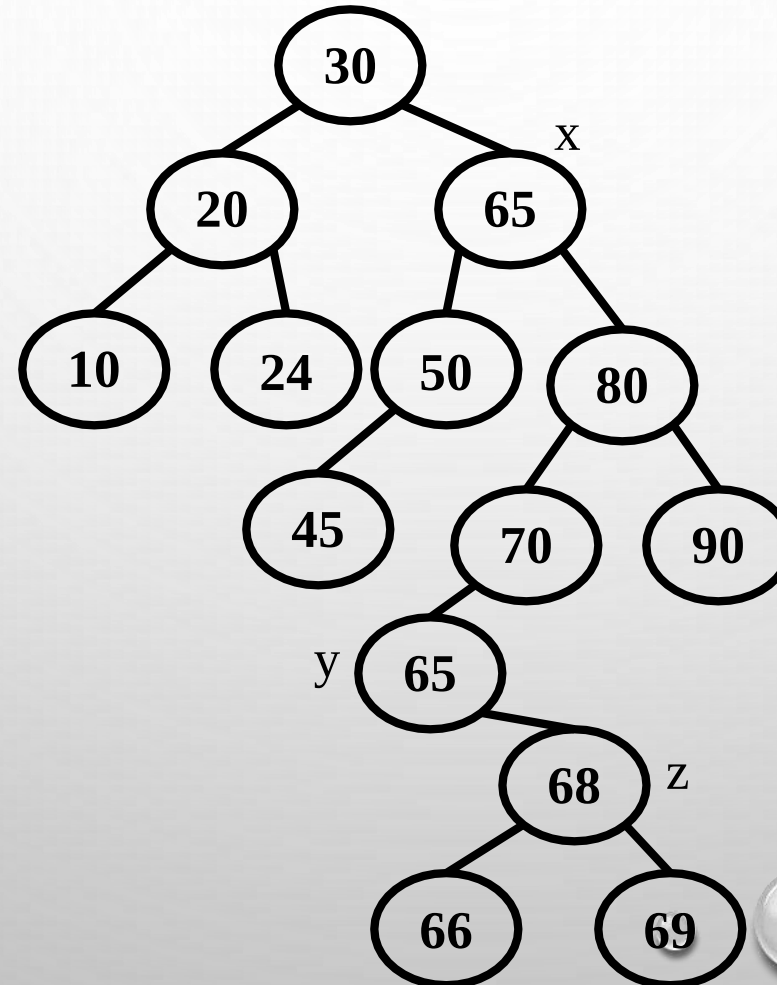
- Find the inorder successor, say y
- Find the right, say z
- Copy the data from y to x
- Replace y by z



Delete 60

Case 3: x having two children

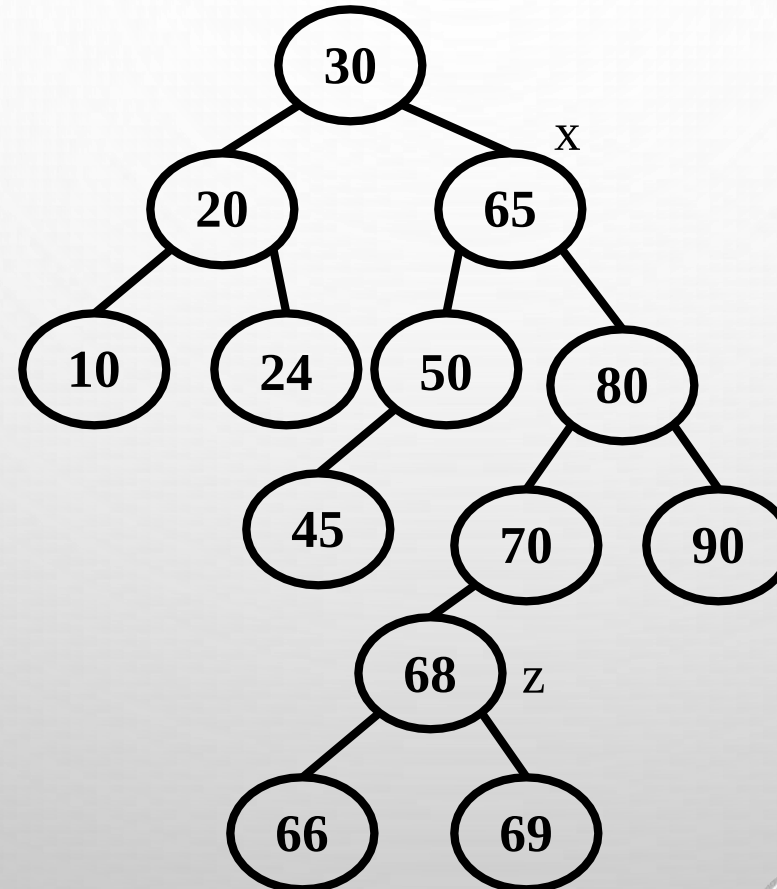
- Find the inorder successor, say y
- Find the right, say z
- Copy the data from y to x
- Replace y by z



Delete 60

Case 3: x having two children

- Find the inorder successor, say y
- Find the right, say z
- Copy the data from y to x
- Replace y by z



Delete 60

Algorithm BST_Deletion(item)

1. ptr=ROOT, flag=0
2. While ptr!=NULL and flag=0 do
 1. If item < ptr→data then
 1. parent=ptr
 2. ptr=ptr→lChild
 2. Else if item > ptr→data then
 1. parent=ptr
 2. ptr=ptr→rChild
 3. Else
 1. flag=1
3. If (flag==0)
 1. Print "ITEM does not exist"
 2. Exit

4. Else

1. If $\text{ptr} \rightarrow \text{rChild} = \text{NULL}$ and $\text{ptr} \rightarrow \text{lChild} = \text{NULL}$ then
 1. If $\text{parent} \rightarrow \text{lChild} = \text{ptr}$ then
 1. $\text{parent} \rightarrow \text{lChild} = \text{NULL}$
 2. Else
 1. $\text{parent} \rightarrow \text{rChild} = \text{NULL}$
2. Else if $\text{ptr} \rightarrow \text{lChild} \neq \text{NULL}$ and $\text{ptr} \rightarrow \text{rChild} \neq \text{NULL}$ then
 1. y is inorder successor of ptr
 2. p is the parent of y
 3. z is the right child of y
 4. Copy the data from y to ptr
 5. $\text{parent} \rightarrow \text{lChild} = \text{z}$
 6. Dispose(y)

3. Else

1. If $\text{parent} \rightarrow \text{lChild} = \text{ptr}$ then

1. If $\text{ptr} \rightarrow \text{lChild} \neq \text{NULL}$ then

1. $\text{parent} \rightarrow \text{lChild} = \text{ptr} \rightarrow \text{lChild}$

2. Else

1. $\text{parent} \rightarrow \text{lChild} = \text{ptr} \rightarrow \text{rChild}$

2. Else

1. If $\text{ptr} \rightarrow \text{lChild} \neq \text{NULL}$ then

1. $\text{parent} \rightarrow \text{rChild} = \text{ptr} \rightarrow \text{lChild}$

2. Else

1. $\text{parent} \rightarrow \text{rChild} = \text{ptr} \rightarrow \text{rChild}$

BINARY SEARCH TREES: APPLICATIONS

- Remove duplicate values
 - Consider a collection of n data items $A_1, A_2, \dots, A_N$. Suppose we want to find and delete all duplicates in the collection. Then, we can represent them as BST and remove the duplicate values, while it occurs.
- Find the smallest data- Traverse the left subtree of BST
- Find the largest data- Traverse the right subtree of BST
- Find a particular data- Traverse the whole BST

3. AVL TREE

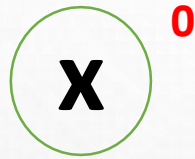
- An AVL(Adelson-Velskii and Landis) tree is a self-balancing binary search tree in Data structure

Characteristics

- **BST** : Left child < Parent < Right child
- **Self-Balancing** : Ensures tree remains balanced
- **Height-Balanced** : For each node in tree the difference between
 $|\text{height of left subtree}| - |\text{height of right subtree}| = \{-1, 0, 1\}$
- This difference is **Balance Factor**

- Suppose we have x, y, z

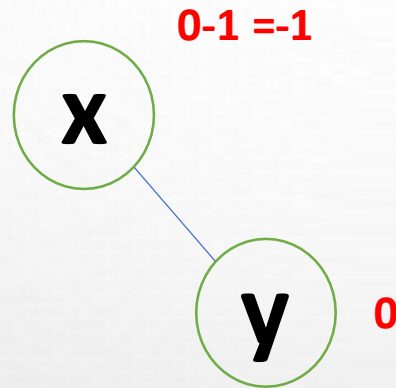
EXAMPLE



Balance Factor

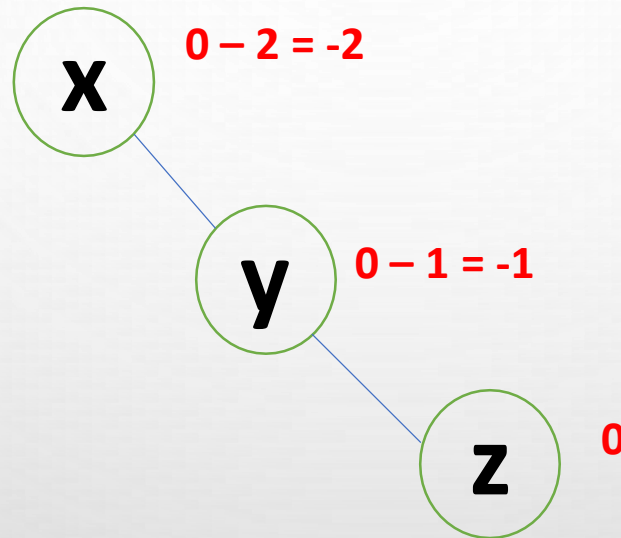
- Suppose we have x, y, z

EXAMPLE Left subtree – right subtree



- Suppose we have x, y, z

EXAMPLE



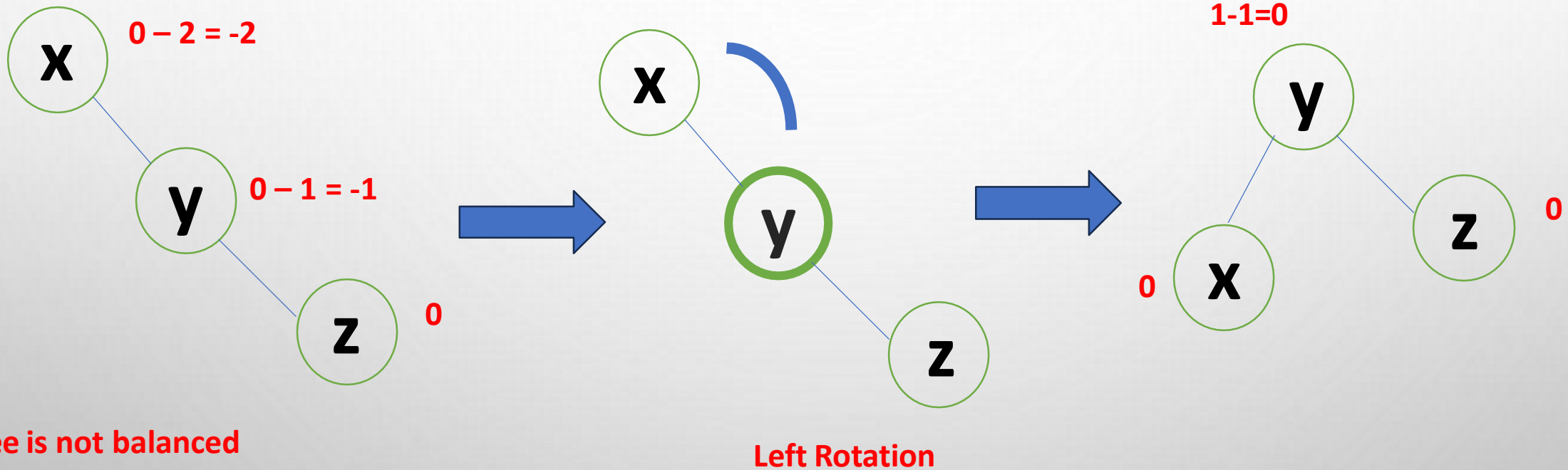
Tree is not balanced

ROTATING THE SUBTREES IN AN AVL TREE:

- AN AVL TREE MAY ROTATE IN ONE OF THE FOLLOWING FOUR WAYS TO KEEP ITSELF BALANCED:
- LEFT ROTATION
- RIGHT ROTATION
- LEFT-RIGHT ROTATION
- RIGHT-LEFT ROTATION

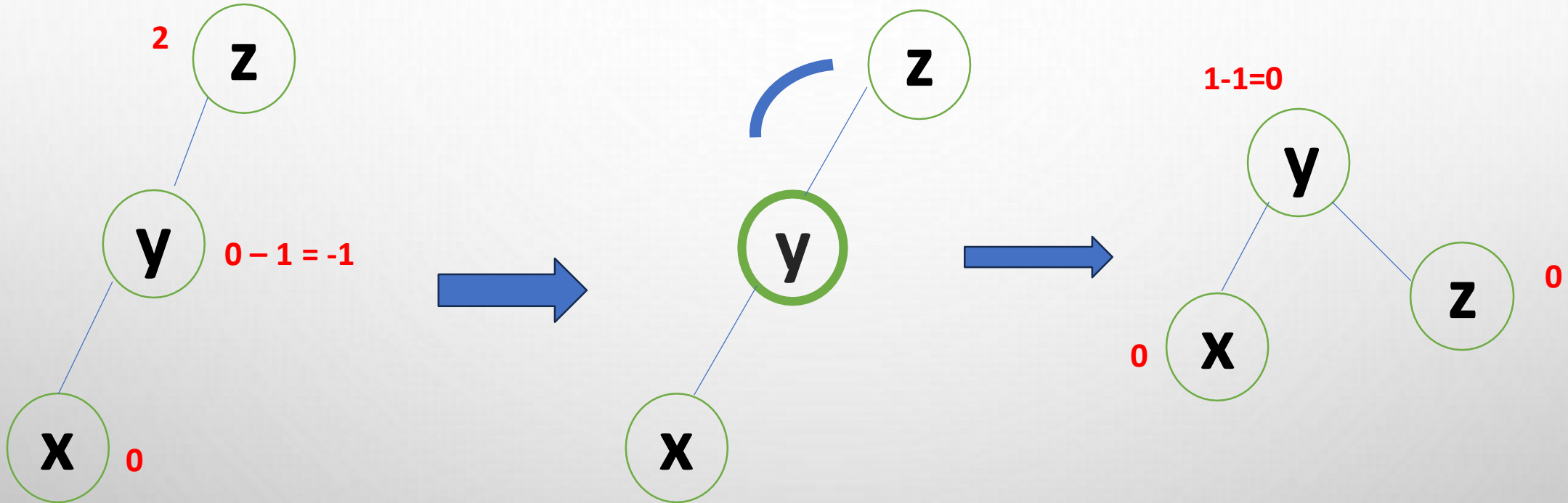
When a node is added into the right subtree of the right subtree, if the tree gets out of balance, we do a single left rotation

- Suppose we have x, y, z, **LEFT ROTATION**



If a node is added to the left subtree of the left subtree, the AVL tree may get out of balance, we do a single right rotation.

- Suppose we have z, y, x **RIGHT ROTATION**

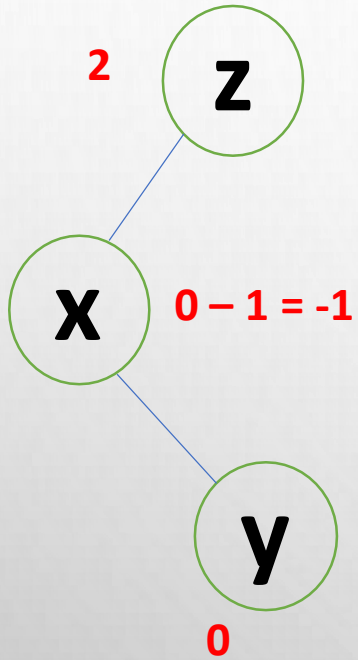


Tree is not balanced

Right Rotation

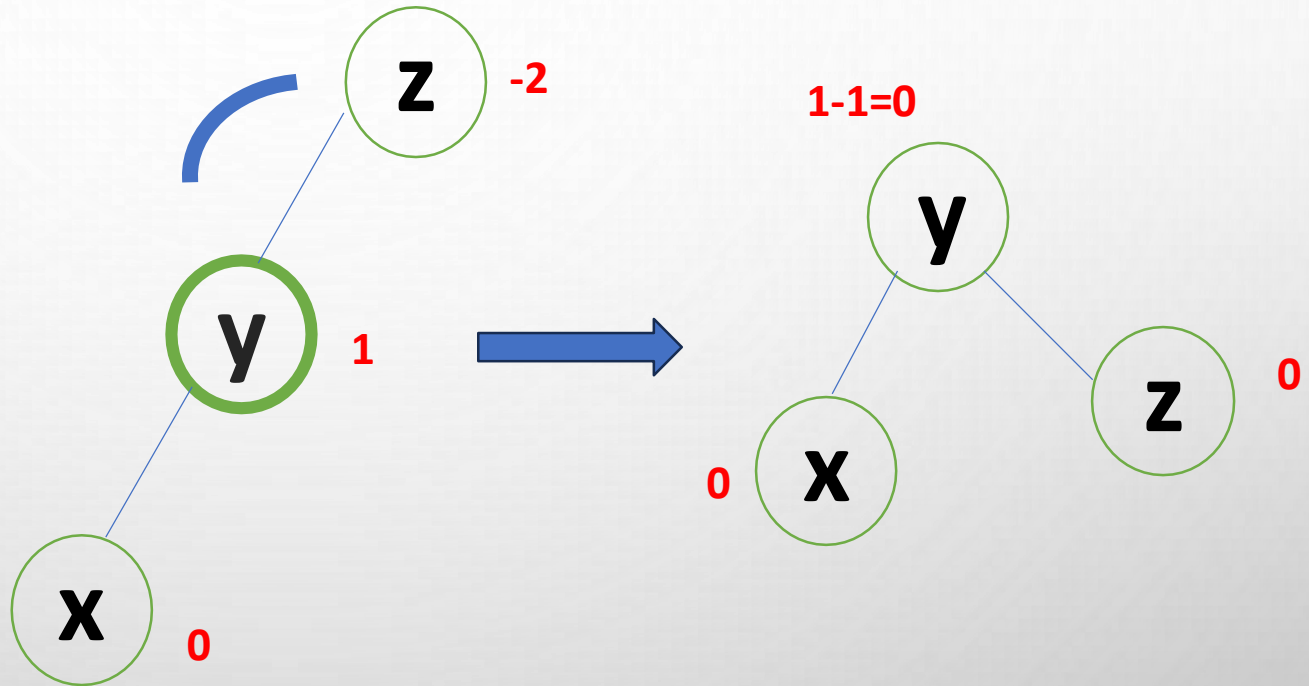
LEFT RIGHT ROTATION

- Suppose we have z, x, y



Tree is not balanced

A left-right rotation is a combination in which first left rotation takes place after that right rotation executes.

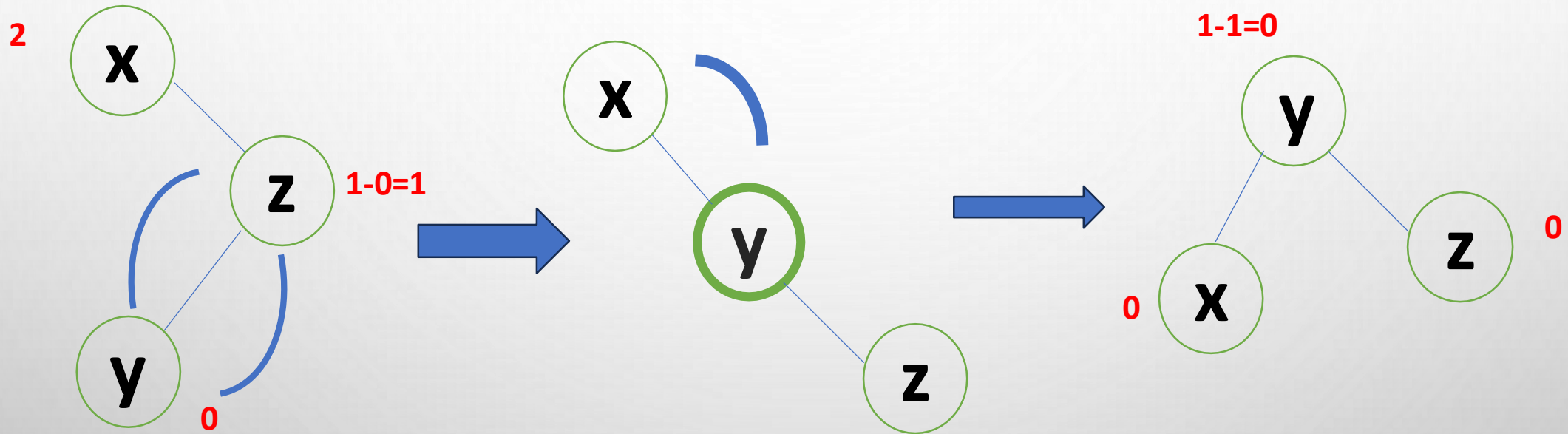


Left Rotation & Right Rotation

RIGHT LEFT ROTATION

- Suppose we have x, z, y

A right-left rotation is a combination in which first right rotation takes place after that left rotation executes.

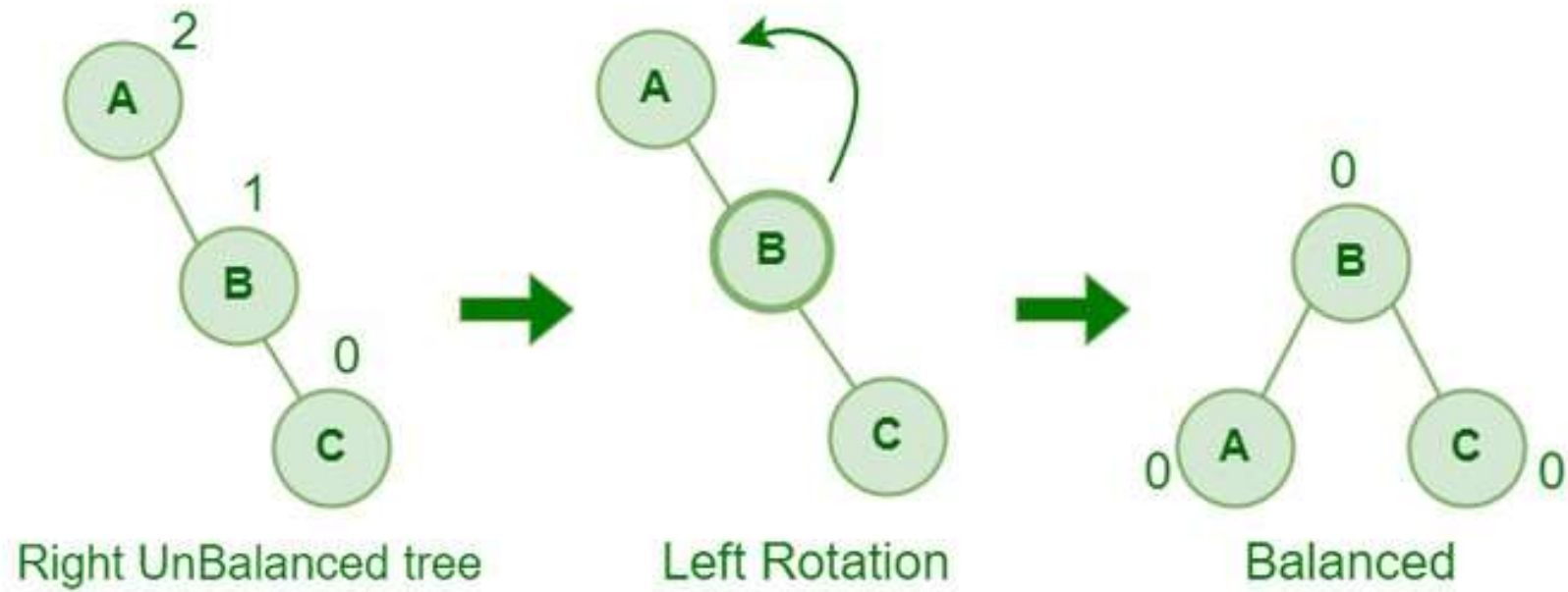


Tree is not balanced

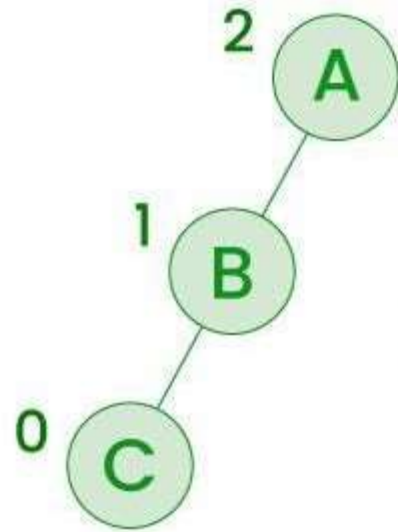
Right Rotation

EXAMPLE 2

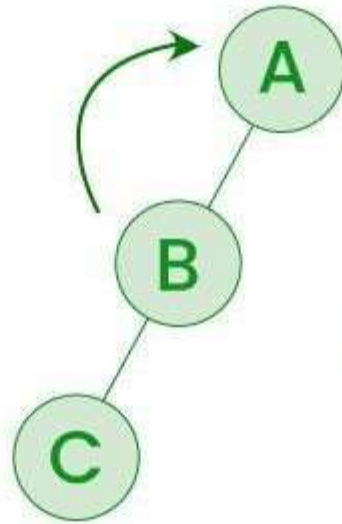
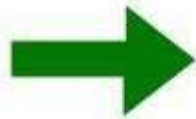
- Suppose you have A , B , C create a balanced AVL with the help of different Rotations



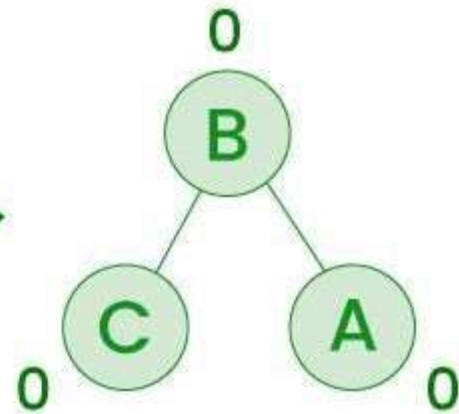
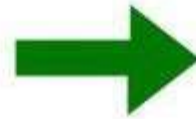
Left Rotation



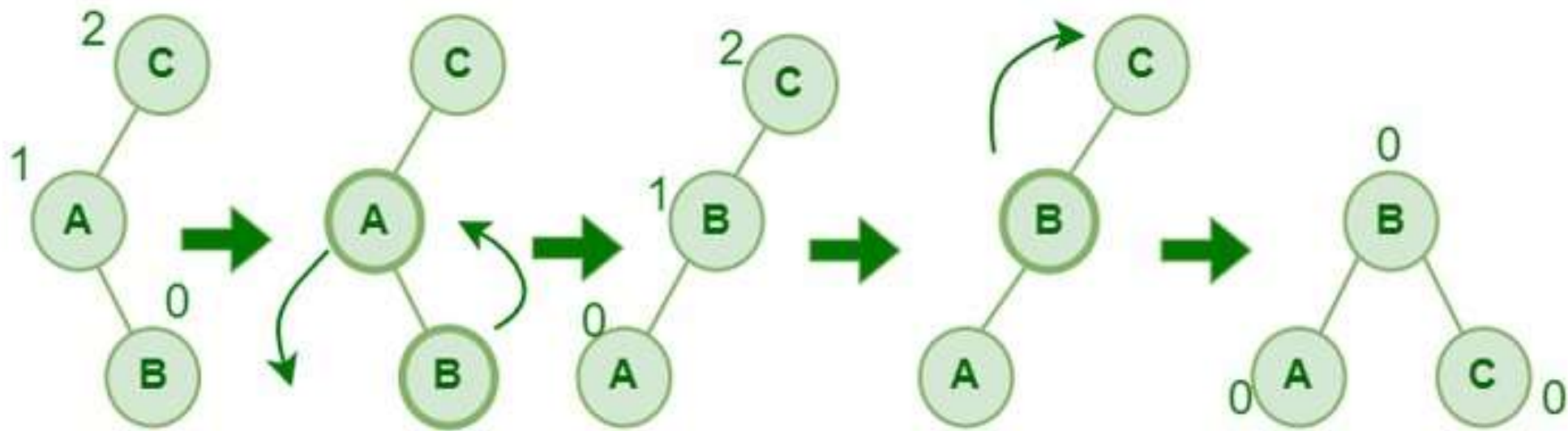
Left UnBalanced tree



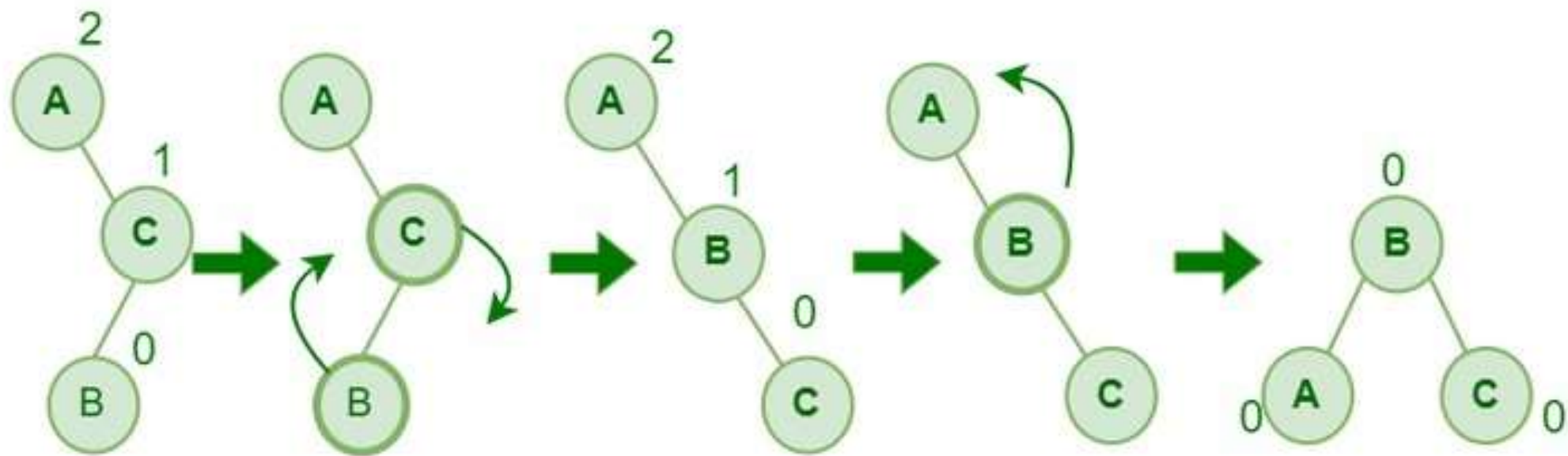
Right Rotation



Balanced



Left-Right Rotation



Right-Left Rotation

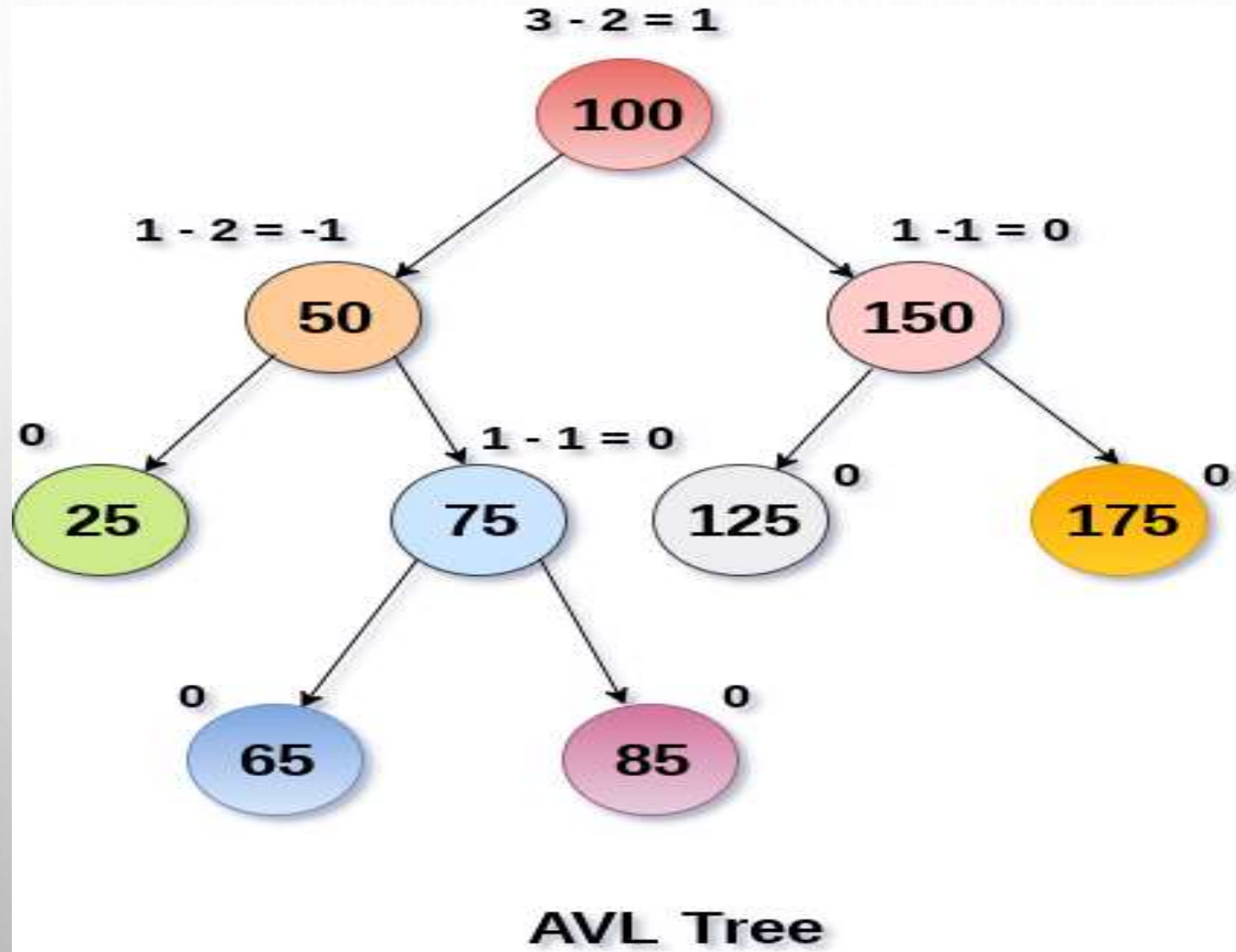
AVL TREE OPERATIONS

- Insertion
- Deletion
- Searching

APPLICATIONS OF AVL TREES:

- **Databases and File Systems:** Used to implement B-trees and other balanced tree structures.
- **Symbol Tables:** Used to store and retrieve symbols in compilers and interpreters.
- **Network Routing:** Used in routing algorithms to efficiently store and retrieve routing information

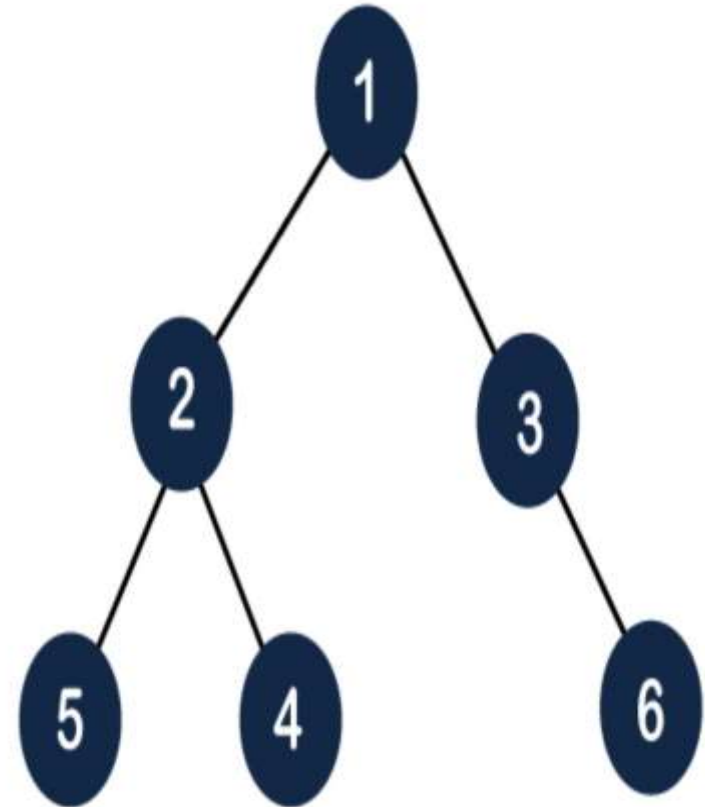
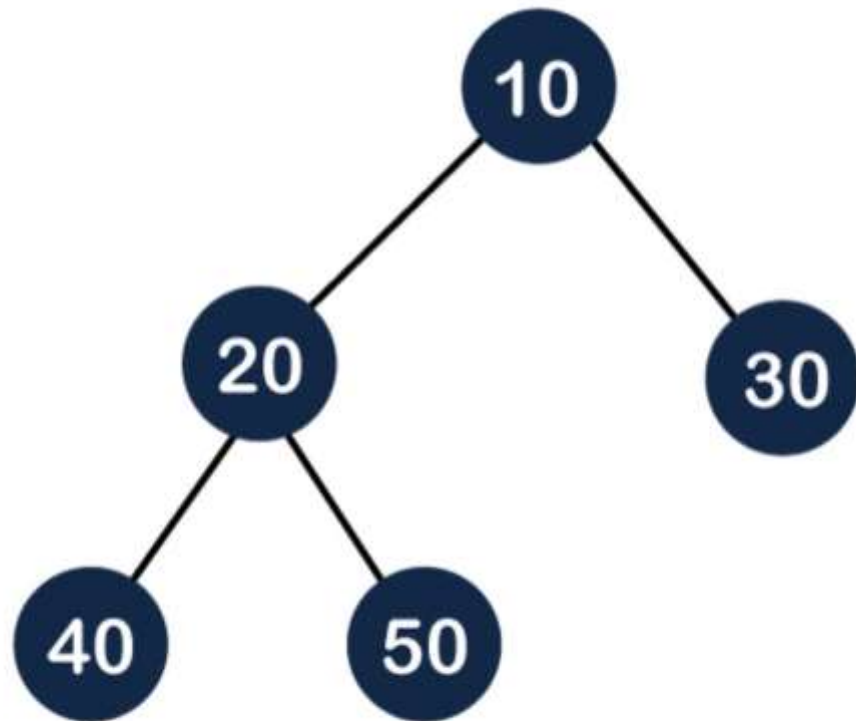
BALANCED AVL TREE



HEAP TREE

- HEAP IS A COMPLETE BINARY TREE, AND THE BINARY TREE IS A TREE IN WHICH THE NODE CAN HAVE UTMOST TWO CHILDREN.
- COMPLETE BINARY TREE IS A BINARY TREE IN WHICH ALL THE LEVELS EXCEPT THE LAST LEVEL, I.E., LEAF NODE SHOULD BE COMPLETELY FILLED, AND ALL THE NODES SHOULD BE LEFT-JUSTIFIED.

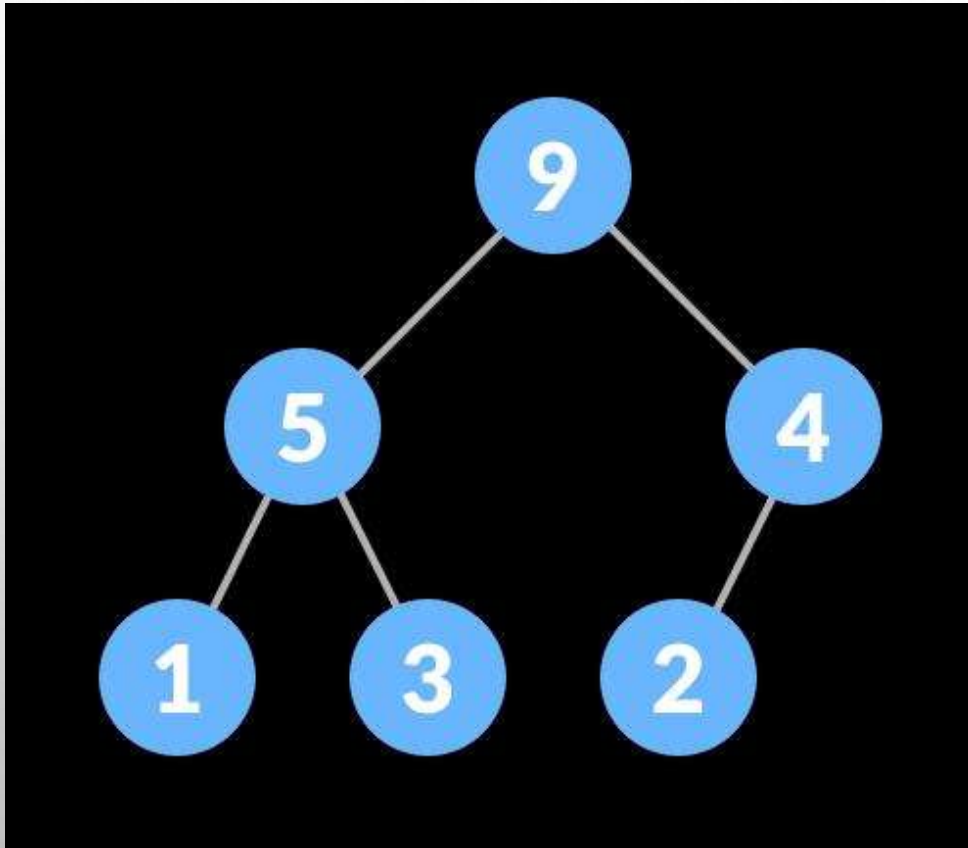
HEAP TREE



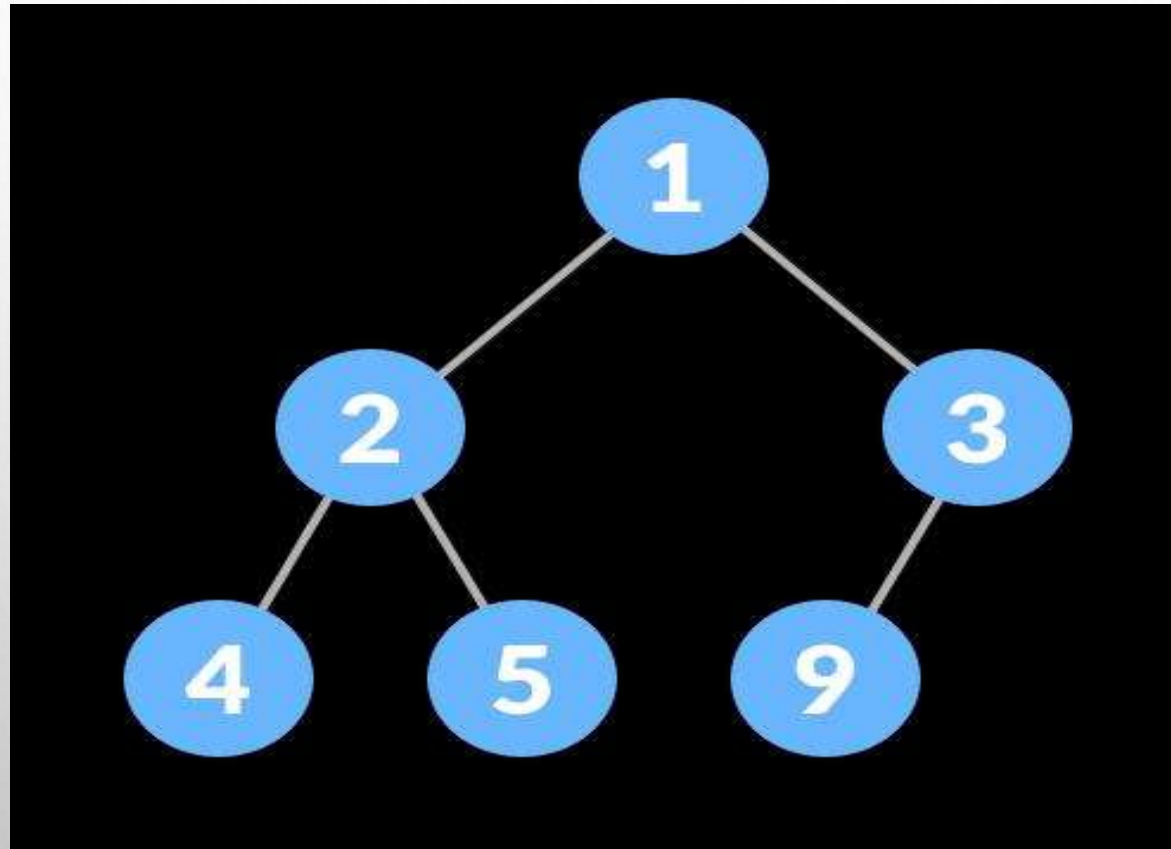
HEAP TREE

- HEAPS CAN BE OF TWO TYPES:
- **MAX-HEAP**: IN A MAX-HEAP THE KEY PRESENT AT THE ROOT NODE MUST BE GREATEST AMONG THE KEYS PRESENT AT ALL OF IT'S CHILDREN. THE SAME PROPERTY MUST BE RECURSIVELY TRUE FOR ALL SUB-TREES IN THAT BINARY TREE.
- **MIN-HEAP**: IN A MIN-HEAP THE KEY PRESENT AT THE ROOT NODE MUST BE MINIMUM AMONG THE KEYS PRESENT AT ALL OF IT'S CHILDREN. THE SAME PROPERTY MUST BE RECURSIVELY TRUE FOR ALL SUB-TREES IN THAT BINARY TREE.

MAX HEAP



MIN HEAP



HEAP TREE

- HEAPIFY IS THE PROCESS OF CREATING A HEAP DATA STRUCTURE FROM A BINARY TREE.

INSERTION IN THE HEAP TREE

44, 33, 77, 11, 55, 88, 66

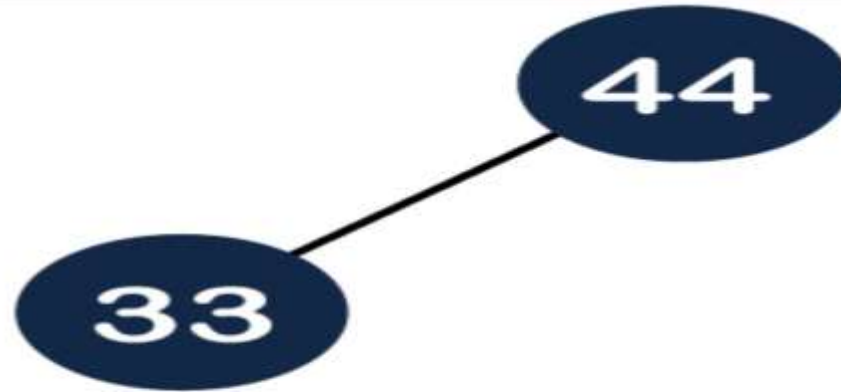
Step 1: First we add the 44 element in the tree as shown below:



Step 2: The next element is 33. As we know that insertion in the binary tree always starts from the left side so 44 will be added at the left of 33 as shown below:

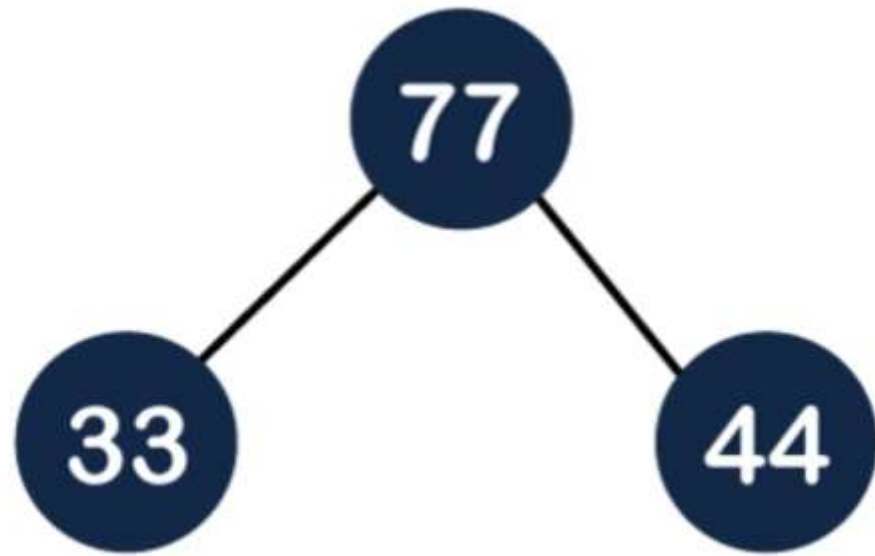
INSERTION IN THE HEAP TREE

44, 33, 77, 11 55 88 66



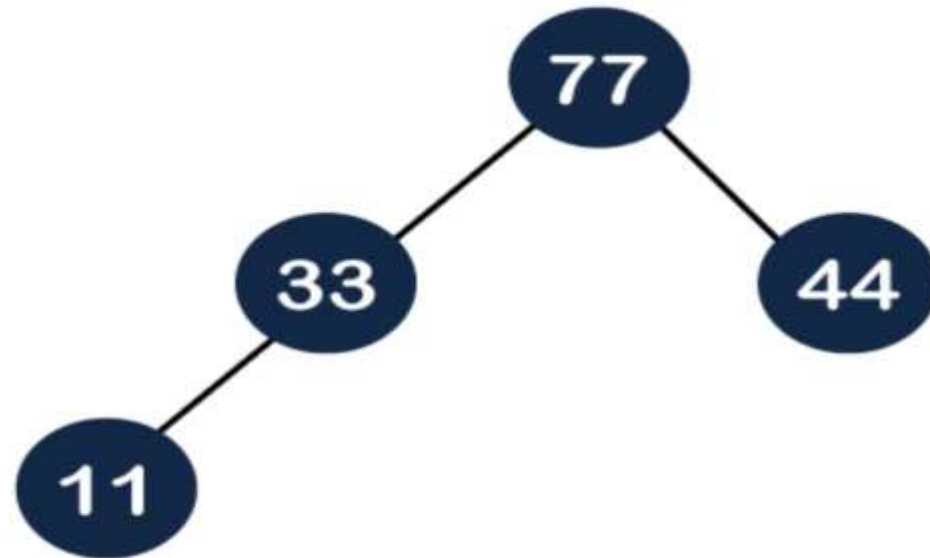
Step 3: The next element is 77 and it will be added to the right of the 44 as shown below:



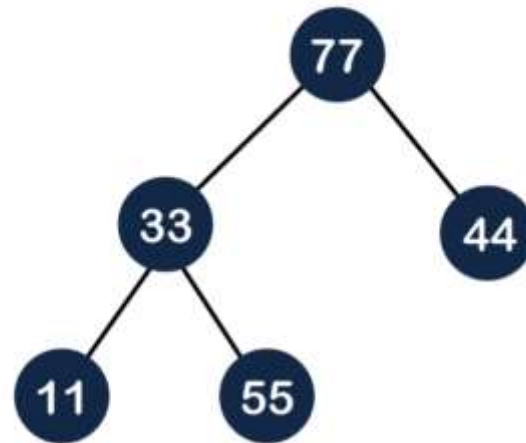


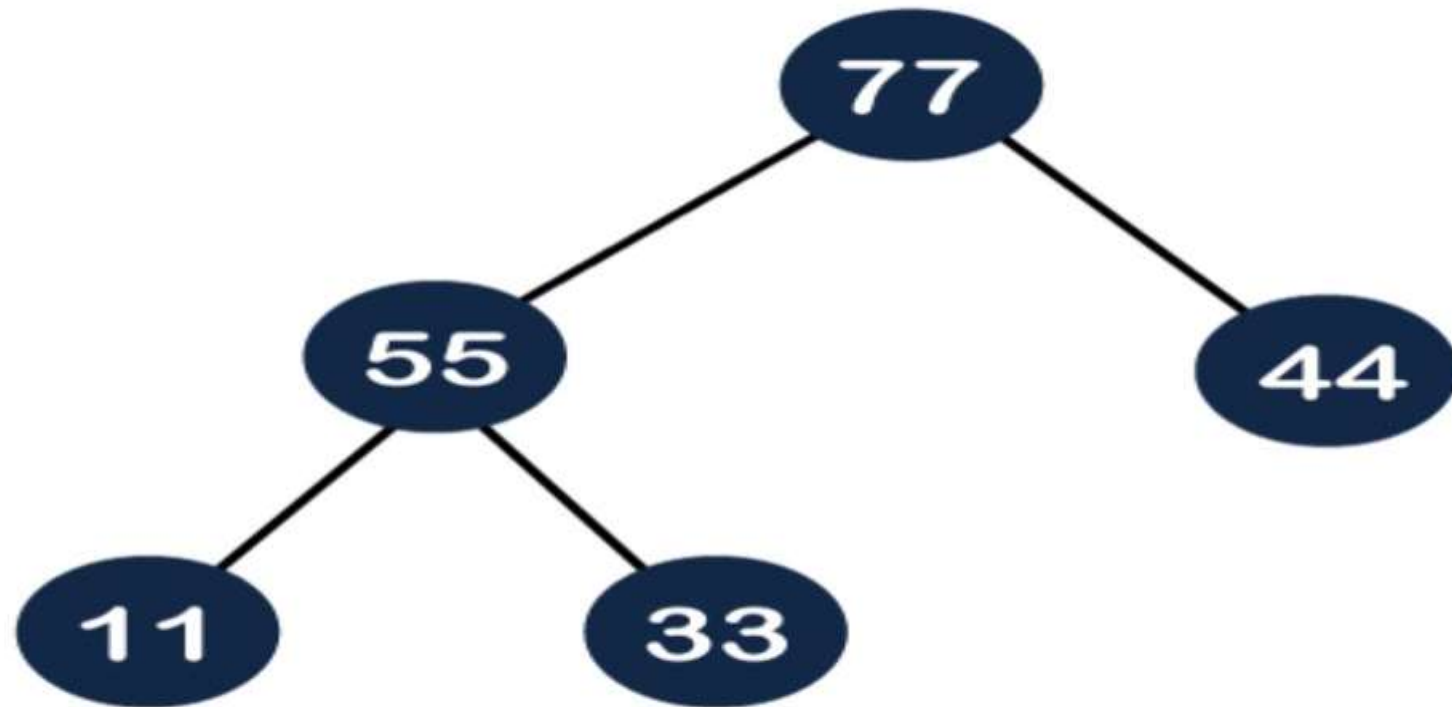
Q: 4: The left child of 77 is 33. The right child of 77 is 44.

Step 4: The next element is 11. The node 11 is added to the left of 33 as shown below:



Step 5: The next element is 55. To make it a complete binary tree, we will add the node 55 to the right of 33 as shown below:





Step 6: The next element is 88. The left subtree is completed so we will add 88 to the left of 44 as shown below:

